나를 바꾸는 C++1

작성자: 오희성[Heesung Oh]

서문 1
1. 왜 **C++**인가**?** 1
2. 소프트웨어 공학 관점에서의 **C++** 1
3. C++을 배우는 방법 2
1부**. C++** 프로그래밍의 기본과 확장 3
00. 개발 환경 준비 3
0.1 Visual Studio 설치 3
0.2 Hello world 출력 3
0.2.1 프로젝트 생성 3
0.2.2 소스 코드 작성 5
0.2.3 빌드(Build) 5
0.2.4 프로그램 실행 5
0.3 C++ 버전 설정 6
01. C++ 언어의 특징과 **C**에서 **C++**로 **8**
1.1 C++ 언어의 특징 8
1.1.1 절차적 프로그래밍과 우수한 성능 8
1.1.2 객체지향 프로그래밍과 유지보수 9
1.1.3 제네릭 프로그래밍과 표준 라이브러리 10
1.1.4 컴파일러의 추론과 자동 처리 10
1.1.5 정적 타입과 컴파일 타임 처리 11
1.1.6 자원 관리와 RAII 11
1.1.7 여러 분야에서 사용되는 C++ 12
1.2 C와 C++의 관계 12
1.2.1 타입을 이용한 안전한 입출력 12
1.2.2 C 문법의 계승 13
1.2.3 C와 C++의 차이 14
1.2.4 C++의 주요 키워드 14
1.2.5 C++을 C처럼만 사용할 때의 한계 15
1.3 C++의 기본형 타입 16
1.3.1 기본형 타입의 분류 16
1.3.2 정수형과 리터럴 접미사 16
1.3.3 논리형(bool) 18
1.3.4 확장된 문자형과 리터럴 접두사 19
1.3.5 부동소수점형의 종류와 접미사 19
1.3.6 void, nullptr, std::nullptr_t 20
1.4 C++에서 추가된 편리한 문법과 기능 21
1.4.1 {new, delete}, {new[], delete[]} 를 이용한 동적 메모리 할당 21
1.4.2 제어문에서 변수 선언하기 23
1.4.3 범위 기반 for문 (Range - for) 25
1.4.4 C++ 난수 라이브러리(C++11) 27
1.4.5 std::sort()를 이용한 정렬 29
1.5 학습 정리 30
1.6 학습 과제 30
02. C++ 프로그래밍의 기본 **32**
2.1 이름 공간 32
2.1.1 이름 공간 정의와 범위 지정 32
2.1.2 이름 공간 다시 열기 32
2.1.3 std 이름 공간과 using 선언 33
2.1.4 using namespace 33
2.1.5 중첩 이름 공간 34
2.1.6 이름 공간 별칭과 전역 범위 35
2.1.7 이름 없는 이름 공간 35
2.1.8 인라인 이름 공간 35
2.2 출력 스트림과 서식 지정 36
2.2.1 cout과 출력 연산자 36
2.2.2 줄 바꿈과 출력 버퍼 36
2.2.3 정수의 진법과 접두사 37
2.2.4 2진수와 비트 단위 출력 37
2.2.5 논리값과 부호 출력 38
2.2.6 필드 폭, 채움 문자와 정렬 38

2.2.7 부동소수점 출력 38
2.2.8 서식 조정자의 적용 범위 39
2.3 입력 스트림과 상태 관리 40
2.3.1 cin과 입력 연산자 40
2.3.2 std::string과 getline() 41
2.3.3 문자 단위 입력 44
2.3.4 공백 처리 44
2.3.5 입력 상태 44
2.3.6 clear()와 ignore() 45
2.3.7 cin과 getline()의 혼합 46
2.4 타입의 크기와 범위 확인 46
2.4.1 데이터 모델과 기본형의 크기 47
2.4.2 sizeof와 CHAR_BIT 47
2.4.3 alignof 48
2.4.4 std::numeric_limits 48
2.4.5 std::size_t와 std::ptrdiff_t 49
2.5 문자형과 문자 인코딩 50
2.5.1 char 계열 50
2.5.2 와이드 문자와 Unicode 코드 단위 50
2.5.3 문자열 배열과 널 문자 51
2.5.4 문자형과 출력 스트림 52
2.6 부동소수점형과 오차 52
2.6.1 실수값의 근사 표현 52
2.6.2 == 비교의 의미 52
2.6.3 절대 오차와 상대 오차 53
2.6.4 머신 입실론(Machine Epsilon) 53
2.6.5 무한대와 NaN 54
2.6.6 정확한 값이 필요한 데이터 54
2.7 초기화와 const 55
2.7.1 초기화와 대입 55
2.7.2 기본 초기화와 값 초기화 55
2.7.3 중괄호 초기화 (List-initialization) 와 축소 변환 (Narrowing Conversion) 56
2.7.4 const와 변경 제한 56
2.7.5 constexpr 57
2.7.6 consteval과 static_assert 58
2.7.7 constinit 58
2.7.8 포인터와 const 58
2.8 타입 표현과 안전성 59
2.8.1 C++ 형 변환 연산자 60
2.8.2 enum class 61
2.8.3 auto 62
2.8.4 decltype 63
2.8.5 using을 이용한 타입 별칭 63
2.9 함수 기능의 확장 64
2.9.1 함수 오버로딩 64
2.9.2 기본 인수 65
2.9.3 인라인 함수 66
2.9.4 함수의 반환 타입 추론 67
2.10 참조 68
2.10.1 참조의 개념 68
2.10.2 값, 주소와 참조에 의한 전달 69
2.10.3 상수 참조(lvalue reference to const) 70
2.10.4 참조 반환 71
2.11 학습 정리 72
2.12 학습 과제 72

서문

1. 왜 C++인가?

C++은 배우기 쉬운 언어가 아니다. 문법의 범위가 넓고, 객체의 수명과 메모리, 타입, 성능까지 개발자가 직접 판단해야 하는 부분도 많다. 그럼에도 C++은 오랫동안 시스템 소프트웨어와 다양한 응용 프로그램 개발에 사용되어 왔다.

C++가 계속 사용되는 이유를 단순히 빠른 언어이기 때문이라고만 설명하기는 어렵다. C++은 서로 다른 세 가지 장점을 하나의 언어 안에서 제공한다.

첫째, C에서 이어진 절차적 프로그래밍과 하드웨어에 가까운 제어를 제공한다. 함수와 포인터, 연속된 메모리, 객체의 배치와 수명을 직접 다룰 수 있으며, 컴파일된 코드는 네이티브 환경에서 실행된다. 이러한 특징은 처리 속도와 메모리 사용을 세밀하게 제어할 수 있는 기반이 된다.

둘째, 클래스와 객체지향 프로그래밍을 지원한다. 프로그램의 규모가 커지면 단순히 함수만 나누는 것으로는 데이터의 변경과 기능 사이의 관계를 관리하기 어려워진다. 객체지향 프로그래밍은 상태와 행위를 하나의 객체로 묶고, 프로그램을 역할과 책임에 따라 나눌 수 있게 한다. 이를 통해 변경 범위를 제한하고, 여러 기능이 연결된 프로그램을 더 체계적으로 관리할 수 있다.

셋째, 방대한 표준 라이브러리를 제공한다. 동적 배열, 연결 리스트, 트리, 해시 테이블과 같은 자료구조를 매번 직접 구현할 필요가 없으며, 검색과 정렬, 변환, 파일 입출력, 시간, 난수, 메모리 관리 등 다양한 기능을 검증된 형태로 사용할 수 있다. 개발자는 자료구조의 원리를 이해하면서도 실제 구현에서는 표준 컨테이너와 알고리즘을 활용해 응용 프로그램의 핵심 기능에 집중할 수 있다.

C++은 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍을 함께 지원하는 언어다. 모든 코드를 반드시 클래스로 작성할 필요도 없고, 성능을 위해 모든 추상화를 포기할 필요도 없다. 문제에 따라 적절한 방법을 선택하고 여러 방식을 조합할 수 있다는 점이 C++의 중요한 특징이다.

이러한 이유로 C++은 운영체제와 시스템 소프트웨어뿐 아니라 그래픽스, 게임 엔진, 시뮬레이션, 로봇, 임베디드 시스템, 금융, 데이터 처리, 네트워크 서버, 데스크톱 응용 프로그램과 개발 도구 등 여러 분야에서 사용되고 있다. 특히 높은 성능과 복잡한 프로그램 구조를 함께 다루어야 하는 응용 분야에서 C++의 장점이 잘 드러난다.

C++은 과거의 언어에 머물러 있지 않다. C++11 이후 이동 의미론, 스마트 포인터, 람다, constexpr, Ranges, Concepts와 같은 기능이 추가되면서 객체의 수명과 소유권을 더 명확하게 표현하고, 표준 라이브러리를 더 안전하고 편리하게 활용할 수 있도록 발전해 왔다. 모던 C++은 기존의 저수준 제어 능력을 유지하면서 더 높은 수준의 추상화를 함께 제공하는 방향으로 계속 변화하고 있다.

2. 소프트웨어 공학 관점에서의 C++

프로그램은 처음 작성할 때보다 기능이 추가되고 수정되는 과정에서 더 복잡해진다. 여러 기능이 하나의 데이터에 의존하고, 객체의 생성과 제거 시점이 서로 얽히면, 한 부분의 변경이 예상하지 못한 곳까지 영향을 미치기도 한다. 따라서 소프트웨어 개발에서는 기능의 구현뿐 아니라 복잡성과 변경 비용을 관리하는 것이 중요하다.

C++은 이러한 문제를 언어의 구조와 직접 연결한다. 클래스는 데이터와 행위를 하나의 책임 단위로 묶고, 접근 제어는 객체의 내부 상태가 임의로 변경되는 것을 제한한다. 상속과 다형성은 공통 인터페이스를 통해 여러 구현을 다룰 수 있게 하고, 템플릿은 타입에 의존하지 않는 일반화된 코드를 만들 수 있게 한다.

또한 C++에서는 객체의 수명과 자원의 소유권이 중요한 설계 요소가 된다. 동적으로 할당한 메모리뿐 아니라 파일, 네트워크 연결, 잠금, 그래픽 자원과 같은 모든 자원에는 획득과 해제 시점이 존재한다. C++의 RAII는 자원을 객체의 생명주기에 연결하여 객체가 생성될 때 자원을 획득하고, 객체가 소멸할 때 자동으로 해제하도록 한다. 스마트포인터는 단순히 포인터를 편리하게 사용하는 도구가 아니라, 누가 객체를 소유하고 언제 제거할지를 코드로 표현하는 수단이다.

C++은 추상화의 비용도 비교적 직접적으로 드러낸다. 가상 함수는 런타임 다형성을 제공하지만 간접 호출이 발생할 수 있고, 동적 메모리 할당은 유연성을 제공하지만 할당과 해제 비용을 가진다. 템플릿은 런타임 비용을 줄일 수 있지만 컴파일 시간과 코드 크기에 영향을 줄 수 있다. 개발자는 각 기능의 장점만이 아니라 그 기능이 요구하는 비용도 함께 고려해야 한다.

이러한 관점은 C++의 제로 오버헤드 원칙과도 연결된다. 사용하지 않는 기능에는 비용을 지불하지 않으며, 추상화를 사용하더라도 직접 작성한 저수준 코드와 비슷한 수준의 성능을 목표로 한다. C++에서 추상화와 성능은 서로 반대되는 개념이 아니라, 함께 고려해야 하는 설계 요소다.

그러나 클래스와 상속을 사용한다고 해서 프로그램의 유지보수성이 자동으로 좋아지는 것은 아니다. 책임이 지나치게 많은 클래스, 깊은 상속 구조, 불필요한 추상화는 오히려 코드를 복잡하게 만든다. 중요한 것은 기능을 많이 사용하는 것이 아니라, 상태와 책임, 수명과 소유권, 객체 사이의 관계를 명확하게 표현하는 것이다.

C와 C++을 오랫동안 사용하면서 개인적으로 느낀 차이도 여기에 있다. C를 사용하면서는 메모리와 주소, 데이터의 배치, 포인터와 함수 호출처럼 컴퓨터가 실제로 어떻게 동작하는지를 더 깊이 생각하게 되었다. 반면 C++을 사용하면서는 프로그램을 어떤 단위로 나누고, 객체의 수명과 소유권을 어떻게 표현하며, 변경의 영향을 어디까지 제한할 것인지를 더 많이 고민하게 되었다. C++은 저수준 제어 능력을 유지하면서 그 위에 프로그램의 구조와 설계를 함께 생각하도록 요구하는 언어라는 인상을 받았다.

모던 C++에서 특히 크게 느낀 변화 중 하나는 추론이다. auto, 템플릿 인수 추론, 람다, Concepts처럼 형태는 서로 다르지만, 컴파일러가 이미 알 수 있는 정보를 프로그래머가 반복해서 기술하지 않도록 하는 방향이 공통적으로 보인다. 프로그래머는 모든 세부를 직접 지정하기보다 의도와 조건을 명확하게 표현하고, 기계적으로 판단할 수 있는 부분은 컴파일러에 맡길 수 있다.

자원 관리에서도 비슷한 변화가 나타난다. 메모리를 언제 해제할 것인지 직접 추적하는 문제는 RAII와 스마트포인터를 통해 누가 자원을 소유하고 그 수명이 어디까지인지를 표현하는 문제로 바뀐다. 구현 기법을 하나씩 기억하고 관리하는 데 머무르기보다 프로그램의 구조와 책임, 자원의 수명과 비용을 더 많이 생각하게 만드는 방향으로 C++이 발전해 왔다고 느낀다.

C++의 기능은 매우 방대하며 실제 개발에서도 모든 기능을 동일하게 사용하지 않는다. 이 책도 언어가 제공하는 모든 기능을 나열하는 것을 목표로 하지 않는다. 현재의 C++을 이해하고 프로그램을 설계하는 데 필요한 개념을 중심으로 선택하고, 각 기능이 객체의 상태와 생명주기, 자원 관리, 타입 안전성, 표준 라이브러리와 어떤 관계를 가지는지 함께 살펴본다.

3. C++을 배우는 방법

이 책은 C언어의 기본 문법을 학습한 독자를 대상으로 한다. 변수와 연산자, 조건문과 반복문, 함수, 배열, 구조체와 포인터를 처음부터 다시 설명하지 않는다. 대신 C를 사용하던 방식에서 C++의 객체 모델과 표준 라이브러리를 활용하는 방식으로 어떻게 전환해야 하는지를 중심으로 다룬다.

먼저 C와 C++의 차이를 살펴보고 C++에서 추가된 기본형 타입, 이름 공간, 스트림 입출력, 함수 오버로딩, 기본 인수와 참조를 정리한다. 이후 구조체에 데이터만 저장하던 방식에서 벗어나, 상태와 행위를 함께 관리하는 클래스와 객체의 구조로 넘어간다.

객체지향 프로그래밍을 학습할 때는 클래스 문법만 익혀서는 충분하지 않다. 객체가 언제 생성되고 소멸하는지, 복사할 때 어떤 의미를 가지는지, 자원을 다른 객체로 이동할 때 소유권이 어떻게 변하는지를 이해해야 한다. 생성자와 소멸자, 복사와 이동, RAII와 스마트 포인터는 서로 분리된 문법이 아니라 객체의 생명주기를 표현하는 하나의 흐름이다.

상속과 다형성도 가상 함수의 사용법만을 배우는 데 그치지 않는다. 객체 사이의 관계를 어떻게 정할 것인지, 상속과 구성 중 무엇을 선택할 것인지, 인터페이스를 어느 범위까지 제공할 것인지 판단해야 한다. SOLID 원칙은 별도의 이론으로 외우기보다 이러한 객체지향 구조를 점검하는 기준으로 활용한다.

템플릿과 제네릭 프로그래밍을 익힌 뒤에는 표준 라이브러리로 범위를 확장한다. 컨테이너의 함수 이름을 외우는 것보다 각 컨테이너가 데이터를 어떤 방식으로 저장하며, 접근과 삽입, 삭제에 어떤 비용이 필요한지를 이해하는 것이 중요하다. 반복자, 알고리즘, 함수 객체와 람다는 컨테이너에 저장된 데이터를 일관된 방식으로 처리하기 위한 구조로 연결해서 학습한다.

모던 C++ 기능은 별도의 문법 묶음으로 분리하지 않는다. auto는 타입을 표현하는 위치에서, constexpr은 초기화와 컴파일 타임 평가에서, 이동 의미론은 복사와 자원 이전에서, 스마트 포인터는 소유권에서, 람다와 Ranges는 알고리즘을 다루는 과정에서 설명한다. 새로운 문법은 그 기능이 필요한 상황과 함께 학습해야 의미를 제대로 이해할 수 있기 때문이다.

마지막으로 대표적인 디자인 패턴을 통해 앞에서 배운 내용을 실제 객체 구조로 연결한다. 싱글턴, 팩토리 메서드, 추상 팩토리, 어댑터, 컴포지트, 퍼사드, 옵저버와 커맨드 패턴을 다루며, 패턴의 클래스 구조를 암기하기보다 어떤 문제를 해결하기 위해 사용되는지 살펴본다. 특히 커맨드 패턴에서는 요청을 객체로 표현하고 실행 기록을 저장하여 Undo와 Redo를 구성하는 과정을 통해 객체지향, 다형성, 스마트 포인터와 표준 컨테이너가 실제 구조에서 어떻게 결합되는지 확인한다.

C++을 배우는 과정은 단순히 새로운 문법을 추가하는 과정이 아니다. 프로그램을 구성하고 자원과 비용을 바라보는 방식을 바꾸는 과정이다. C++에 익숙해질수록 문법 자체보다 문제를 분석하고, 책임을 나누고, 객체와 시스템의 관계를 설계하는 데 더 많은 관심을 두게 된다. 이러한 변화는 작은 코드의 구현에서 시작해 프로그램의 구조를 설계하고, 여러 구성 요소의 관계와 변경 영향을 판단하는 능력으로 이어진다. 결국 C++ 학습은 단순히 언어 하나를 익히는 데서 끝나지 않고, 구현 중심의 개발자에서 분석과 설계를 수행하며 시스템 전체의 구조를 바라볼 수 있는 개발자로 성장하는 과정이 된다.

1부. C++ 프로그래밍의 기본과 확장

00. 개발 환경 준비

C++ 프로그램을 작성하고 실행하려면 C++ 소스 코드를 기계어로 변환하는 컴파일러가 필요하다. 대표적인 C++ 컴파일러로는 Microsoft의 MSVC, GCC와 Clang이 있다. 컴파일러와 함께 코드 편집기, 디버거와 프로젝트 관리 기능을 제공하는 통합 개발 환경을 사용하면 프로그램을 보다 편리하게 개발할 수 있다.

본 교재에서는 Windows 환경에서 널리 사용되는 Microsoft Visual Studio와 MSVC를 이용해 C++ 프로그램을 작성하고 실행한다.

0.1 Visual Studio 설치

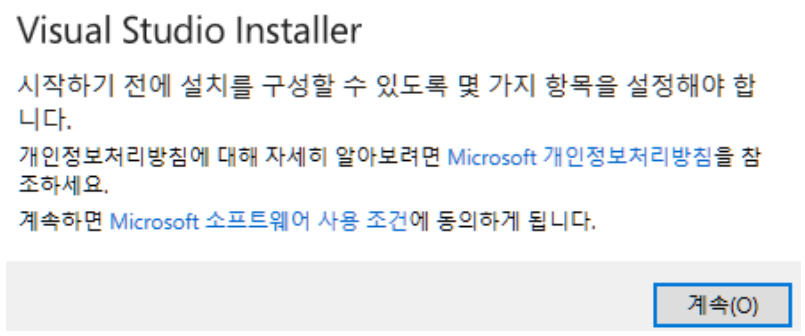
Visual Studio는 코드 편집기, MSVC 컴파일러, 디버거와 프로젝트 관리 도구 등이 통합된 개발 환경이다. Visual Studio Community는 개인 개발, 수업과 교육, 학술 연구와 오픈소스 개발 등에 무료로 사용할 수 있다. 조직에서 그 밖의 목적으로 사용하는 경우에는 별도의 라이선스 조건을 확인해야 한다.

현재 공식 다운로드 페이지에서는 Visual Studio Community 2026을 제공하고 있다. 본 교재는 C++20 학습 환경과 화면 구성을 통일하기 위해 Visual Studio 2022 Community를 기준으로 작성하였다. Visual Studio 버전에 따라 화면 구성과 메뉴 이름에 약간의 차이가 있을 수 있지만, 프로젝트 생성과 C++ 언어 표준 설정 방법은 거의 같다.

Visual Studio Community는 Microsoft의 공식 Visual Studio 다운로드 페이지에서 내려받을 수 있다.

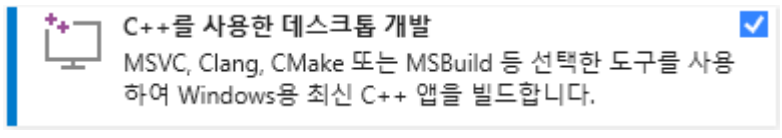
- 최신 Visual Studio Community 다운로드: https://visualstudio.microsoft.com/ko/free-developer-offers/
- Visual Studio 2026 Community 다운로드: https://aka.ms/vs/18/Stable/vs_community.exe
- **Visual Studio 2022 Community 다운로드: https://aka.ms/vs/17/release/vs_community.exe**

<Visual Studio Installer>



Visual Studio 설치 관리자를 실행한 뒤 워크로드 목록에서 C++를 사용한 데스크톱 개발을 선택한다. 이 워크로드를 설치하면 C/C++ 개발에 필요한 MSVC 도구 집합, 표준 라이브러리와 기본 개발 도구가 함께 설치된다.

<C++를 사용한 데스크톱 개발 워크로드 선택>



구성 요소를 선택한 뒤 설치 버튼을 클릭한다. 설치가 완료되면 Visual Studio에서 C++ 프로젝트를 생성하고 프로그램을 작성할 수 있다.

C++11 이후에는 대체로 3년 주기로 새로운 C++ 표준이 발표되고 있다. 그러나 실제 프로젝트에서는 기존 코드, 라이브러리와 개발 도구의 호환성 때문에 최신 버전을 즉시 도입하지 않을 수도 있다. 이 교재는 C++20을 기준으로 하며, C++20 이후에 추가된 기능은 필요한 경우 별도로 버전을 표시한다.

0.2 Hello world 출력

C++ 프로그램의 기본적인 작성 과정을 확인하기 위해 화면에 Hello, World!를 출력하는 프로그램을 만들어 보자.

Hello, World는 브라이언 커니핸이 작성한 1972년 B 언어 튜토리얼에서 사용되었으며, 1974년의 『Programming in C: A Tutorial』과 1978년의 『The C Programming Language』를 통해 널리 알려졌다.

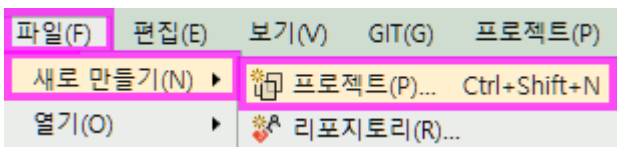
Visual Studio를 이용한 기본적인 프로그램 개발 과정은 다음과 같다.

```
프로젝트 생성 → 코드 작성 → 빌드 → 실행
```

0.2.1 프로젝트 생성

Visual Studio를 실행한 뒤 메뉴에서 해당 항목을 선택한다.

[파일] → [새로 만들기] → [프로젝트]



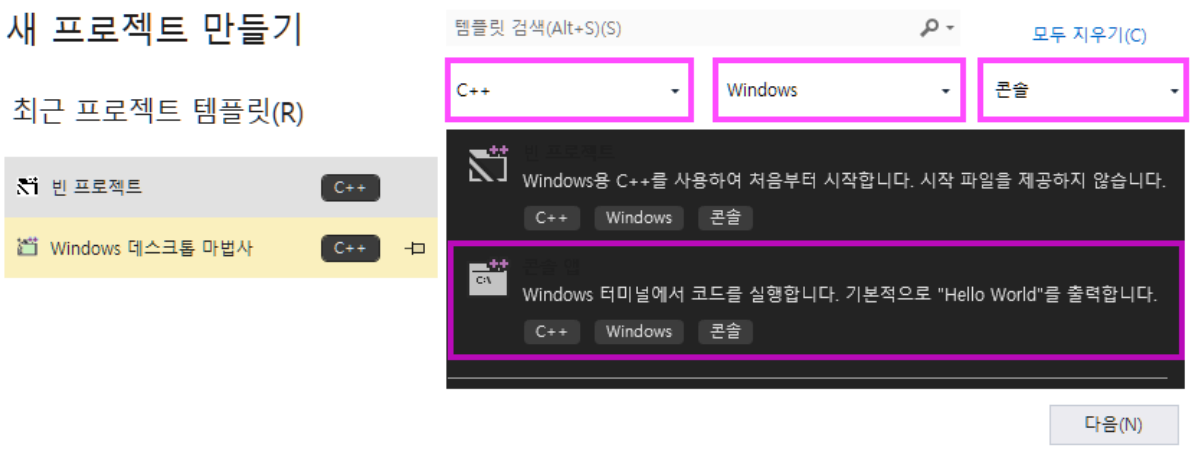
새 프로젝트 만들기 창에서 콘솔 앱을 선택한다. 찾기 어려운 경우 필터는 언어, 플랫폼과 프로젝트 종류로 지정한다.

- 언어(Language): C++
- 플랫폼(Platform): Windows
- 프로젝트 종류: 콘솔(Console)

C++를 사용한 데스크톱 개발 워크로드가 설치되어 있어야 C++ 콘솔 앱 템플릿이 표시된다.

콘솔 프로그램은 그래픽 창을 사용하지 않고 문자 기반의 입력과 출력을 처리하는 프로그램이다. 이 교재의 기본 예제는 대부분 콘솔 프로그램으로 작성한다.

<콘솔 앱 선택>



새 프로젝트 구성 화면에서 프로젝트 이름과 저장 위치를 지정한다.

- 프로젝트 이름(Project name): 원하는 영문 이름 입력. ex) myHelloWorld
- 위치(Location): 미리 준비한 작업 폴더 사용 권장
- 솔루션 및 프로젝트를 같은 디렉토리에 배치: 체크 권장
  (솔루션 파일(.sln)과 프로젝트 파일(.vcxproj)이 한 폴더에 있어 관리가 편리하다.)

C++을 학습하는 단계에서 프로젝트를 이동하고 외부 개발 도구와 함께 사용할 수 있도록 프로젝트 이름과 경로에는 영문자, 숫자, 그리고 밑줄 "_" 을 사용한다. 한글이나 공백을 Visual Studio에서 사용할 수 있지만, 일부 외부 도구나 빌드 스크립트와 함께 사용할 때 문제가 발생할 수 있으므로 사용하지 않는다.

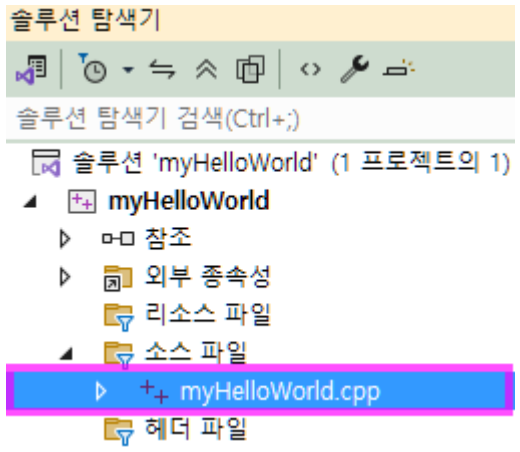
모든 설정을 확인한 뒤 [만들기] 버튼을 클릭한다.

<프로젝트 생성 창>

0.2.2 소스 코드 작성

프로젝트가 생성되면 기본 C++ 소스 파일이 함께 만들어진다. 파일 이름은 프로젝트 템플릿과 Visual Studio 버전에 따라 HelloWorld.cpp 또는 프로젝트 이름과 같은 형태로 생성될 수 있다.

Visual Studio의 MSVC는 기본적으로 .c 파일을 C 언어로, .cpp 파일을 C++ 언어로 컴파일한다. 생성된 소스코드 (ex: myHelloWorld.cpp)를 열어 본다.



<자동으로 생성된 기본 코드>

```cpp
#include <iostream>

int main()
{
    std::cout << "Hello World!\n";
}
```

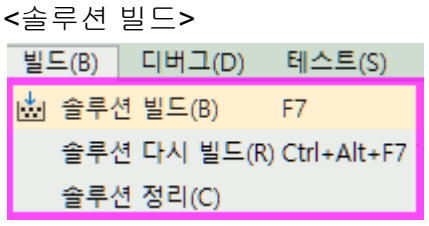
0.2.3 빌드(Build)

작성한 소스 코드는 컴파일과 링크 과정을 거쳐 실행 파일로 만들어진다. 이러한 전체 과정을 빌드라고 한다.

메뉴에서 솔루션 빌드를 선택한다.

```
[빌드] → [솔루션 빌드]
```

<솔루션 빌드>



단축키를 활용하면 좀더 편리하다. 빌드 관련 단축키들은 작업할 때 자주 사용되므로, 꼭 기억하기 바란다.

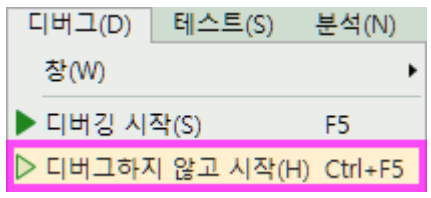
```
Ctrl + Shift + B
```

출력 창에 빌드 결과가 나오는데 문제 없다면 1성공, 0 실패 로그가 출력 될 것이다.

```
1>myHelloWorld.cpp
1>myHelloWorld.vcxproj -> xxx\myHelloWorld\x64\Debug\myHelloWorld.exe
========== 모두 다시 빌드: 1 성공, 0 실패, 0 건너뛰기 ==========
```

0.2.4 프로그램 실행

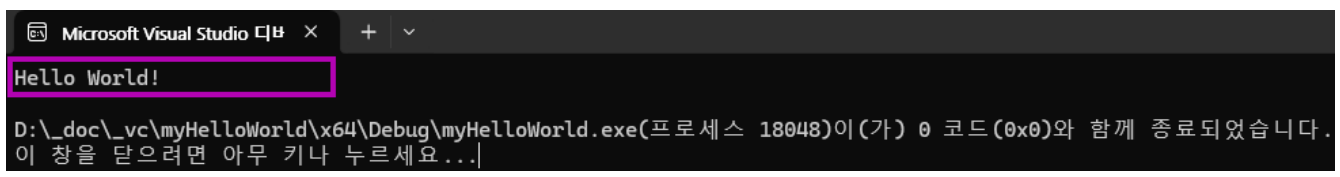
빌드가 완료되면 메뉴에서 "디버그하지 않고 시작" 을 선택한다.



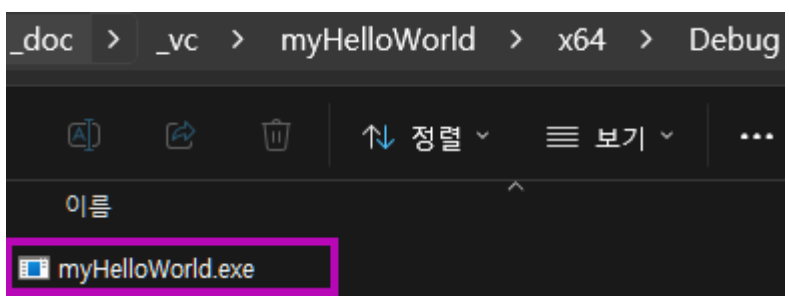

```
[디버그] → [디버그하지 않고 시작]

단축키: Ctrl + F5
```

콘솔 창에 Hello World! 가 출력되는지 확인한다.



실행 파일이 생성된 경로가면 myHelloWorld.exe를 확인 할 수 있다. 이것을 도스 창에서 열고 실행하면 앞서 보인 동일한 결과를 확인할 수 있다.

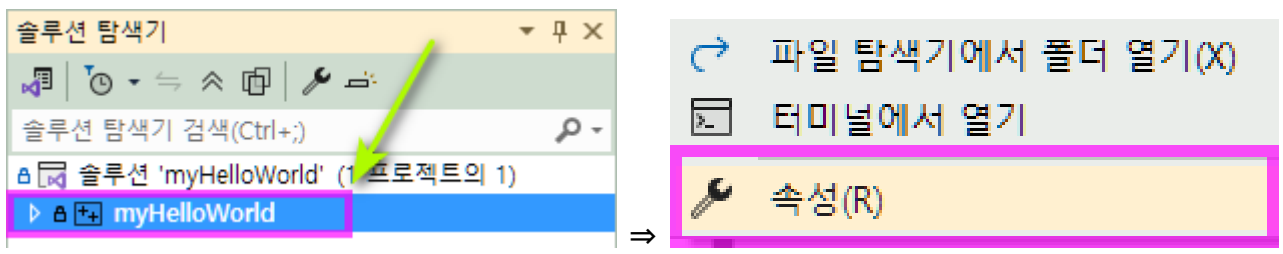


0.3 C++ 버전 설정

MSVC 2022는 별도의 언어 표준을 지정하지 않으면 기본적으로 C++14 모드를 사용한다. C++20 모드를 사용하기 위해서, 프로젝트의 C++ 언어 표준을 변경해야 한다.

C++ 언어 표준 변경은 먼저, 솔루션 탐색기에서 솔루션에 포함되어 있는 프로젝트를 선택한 뒤 마우스 오른쪽 버튼을 눌러 속성을 선택한다. 컨텍스트 메뉴의 하단에 있는 속성을 선택한다.

<프로젝트 선택>

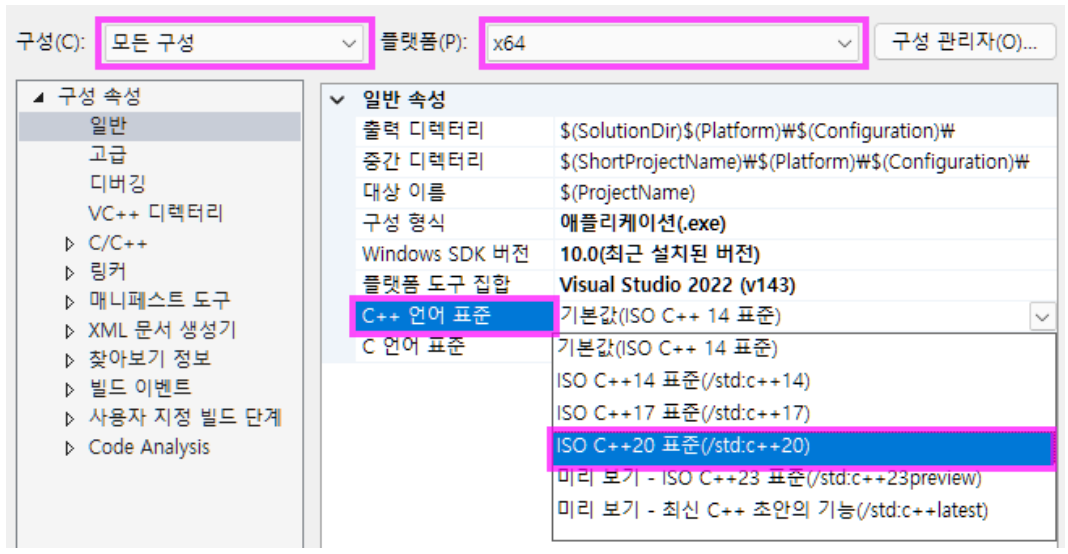


속성페이지의 구성을 모든 구성으로 변경한다. 플랫폼 32, 64 비트까지 적용할 것이면 여기도 모든 구성을 선택한다.

C++ 언어 표준에서 ISO C++20 표준(/std:c++20) 을 선택한다.

```
구성: 모든 구성 플랫폼: 모든 플랫폼
구성 속성 → C/C++ → 언어 → C++ 언어 표준 → ISO C++20 표준(/std:c++20)
```

<C++20 언어 표준 설정>



모든 구성을 선택하면 Debug와 Release 구성에 같은 설정이 적용된다. 모든 플랫폼을 선택하면 Win32와 x64 등의 플랫폼에도 같은 설정이 적용된다.

/std:c++latest는 컴파일러와 표준 라이브러리에 현재 구현된 차기 표준 초안의 기능까지 활성화한다. 그러나 선택한 옵션만으로 앞으로 제안된 모든 기능을 사용할 수 있는 것은 아니며, 사용할 수 있는 기능은 Visual Studio와 MSVC 도구 집합의 버전에 따라 달라진다.

본 교재의 예제는 C++20에 맞춰서 작성되었기 때문에, 학습자는 ISO C++20 표준(/std:c++20) 이상을 설정해서 사용한다.

C++23 이후 최신 기능 사용

C++23 이후에 추가된 기능을 사용하려면 먼저 Visual Studio Installer를 실행하여 Visual Studio 2022와 C++ 개발 도구를 최신 버전으로 업데이트한다.

다음으로 프로젝트 속성의 C++ 언어 표준에서 **미리 보기 최신 C++ 초안의 기능(/std:c++latest)**을 선택한다.

```
프로젝트 속성 → 구성 속성 → C/C++ → 언어
 → C++ 언어 표준 → 미리 보기 최신 C++ 초안의
기능(/std:c++latest)
```

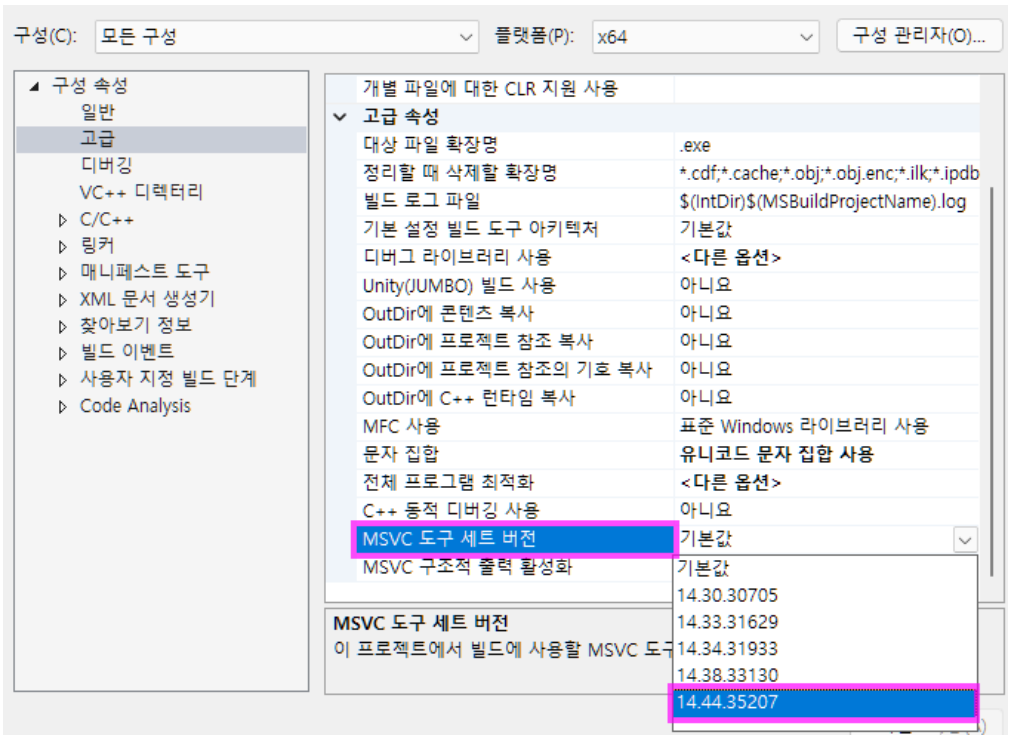
/std:c++latest는 현재 프로젝트에서 사용하는 MSVC 컴파일러가 지원하는 최신 C++ 문법을 활성화한다. 그러나 이 옵션을 선택하는 것만으로 실제 빌드에 사용하는 MSVC 도구 세트가 최신 버전으로 변경되는 것은 아니다. 최신 도구 세트를 설치했더라도 MSVC 도구 세트 버전이 기본값으로 설정되어 있으면 기존 기본 도구 세트를 계속 사용할 수 있다.

따라서 C++23 이후의 최신 기능을 사용하려면 다음 위치에서 설치된 MSVC 도구 세트 중 최신 버전을 직접 선택한다.

```
프로젝트 속성 → 구성 속성 → 고급
 → MSVC 도구 세트 버전 → 설치된 최신 버전
```

이 설정을 완료하면 선택한 MSVC 도구 세트가 지원하는 최신 C++ 기능을 사용할 수 있다. 다만 특정 MSVC 도구 세트 버전을 직접 선택하면 프로젝트가 해당 버전에 종속되며, 같은 버전이 설치되지 않은 다른 개발 환경에서는 빌드할 수 없을 수 있다. 따라서 프로젝트의 호환성이 낮아질 수 있으므로 최신 기능이 필요한 경우에만 사용한다.

<MSVC 도구 세트 최신화>



01. C++ 언어의 특징과 C에서 C++로

C++는 한 가지 프로그래밍 방식만을 강요하는 언어가 아니다. C에서 이어진 절차적 프로그래밍을 사용할 수 있고, 클래스와 객체를 중심으로 프로그램을 구성할 수도 있으며, 템플릿과 표준 라이브러리를 이용해 다양한 자료구조와 알고리즘을 재사용할 수도 있다.

C++가 오랫동안 여러 분야에서 사용되는 이유는 이러한 특징이 함께 존재하기 때문이다. 하드웨어와 가까운 수준에서 프로그램을 제어할 수 있어 높은 성능을 얻을 수 있고, 객체지향 프로그래밍을 통해 규모가 커진 프로그램을 역할과 책임에 따라 나눌 수 있으며, 표준 라이브러리에 포함된 다양한 기능을 이용해 이미 검증된 자료구조와 알고리즘을 활용할 수 있다. 또한 정적 타입 검사와 타입 추론, 컴파일 타임 계산을 통해 안정성과 성능을 함께 고려할 수 있다.

이러한 특징으로 C++는 운영체제와 시스템 소프트웨어뿐 아니라 그래픽스, 게임 엔진, 시뮬레이션, 로봇, 임베디드 시스템, 금융, 데이터 처리, 데스크톱 응용 프로그램 등 성능과 복잡한 구조를 함께 다루어야 하는 분야에서 널리 사용된다.

이 장에서는 C++의 특징을 먼저 살펴본 뒤, C를 학습한 개발자가 C++로 넘어갈 때 알아야 할 언어적 차이를 정리한다. C 문법을 처음부터 반복하지 않고, C++에서 추가되거나 의미가 달라진 기능을 중심으로 설명한다.

1.1 C++ 언어의 특징

C++는 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍을 함께 지원하는 다중 패러다임 언어이다. 프로그램의 모든 부분을 반드시 클래스로 작성할 필요는 없으며, 문제의 성격에 따라 적절한 방식을 선택하거나 여러 방식을 함께 사용할 수 있다.
C++의 주요 특징은 네 가지 관점으로 정리할 수 있다.

```
- 절차적 프로그래밍과 하드웨어에 가까운 제어를 통한 성능
- 객체지향 프로그래밍을 통한 구조화와 유지보수
- 방대한 표준 라이브러리를 통한 자료구조와 알고리즘의 재사용
- 정적 타입 검사와 컴파일 타임 처리를 통한 안정성과 성능
```

1.1.1 절차적 프로그래밍과 우수한 성능

C++는 C의 문법과 실행 모델을 상당 부분 계승했다. 변수, 조건문, 반복문, 함수, 배열, 구조체, 포인터를 이용해 절차적으로 프로그램을 작성할 수 있다. 작은 함수들을 조합하여 처리 과정을 명확하게 표현하는 방식은 지금도 유효하며, 간단하거나 성능이 중요한 코드에서 널리 사용된다.
다만 절차적 프로그래밍 자체가 자동으로 높은 성능을 만드는 것은 아니다.

C++가 높은 성능을 얻을 수 있는 이유는 높은 성능은 네이티브 실행과 직접적인 자원 제어, 컴파일 단계 최적화에서 비롯된다.

```
- 컴파일된 코드의 네이티브 기계어 실행
- 메모리 배치와 객체 수명의 직접 제어
- 포인터와 참조를 이용한 데이터 직접 접근
- 목적에 따른 스택과 동적 메모리 선택
- 사용하지 않는 기능에 대한 비용 배제
- 템플릿 기반 추상화의 컴파일 단계 최적화
```

C++에서는 더 안전하고 명확한 표현을 사용하면서도 불필요한 런타임 비용을 줄일 수 있다. 이를 제로 오버헤드 원칙(zero-overhead principle)이라고 한다. 사용하지 않는 기능에는 비용이 발생하지 않으며, 추상화를 사용한 코드도 같은 기능을 직접 구현한 저수준 코드와 비슷한 수준의 성능을 목표로 한다.

C 방식으로 배열을 함수에 전달하면 배열의 첫 번째 원소를 가리키는 포인터와 원소 수를 각각 전달해야 한다.

```
int SumLowLevel(const int* values, int count)
{
    int result = 0;
    for (int i = 0; i < count; ++i)
    {
        result += values[i];
    }
    return result;
}
```

같은 기능을 C++의 std::span과 범위 기반 for문으로 표현할 수 있다.

```
#include <span>
int Sum(std::span<const int> values)
{
    int result{};
    for (int value : values)
    {
```

```
        result += value;
    }
    return result;
}
```

첫 번째 함수는 배열의 시작 주소와 원소 수를 별도의 인수로 전달하므로 호출하는 쪽에서 두 값이 서로 일치하도록 관리해야 한다. 두 번째 함수는 시작 주소와 원소 수를 std::span이라는 하나의 범위로 묶어 전달한다. 배열을 전달하면 원소 수가 자동으로 반영되므로 호출자가 배열 크기를 별도의 인수로 작성하지 않아도 된다.

std::span은 배열의 원소를 복사하거나 새로운 저장 공간을 할당하지 않는다. 범위 기반 for문도 각 원소를 순서대로 접근하는 반복 코드로 처리된다.

```
#include <iostream>
#include <span>
int SumLowLevel(const int* values, int count);
int Sum(std::span<const int> values);
int main()
{
    int values[]{ 10, 20, 30, 40 };
    int lowLevelResult{ SumLowLevel(values, 4) };
    int spanResult{ Sum(values) };

    std::cout << "포인터와 원소 수: " << lowLevelResult << '\n';
    std::cout << "std::span: " << spanResult << '\n';
}
// SumLowLevel, Sum 함수 정의
// ...
```

Sum()은 포인터와 원소 수를 따로 전달하는 함수보다 호출 방법이 명확하고, 원소 수를 잘못 전달할 가능성도 줄어든다. std::span은 일반적으로 시작 주소와 원소 수만 저장하는 가벼운 객체이며, 컴파일러가 최적화하면 두 함수는 비슷한 형태의 기계어로 변환될 수 있다.

개발자는 배열의 크기를 별도로 맞추는 작업보다 실제 데이터 처리에 집중하면서도 불필요한 실행 시간 비용을 추가하지 않을 수 있다. 이것이 C++에서 말하는 제로 오버헤드 원칙의 대표적인 예이다.

C++가 객체지향 프로그래밍을 지원한다고 해서 모든 연산을 클래스에 넣어야 하는 것은 아니다. 단순한 계산은 일반 함수로 작성하고, 데이터와 동작을 하나의 개념으로 묶어야 할 때는 클래스를 사용하는 등 문제에 적합한 구조를 선택하는 것이 중요하다.

1.1.2 객체지향 프로그래밍과 유지보수

프로그램의 규모가 커지면 단순히 함수를 나누는 것만으로는 전체 구조를 관리하기 어려워진다. 여러 함수가 같은 데이터를 공유하고, 데이터의 변경 규칙이 여러 위치에 흩어지면 한 부분을 수정했을 때 다른 기능까지 영향을 받을 수 있다.

객체지향 프로그래밍은 데이터와 그 데이터를 처리하는 기능을 하나의 객체 안에 묶는다. C++에서는 클래스를 이용하여 객체의 상태와 행위를 함께 정의할 수 있다.

```
class Character
{
public:
    void TakeDamage(int damage)
    {
        hp -= damage;
        if (hp < 0)
            hp = 0;
    }
    int GetHp() const
    {
        return hp;
    }
private:
    int hp = 100;
};
```

이 클래스는 체력이라는 상태와 피해를 처리하는 행위를 함께 관리한다. 외부 코드가 hp 값을 임의로 변경하지 못하도록 제한하고, TakeDamage()를 통해서만 상태가 변경되도록 구성했다.

이와 같은 구조는 많은 장점을 제공한다.

<객체지향 프로그래밍 장점>

```
- 데이터 변경 범위 제한
- 객체의 유효 상태 유지
```

```
- 관련 데이터와 기능 통합
- 역할과 책임에 따른 구조 분리
- 내부 구현 변경 영향 최소화
- 기능별 개발 분담과 협업
```

객체지향 프로그래밍을 사용한다고 해서 유지보수성이 자동으로 좋아지는 것은 아니다. 책임이 지나치게 많은 클래스, 깊은 상속 구조, 불필요한 추상화는 오히려 프로그램을 복잡하게 만들 수 있다. 객체지향의 목적은 클래스를 많이 만드는 것이 아니라, 변경될 가능성이 있는 부분을 분리하고 객체 사이의 책임을 명확하게 만드는 데 있다.

1.1.3 제네릭 프로그래밍과 표준 라이브러리

C++ 표준 라이브러리는 문자열, 자료구조, 알고리즘, 파일 입출력, 시간, 난수, 메모리 관리 등 프로그램 작성에 필요한 다양한 기능을 제공한다.
C에서 여러 개의 데이터를 관리하려면 배열을 직접 할당하고 크기를 별도로 저장하거나, 연결 리스트와 트리 같은 자료구조를 직접 구현해야 하는 경우가 많았다. C++에서는 표준 라이브러리의 컨테이너를 이용해 이미 구현되고 검증된 자료구조를 사용할 수 있다.

<대표적으로 자주 사용되는 컨테이너>

```
- std::array: 크기가 고정된 배열
- std::vector: 크기가 변하는 연속 배열
- std::deque: 앞과 뒤에서 원소를 추가하거나 제거할 수 있는 컨테이너
- std::list: 연결 리스트
- std::map: 정렬된 키와 값
- std::set: 중복되지 않는 정렬된 값
- std::unordered_map: 해시를 이용한 키와 값
- std::string: 문자열
```

표준 라이브러리는 자료구조만 제공하는 것이 아니다. 컨테이너의 원소를 검색하고 정렬하고 변환하는 알고리즘도 함께 제공한다.

```cpp
#include <algorithm>
#include <iostream>
#include <vector>
int main()
{
    std::vector<int> values = { 30, 10, 50, 20, 40 };
    std::sort(values.begin(), values.end());
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

std::vector는 데이터 저장을 담당하고, std::sort는 정렬을 담당한다. 개발자가 동적 배열과 정렬 알고리즘을 매번 직접 구현할 필요가 없다. 표준 라이브러리의 장점은 단순히 코드가 짧아지는 데 있지 않다.

<표준 라이브러리 장점>

```
- 여러 환경에서 널리 사용되고 검증된 구현 활용
- 컨테이너와 알고리즘의 일관된 구성
- 템플릿을 통한 다양한 타입의 재사용
- 타입에 맞는 코드의 컴파일 단계 최적화
- 자료구조 구현에 필요한 시간과 오류 감소
```

자료구조의 원리와 시간 복잡도는 여전히 이해해야 한다. 그러나 실제 프로그램에서 연결 리스트, 동적 배열, 해시 테이블을 매번 처음부터 구현할 필요는 없다. 개발자는 문제에 적합한 컨테이너를 선택하고, 데이터의 저장 방식과 탐색 비용을 판단하는 데 집중할 수 있다.

1.1.4 컴파일러의 추론과 자동 처리

C++는 개발자가 모든 타입과 동작을 직접 명시하는 방식보다, 코드에 충분한 정보를 제공하고 컴파일러가 그 정보를 이용해 필요한 타입과 동작을 결정하도록 하는 방식을 폭넓게 지원한다. C에서도 컴파일러가 형 변환이나 최적화 같은 처리를 수행하지만, C++는 타입 추론, 템플릿 인자 추론, 오버로드 선택, 특수 멤버 함수의 자동 생성, 이동 처리, 복사 생략 등 언어 차원에서 컴파일러가 판단하는 영역이 훨씬 넓다.

모던 C++에서는 컴파일러가 이미 알고 있는 정보를 개발자가 반복해서 작성하지 않고, 컴파일러가 정확하게 판단할 수 있도록 코드를 구성하는 것이 중요하다. 개발자가 모든 것을 직접 지정하는 것보다 필요한 정보만 명확하게 표현하고 나머지는 컴파일러에 맡기는 편이 코드의 중복을 줄이고 변경에도 유리한 경우가 많다.

타입 추론, 템플릿 인자 추론, 오버로딩, 이동 의미론, 반환 값 최적화 등은 서로 다른 기능이지만 공통적으로 컴파일러가 코드의 정보를 분석하여 적절한 처리를 결정한다는 특징을 가진다. 개발자는 이러한 기능을 억지로 제어하기보다 컴파일러가 판단할 수 있는 정보를 명확하게 제공하고, 불필요한 개입으로 추론이나 최적화의 기회를 줄이지 않도록 코드를 작성하는 것이 좋다.

```cpp
#include <string>
std::string MakeTitle(const std::string& name)
{
    std::string result = "Player: ";
    result += name;

    return result;
}
```

예를 들어, 컴파일러는 조건이 맞으면 NRVO(Named Return Value Optimization)를 적용하여 반환받을 객체의 저장 공간에 result를 직접 생성할 수 있다. NRVO가 적용되지 않는 경우에도 이동이 가능한 객체라면 이동 처리가 사용될 수 있다.

이와 같이 모던 C++에서는 개발자가 복사와 이동 과정을 모두 직접 지시하기보다 객체의 수명과 타입 관계를 명확하게 표현하고, 컴파일러가 적절한 방법을 선택할 수 있도록 작성하는 방식이 중요하다.

<모던 C++의 추론과 자동 처리를 활용하는 습관>

```
- 컴파일러가 이미 알고 있는 타입을 불필요하게 반복하지 않음
- 복잡하거나 변경 가능성이 큰 타입은 auto를 활용
- 템플릿 인자를 추론할 수 있으면 불필요하게 직접 지정 안함
- 직접 작성할 필요가 없는 특수 멤버 함수는 컴파일러의 자동 생성
활용
- 직접 자원 관리보다 RAII 타입을 사용. Rule of Zero를 우선적으로
고려
- 지역 객체 반환에서 NRVO와 복사 생략 기회 유지
- 코드의 의미를 더 분명하게 만드는 경우 명시적으로 표현
- 소유권, const, 타입 관계와 객체 수명을 코드에 명확하게 표현
```

컴파일러에 맡긴다는 것은 동작 원리를 몰라도 된다는 뜻이 아니다. 개발자는 컴파일러가 어떤 정보를 이용하여 타입과 함수를 선택하고 객체의 복사와 이동을 처리하는지 이해해야 한다. 모던 C++에서는 개발자의 의도는 명확하게 표현하고, 컴파일러가 이미 판단할 수 있는 부분은 컴파일러에 맡기는 코딩 습관이 중요하다.

1.1.5 정적 타입과 컴파일 타임 처리

C++는 변수와 표현식의 타입이 컴파일 시점에 결정되는 정적 타입 언어이다. 컴파일러는 서로 대입할 수 없는 타입, 잘못된 함수 호출, 조건을 만족하지 않는 템플릿 사용 등을 프로그램이 실행되기 전에 검사한다.

C++는 C의 기본적인 타입 체계를 계승하면서도 잘못된 사용을 줄이기 위한 기능을 추가하였다. nullptr는 정수값 0과 널 포인터를 구분하고, enum class는 열거형 값의 범위를 제한한다. 또한 C++ 형 변환 연산자를 사용하면 어떤 종류의 변환을 수행하는지 코드에 명확하게 표현할 수 있다.

C++는 정적 타입 언어이지만 모든 타입을 직접 작성할 필요는 없다. auto와 decltype을 사용하면 컴파일러가 초기값이나 표현식을 분석하여 타입을 결정한다. 타입 추론은 타입을 없애는 기능이 아니다. 컴파일러가 타입을 대신 결정하는 것이며, 결정된 타입은 실행 중에 변경되지 않는다.

C++는 일부 계산과 검사도 컴파일 시점에 수행할 수 있다. constexpr는 값이나 함수를 상수 표현식으로 사용할 수 있게 하고, static_assert는 지정한 조건이 만족되지 않으면 컴파일을 중단한다.

```cpp
constexpr int Square(int value)
{
    return value * value;
}
constexpr int size = Square(10);
static_assert(size == 100);
```

constexpr 함수의 모든 호출이 반드시 컴파일 시점에 계산되는 것은 아니다. 상수 표현식이 필요한 상황에서는 컴파일 시점에 평가되지만, 실행 중에 결정되는 값을 전달하면 일반 함수처럼 실행 시점에 계산될 수도 있다. 반드시 컴파일 시점에 평가해야 하는 함수에는 consteval을 사용한다.

이러한 기능은 잘못된 타입 사용을 실행 전에 발견하고, 일부 계산을 컴파일 과정으로 옮겨 실행 시간의 비용을 줄일 수 있게 한다. C++는 하드웨어에 가까운 제어 능력을 유지하면서도 타입 검사와 컴파일 타임 기능을 통해 안정성과 성능을 함께 고려할 수 있는 언어이다.

nullptr, enum class, auto, decltype과 형 변환 연산자의 구체적인 사용법은 1.7절에서 다시 살펴본다. constexpr, consteval, constinit은 객체 초기화와 컴파일 타임 평가를 다루는 단원에서 자세히 설명한다.

1.1.6 자원 관리와 RAII

프로그램은 메모리뿐 아니라 파일, 네트워크 연결, 잠금, 그래픽스 자원 등 다양한 자원을 사용한다. 이러한 자원은 사용이 끝난 시점에 정확하게 반환해야 한다.

C++에서는 객체의 생성과 소멸을 자원 관리에 연결하는 RAII(Resource Acquisition Is Initialization) 방식을 널리 사용한다. 객체가 생성될 때 자원을 확보하고, 객체의 수명이 끝날 때 소멸자를 통해 자원을 자동으로 반환한다.

```
#include <memory>
void ProcessData(bool cancel)
{
    auto values = std::make_unique<int[]>(100);
    values[0] = 10;
    values[1] = 20;
    if (cancel)
    {
        return;
    }
    values[2] = values[0] + values[1];
}
```

std::make_unique<int[]>(100)은 정수 100개를 저장할 동적 메모리를 확보하고, 그 메모리의 소유권을 std::unique_ptr 객체에 전달한다. auto는 초기값을 분석하여 values의 타입을 자동으로 결정한다.

values가 함수 범위를 벗어나면 std::unique_ptr의 소멸자가 호출되어 동적 메모리를 자동으로 해제한다. 함수가 정상적으로 끝나는 경우뿐 아니라 중간에 return이 실행되거나 예외가 발생하는 경우에도 객체의 소멸 과정은 유지된다. 따라서 개발자가 delete[]를 직접 호출할 필요가 없다.

<RAII가 자원 관리에 활용되는 예>

```
- 동적 메모리의 자동 해제
- 파일과 네트워크 연결의 종료
- 잠금의 자동 해제
- 운영체제 및 그래픽스 자원의 반환
- 예외 발생 시 자원 누수 방지
```

std::vector, std::string, 스마트 포인터와 같은 표준 라이브러리 타입도 RAII를 기반으로 자원을 관리한다. 개발자는 자원의 해제 시점을 일일이 추적하기보다 객체의 수명과 소유 관계를 설계하는 데 집중할 수 있다.

1.1.7 여러 분야에서 사용되는 C++

C++는 높은 성능만 필요한 언어도 아니고, 객체지향만을 위한 언어도 아니다. 성능, 메모리 제어, 복잡한 프로그램 구조, 재사용 가능한 라이브러리가 함께 필요한 분야에서 장점을 가진다.

<C++언어의 활용 분야>

```
- 운영체제와 시스템 소프트웨어
- 게임 엔진과 실시간 콘텐츠
- 그래픽스와 렌더링
- 물리 및 과학 시뮬레이션
- 로봇과 자율주행 시스템
- 임베디드 시스템
- 금융 및 대규모 데이터 처리
- 데이터베이스와 네트워크 서버
- 웹 브라우저와 데스크톱 응용 프로그램
- 개발 도구와 콘텐츠 제작 도구
```

특히 응용 프로그램은 성능만으로 완성되지 않는다. 사용자 기능이 계속 추가되고, 데이터 구조가 변하며, 여러 시스템이 서로 연결된다. C++는 절차적 코드의 직접성과 객체지향 구조의 관리 능력, 표준 라이브러리의 재사용성, 컴파일 타임 처리 능력을 함께 제공하므로 복잡한 응용 프로그램을 만드는 데 적합하다.

1.2 C와 C++의 관계

C++는 C 언어를 기반으로 발전했으며, 변수와 연산자, 제어문(조건문, 반복문), 함수, 배열, 구조체 및 포인터 등 C의 핵심 문법과 표준 라이브러리를 상당 부분 계승했다. 따라서 C 언어를 이미 학습한 독자라면 기존 지식을 바탕으로 더욱 수월하게 C++를 익힐 수 있다.

다만, C++가 단순히 C 언어에 클래스(Class) 문법만 추가한 것은 아니다. 보다 엄격해진 타입 검사(Type Checking)와 확장된 함수 문법을 비롯해 참조(Reference), 이름공간(Namespace), 템플릿(Template), 그리고 향상된 표준 라이브러리(STL) 등 프로그램을 더 안전하고 체계적으로 구성할 수 있는 강력한 기능들이 대거 추가되었다.

이 절에서는 앞으로 다룰 예제들의 입력과 실행 결과를 확인하기 위해 필요한 기본 입출력 방식을 먼저 살펴본 뒤, C 언어에서 그대로 이어지는 부분과 C++에서 새롭게 달라진 특징들을 차례로 정리한다.

1.2.1 타입을 이용한 안전한 입출력

C에서는 값을 입력하고 출력할 때 주로 scanf()와 printf()를 사용한다.

```
int level = 0;
double speed = 0.0;
scanf("%d %lf", &level, &speed);
printf("Level: %d, Speed: %.1f\n", level, speed);
```

scanf()는 값을 저장할 변수의 주소를 전달해야 하므로 일반 변수 앞에 주소 연산자 &를 붙인다. 또한 %d, %f, %lf와 같은 형식 지정자를 변수의 타입에 맞게 작성해야 한다. 주소 연산자를 빠뜨리거나 형식 지정자와 변수의 타입을 잘못 연결하면 예상하지 못한 결과가 발생할 수 있다.

C++에서는 표준 입력 객체인 cin과 표준 출력 객체인 cout을 사용할 수 있다. 두 객체는 C++ 표준 라이브러리의 std 이름 공간에 정의되어 있다.

```
#include <iostream>
int main()
{
    using std::cin;
    using std::cout;

    int level = 0;
    double speed = 0.0;
    cout << "레벨과 이동 속도를 입력하세요: ";
    cin >> level >> speed;
    cout << "Level: " << level
         << ", Speed: " << speed << '\n';
}
```

cin과 cout은 변수와 표현식의 타입에 따라 적절한 입출력 동작을 선택한다. 따라서 형식 지정자를 직접 작성하지 않으며, cin으로 일반 변수에 값을 입력할 때 주소 연산자 "&"도 사용하지 않는다.

```
cin >> level;
cout << level << '\n';
```

공백이 포함된 한 줄의 문자열은 std::string과 getline()을 이용해 입력할 수 있다.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string name;
    cout << "이름을 입력하세요: ";
    getline(cin, name);
    cout << "입력한 이름: " << name << '\n';
}
```

C++의 입출력도 잘못된 입력이나 입력 버퍼와 같은 문제를 별도로 처리해야 한다. 그러나 변수의 주소와 형식 지정자를 개발자가 직접 맞춰야 하는 부담을 줄이고, 타입에 맞는 입출력 동작을 선택함으로써 단순한 실수의 가능성을 낮춘다.

이 장부터 예제의 입력과 출력에는 scanf()와 printf() 대신 cin, cout, getline()을 사용한다. 이름 공간과 입출력 스트림의 구조, 공백을 포함한 문자열 입력, 입력 오류와 버퍼 처리, 출력 형식 지정은 입출력 단원에서 자세히 살펴본다.

1.2.2 C 문법의 계승

C++는 C의 문법과 실행 방식을 상당 부분 계승하였다. C에서 사용한 다음 개념은 C++에서도 그대로 활용할 수 있다.

- 기본형 타입과 변수
- 산술·관계·논리 연산자
- 조건문과 반복문
- 함수
- 배열과 구조체
- 포인터와 함수 포인터
- 동적 메모리
- 파일 입출력

예를 들어 다음과 같은 C의 구조체와 함수 작성 방식은 C++에서도 사용할 수 있다.

```
struct Item
{
    char name[32];
};
struct Character
{
    int hp;
    int attack;
    struct Item* pItem;
};
void TakeDamage(struct Character* character, int damage)
{
    character->hp -= damage;
    if (character->hp < 0)
    {
        character->hp = 0;
    }
}
```

C++에서는 구조체 이름 자체가 타입 이름이므로 struct를 반복하지 않아도 된다.

```
struct Character
{
    // ...
    Item* pItem;
};
void TakeDamage(Character* character, int damage)
{
    // ...
}
```

C를 학습한 독자는 함수, 배열, 구조체와 포인터에 관한 기존 지식을 그대로 활용할 수 있다. 그 위에 참조, 클래스, 템플릿과 표준 라이브러리 등의 기능을 단계적으로 더하면 된다.

다만 C에서 사용하던 모든 작성 방식이 C++에서도 가장 적절한 것은 아니다. C++에서는 같은 기능을 더 안전하고 명확하게 표현할 수 있는 문법과 라이브러리를 제공하므로 기존 C 문법을 바탕으로 C++에 적합한 작성 방식으로 확장할 필요가 있다.

1.2.3 C와 C++의 차이

C++는 C에 많은 기능들이 추가하거나 확장되었다.

- 이름 공간
- 참조
- 함수 오버로딩과 기본 인수
- 클래스와 접근 제어
- 생성자와 소멸자
- 상속과 다형성
- 예외 처리
- 템플릿과 템플릿 인수 추론
- 표준 라이브러리
- RAII와 스마트 포인터
- auto와 decltype
- nullptr와 enum class
- 람다 표현식과 Ranges

C++는 타입 검사에서도 C보다 엄격한 부분이 있다. 예를 들어 C에서는 void*를 다른 포인터 타입에 암시적으로 대입할 수 있지만, C++에서는 명시적인 형 변환이 필요하다.

```
void* memory = nullptr;
// int* pointer = memory; // C++에서는 오류
int* pointer = static_cast<int*>(memory);
```

C++에서는 함수 오버로딩과 템플릿처럼 타입에 따라 동작이 달라지는 기능을 제공하므로, 타입을 더 명확하게 구분해야 한다. 이 때문에 C에서 허용되는 코드가 C++에서는 허용되지 않거나 의미가 달라지는 경우가 있다.
따라서 C++는 C의 문법을 대부분 계승하지만 C의 모든 프로그램을 그대로 받아들이는 완전한 상위 호환 언어는 아니다.

1.2.4 C++의 주요 키워드

키워드(keyword)는 언어의 문법에서 특별한 의미로 사용하도록 예약된 단어이다. 키워드는 변수, 함수, 구조체와 클래스의 이름으로 사용할 수 없다.

```
// int class = 10;     // 오류: class는 키워드
// void return();      // 오류: return은 키워드
```

C++는 C의 키워드를 대부분 계승하면서 객체지향, 템플릿, 예외 처리와 타입 안전성을 표현하기 위한 키워드를 추가하였다.

<C++ 코드 주요 키워드>

구분	주요 키워드
논리값과 널 포인터	bool, true, false, nullptr
이름과 타입 정의	namespace, using, class, typename
객체와 접근 제어	public, protected, private, this, friend
상속과 다형성	virtual, override, final
템플릿과 타입 추론	template, auto, decltype
형 변환	static_cast, const_cast, reinterpret_cast, dynamic_cast
예외 처리	try, catch, throw, noexcept
컴파일 타임 처리	constexpr, consteval, constinit, static_assert
메모리와 연산자	new, delete, operator, explicit

이 표는 C++의 모든 키워드를 나열한 것이 아니라 이후 단원에서 자주 사용할 키워드를 기능별로 정리한 것이다. 각 키워드의 문법과 사용 목적은 관련 단원에서 자세히 살펴본다.

1.2.5 C++을 C처럼만 사용할 때의 한계

C++에서도 C 방식으로 배열을 만들고 malloc()과 free()로 메모리를 관리하며 구조체와 전역 함수를 중심으로 프로그램을 작성할 수 있다. 규모가 작고 동작이 단순한 프로그램에서는 이러한 방식도 사용할 수 있다.

그런데 프로그램의 기능과 데이터가 늘어나면 다음과 같은 문제를 개발자가 직접 관리해야 한다.

- 할당한 메모리를 해제하지 않아 발생하는 메모리 누수
- 이미 해제되었거나 수명이 끝난 대상을 가리키는 댕글링 포인터
- 배열 범위를 벗어난 접근
- 여러 함수가 같은 데이터를 변경하면서 발생하는 상태 관리 문제
- 자원의 소유자와 해제 책임이 불분명하여 발생하는 중복 해제
- 한 부분의 수정이 다른 부분에 미치는 예상하지 못한 영향

동적 배열을 C 방식으로 관리하는 예를 들어보자

```
#include <cstdlib>
void ProcessData()
{
    int* values = static_cast<int*>(std::malloc(sizeof(int) * 100));
    if (values == nullptr)
    {
        return;
    }
    // 데이터 사용
    // ...
    std::free(values);
}
```

이 코드에서는 메모리의 소유자와 해제 위치를 개발자가 직접 관리한다. 함수가 중간에 끝나거나 예외가 발생하여 std::free()에 도달하지 못하면 메모리 누수가 발생할 수 있다.

C++에서는 같은 목적을 표준 컨테이너로 표현할 수 있다.

```
#include <vector>
void ProcessData()
{
    std::vector<int> values(100);
    // 데이터 사용
    // ...
}
```

std::vector는 저장 공간과 원소 수를 하나의 객체로 관리한다. values의 수명이 끝나면 내부 메모리가 자동으로 해제되므로 해제 위치를 직접 추적할 필요가 없다.

C 방식과 C++ 방식의 차이는 단순히 문법이 짧아지는 데 있지 않다. 데이터의 변경 규칙, 자원의 수명과 소유 관계를 프로그램의 구조 안에 표현하는 방식이 달라진다.
C++를 사용한다고 모든 오류가 자동으로 사라지는 것은 아니다. 그러나 클래스, RAII, 표준 컨테이너, 스마트 포인터와 타입 시스템을 적절히 활용하면 개발자가 직접 추적해야 하는 상태와 자원 관리의 범위를 줄일 수 있다.

1.3 C++의 기본형 타입

C++의 타입은 크게 기본형 타입, 복합 타입과 사용자 정의 타입으로 나눌 수 있다. 기본형 타입은 언어가 직접 제공하는 타입이다. 복합 타입에는 포인터, 참조, 배열과 함수 타입 등이 포함된다. 클래스, 구조체, 공용체와 열거형은 사용자 정의 타입에 해당한다.

C++는 C의 기본형 타입을 대부분 그대로 사용하면서 논리형과 Unicode 문자형, 널 포인터를 나타내는 타입 등을 추가하였다. 이 절에서는 C와 같은 부분을 반복하기보다 C++에서 추가되거나 주의해야 할 부분을 중심으로 살펴본다.

1.3.1 기본형 타입의 분류

C++의 기본형 타입은 C의 기본형 타입을 대부분 계승한다. 여기에 논리값을 표현하는 bool, 와이드 문자와 Unicode 코드 단위를 표현하는 wchar_t, char8_t, char16_t, char32_t, 널 포인터를 나타내는 std::nullptr_t가 포함된다.

C에도 _Bool, wchar_t, char16_t, char32_t와 같은 대응 타입이 있다. _Bool은 C의 논리형이고, wchar_t, char16_t, char32_t는 헤더에서 기존 정수형의 별칭으로 정의된다. 반면 C++에서는 bool, wchar_t, char8_t, char16_t, char32_t가 서로 구분되는 기본형 타입이다.

<C++의 기본형 타입>

분류	타입
논리형	bool
문자형	char, signed char, unsigned char, wchar_t, char8_t, char16_t, char32_t
정수형	short, int, long, long long과 각각의 unsigned 형식
부동소수점형	float, double, long double
빈 타입	void
널 포인터 타입	std::nullptr_t

char, short, int, long, float, double과 같은 기존 기본형은 C와 같은 역할로 사용한다. 정확한 크기는 컴파일러와 실행 환경에 따라 달라질 수 있으며, 같은 플랫폼과 ABI의 C와 C++에서는 대응하는 기본형이 같은 데이터 모델을 사용한다.

1.3.2 정수형과 리터럴 접미사

C++의 정수형은 C에서 사용하던 short, int, long을 계승하며 long long을 표준 정수형으로 제공한다.

```
short smallValue = 10;
int value = 100;
long largeValue = 1000L;
long long veryLargeValue = 10000LL;
```

정수 리터럴은 접미사에 따라 타입이 달라질 수 있다.

접미사	리터럴 타입	예
없음	값을 표현할 수 있는 표준 정수형	100

U 또는 u	부호 없는 정수형	100**U**
L 또는 l	long	100**L**
LL 또는 ll	long long	100**LL**
UL, LU	unsigned long	100**UL**
ULL, LLU	unsigned long long	100**ULL**

소문자 l은 숫자 1과 구분하기 어려우므로 long과 long long 리터럴에는 대문자 L을 사용하는 편이 좋다.

정수형은 부호 있는 형식과 부호 없는 형식으로 나뉜다.

```
int signedValue = -10;
unsigned int unsignedValue = 10U;
```

부호 없는 정수형은 음수를 표현하지 않는 대신 같은 크기의 부호 있는 정수형보다 더 큰 양수를 표현할 수 있다. 그러나 일반적인 계산에 무조건 unsigned를 사용하면 부호 있는 정수형과의 비교나 뺄셈에서 예상하지 못한 결과가 발생할 수 있다.
일반적인 수치 계산에는 부호 있는 정수형을 사용하고, 비트 연산이나 크기처럼 음수가 의미 없는 값을 나타낼 때 부호 없는 정수형을 목적에 맞게 사용한다.

2진수 접두어 **0b**와 자릿수 구분**(')**
C++14부터 정수 리터럴 앞에 0b 또는 0B를 붙여 2진수로 작성할 수 있다.

```
int flags = 0b1010;
unsigned int mask = 0B11110000U;
```

긴 숫자는 작은따옴표를 자릿수 구분자로 사용할 수 있다. 자릿수 구분자는 값에 영향을 주지 않는다.

```
int binaryValue = 0b1010'1100;
int decimalValue = 1'000'000;
```

2진수 리터럴은 값을 2진수 표기로 작성하는 문법이다. 값을 실제 비트 형태로 출력하는 방법은 2장의 출력 형식 지정에서 살펴본다.
정확한 비트 폭이 필요한 경우에는 <cstdint>의 고정 폭 정수형을 사용할 수 있다.

```
#include <cstdint>
std::int32_t id = 100;
std::uint64_t fileSize = 5000000000ULL;
```

std::int32_t는 정확히 32비트인 부호 있는 정수형이고, std::uint64_t는 정확히 64비트인 부호 없는 정수형이다. 이러한 타입은 해당 비트 폭을 정확하게 표현할 수 있는 정수형이 실행 환경에 존재할 때 정의된다.

고정 폭 정수형은 파일 형식, 네트워크 패킷과 바이너리 데이터처럼 비트 폭이 정확하게 정해져야 하는 경우에 유용하다.

Visual C++의 오래된 코드에서는 64비트 정수를 나타내기 위해 Microsoft 전용 타입인 __int64를 사용하기도 했다. __int64는 비표준 확장 타입으로, 다른 컴파일러에서는 지원되지 않을 수 있다.

```
__int64 legacyValue = 100;        // Microsoft 전용 비표준 타입
long long standardValue = 100LL;  // 표준 C++ 타입
std::int64_t fixedValue = 100;    // 정확히 64비트인 정수형
```

long long은 C++11부터 표준 C++의 기본 정수형으로 정식 채택되었으며 최소 64비트의 표현 범위를 보장한다. 정확히 64비트인 정수형이 필요한 경우에, <cstdint>에 정의된 std::int64_t 또는 std::uint64_t를 사용한다.
새로 작성하는 코드에서는 표준 정수형인 long long 또는 고정 폭 정수형을 사용하고, __int64는 오래된 Visual C++ 코드를 이해할 때 알아두면 된다.

<정수형과 고정폭 정수형>

```
#include <cstdint>
#include <iostream>
#include <limits>
using namespace std;

int main()
{
    int signedValue = 0xFFFFFFFF;
    unsigned int unsignedValue = 0xFFFFFFFFU;
    long largeValue = 10000000L;
```

```
    long long veryLargeValue = 0x7FFFFFFFFFFFFFFFLL;
    std::int32_t id = 100;
    std::uint64_t fileSize = 5000000000ULL;

    cout << "signedValue(int)           : "  << signedValue << '\n';
    cout << "unsignedValue(unsigned int): "  << unsignedValue << '\n';
    cout << "largeValue(long)           : "  << largeValue << '\n';
    cout << "veryLargeValue(long long)  : "  << veryLargeValue << '\n';
    cout << "id(int32_t)                : "  << id << '\n';
    cout << "fileSize(uint64_t)         : "  << fileSize << "\n\n";

#ifdef _MSC_VER
    cout << "__in64(Microsoft type)  ---------\n";
    __int64 legacyValue = 100;
    cout << "sizeof(__int64)            : "  << sizeof(__int64) << "byte\n";
    cout << "legacyValue(__int64)       : "  << legacyValue << "\n\n";
#endif
}
```

1.3.3 논리형(bool)

C++는 참과 거짓을 표현하기 위해 bool 타입을 제공한다. bool 타입에 저장할 수 있는 논리값은 true와 false이다.

```
bool isRunning = true;
bool isFinished = false;
```

초기의 C언어에는 참과 거짓을 나타내는 별도의 논리형이 없었다. 따라서 조건이나 상태를 표현할 때 주로 정수형을 사용했다.

```
int isRunning = 1;   // 참
int isFinished = 0;  // 거짓
```

```
bool first = 0;    // false
bool second = 10;  // true
bool third = -1;   // true
```

정수값이 bool로 변환되면 원래 정수값이 그대로 저장되는 것이 아니라 false 또는 true가 저장된다.

```
int value = 10;
bool result = value; // true
```

반대로 bool 값이 정수형으로 변환되면 false는 0, true는 1이 된다.

```
int firstValue = false; // 0
int secondValue = true; // 1
```

비교 연산자와 논리 연산자의 결과도 bool 타입이다.

```
int hp = 50;
bool isAlive = hp > 0;
bool isDead = hp <= 0;
bool canContinue = isAlive && !isFinished;
```

무한 반복 도 정수 1보다 논리값 true를 사용하여 의도를 표현할 수 있다.

```
while (true)
{
    // 반복 처리
}
```

while (1)도 C++에서 정상적으로 동작하지만, 이 교재에서는 조건이 항상 참이라는 뜻을 직접 나타내기 위해 while (true)를 사용한다.

<논리형과 값 변환>

```
#include <iostream>
using namespace std;
int main()
{
```

```
    int cStyleRunning = 1;
    bool cppStyleRunning = true;

    bool first  = 0;
    bool second = 10;
    bool third  = -1;

    cout << std::boolalpha; // bool 값을 1,0대신, true, false로 출력.

    cout << "C   방식 논리 상태: " << cStyleRunning << '\n';
    cout << "C++ 방식 논리 상태: " << cppStyleRunning << "\n\n";

    cout << "first  : " << first   << '\n';
    cout << "second : " << second  << '\n';
    cout << "third  : " << third   << "\n\n";
}
```

cout은 기본적으로 bool 값을 0과 1로 출력한다. 예제에서는 std::boolalpha를 사용하여 false와 true로 출력하도록 설정했다. 출력 형식과 서식 조정자는 입출력 형식 지정을 다루는 단원에서 살펴본다.

1.3.4 확장된 문자형과 리터럴 접두사

C++는 일반 문자형인 char 외에도 와이드 문자와 Unicode 코드 단위를 표현하기 위한 문자형을 제공한다.

```
char asciiCharacter = 'A';
wchar_t wideCharacter = L'A';
char8_t utf8Character = u8'A';
char16_t utf16Character = u'A';
char32_t utf32Character = U'A';
```

문자 리터럴 앞에 붙는 접두사에 따라 리터럴의 타입이 달라진다.

표기	타입	용도
'A'	char	일반 문자 코드 단위
L'A'	wchar_t	와이드 문자 코드 단위
u8'A'	char8_t	UTF-8 코드 단위
u'A'	char16_t	UTF-16 코드 단위
U'A'	char32_t	UTF-32 코드 단위

char, signed char, unsigned char는 서로 구분되는 타입이다.

```
char character = 'A';
signed char signedValue = -10;
unsigned char unsignedValue = 200;
```

일반 char가 부호 있는 타입처럼 동작하는지 부호 없는 타입처럼 동작하는지는 실행 환경에 따라 달라진다. 작은 정수값의 부호를 명확하게 나타내려면 signed char 또는 unsigned char를 사용한다.
wchar_t의 크기와 인코딩 방식은 실행 환경에 따라 달라질 수 있다. char8_t는 UTF-8, char16_t는 UTF-16, char32_t는 UTF-32의 코드 단위를 나타낸다.
이 타입들은 화면에 표시되는 문자 하나보다 문자 인코딩을 구성하는 코드 단위를 저장한다고 이해하는 것이 정확하다. 하나의 문자는 인코딩 방식에 따라 여러 코드 단위로 표현될 수 있다.

문자형의 크기와 UTF-8·UTF-16·UTF-32의 코드 단위 수는 문자열을 다루는 단원에서 살펴본다.

1.3.5 부동소수점형의 종류와 접미사

C++의 부동소수점형은 C와 마찬가지로 float, double, long double로 구분한다.

```
float speed = 3.5f;
double distance = 1250.75;
long double extendedPi = 3.14159265358979323846264338327795L;
```

부동소수점 리터럴은 접미사에 따라 타입이 결정된다.

접미사	리터럴 타입	예
없음	double	3.5
f 또는 F	float	3.5f
l 또는 L	long double	3.5L

접미사가 없는 부동소수점 리터럴은 기본적으로 double 타입이다. 소문자 l은 숫자 1과 구분하기 어려우므로 long double 리터럴에는 대문자 L을 사용하는 편이 좋다.

매우 크거나 작은 값은 지수 표기법으로 작성할 수 있다.

```
double lightSpeed = 2.99792458e8;
double epsilon = 1.0e-6;
float smallValue = 1.0e-3f;
```

부동소수점형의 크기와 정밀도, 근사 표현과 오차는 C에서 다룬 내용과 같다. 여기서는 C++에서도 같은 부동소수점형과 지수 표기법을 사용하며, 리터럴 접미사에 따라 타입이 달라진다는 점만 확인한다.

1.3.6 void, nullptr, std::nullptr_t

void는 값이 없음을 나타내는 타입이다. 함수의 반환 타입에 void를 사용하면 호출한 곳으로 값을 반환하지 않는다는 뜻이다.

```
void PrintMessage()
{
    std::cout << "Hello" << '\n';
}
```

반환 타입이 void가 아닌 일반 함수는 return 값;을 사용하여 반드시 반환값을 전달해야 한다.

참고로 main() 함수는 끝에 도달하면 return 0;을 실행한 것과 같이 처리되므로 생략할 수 있다. 본 교재에서는 정상적으로 종료되는 main() 함수에 **return 0;** 을 작성하지 않았다.

반환 타입이 void인 함수에서도 함수의 실행을 중간에 끝내기 위해 return을 사용할 수 있다. 이때는 반환값을 작성하지 않는다.

```
void PrintPositive(int value)
{
    if(value < 0)
        return;
    std::cout << value << '\n';
}
```

C++에서는 매개변수가 없는 함수를 빈 괄호 ()로 선언한다.

```
void PrintMessage();
```

C17까지의 C에서는 빈 괄호 ()가 매개변수의 개수와 타입을 지정하지 않았다는 뜻이었으며, 매개변수가 없음을 명확히 나타내려면 (void)를 사용했다.

표기	C17까지의 C	C++
Function()	매개변수 정보가 지정되지 않음	매개변수가 없음
Function(void)	매개변수가 없음	매개변수가 없음

C와의 문법적 호환을 위해 C++에서도 (void) 표기를 사용할 수 있지만, C++에서는 빈 괄호 ()를 사용한다. C23부터 C에서도 빈 괄호 ()가 매개변수가 없는 함수를 의미한다. 그러나 기존 C 코드와 C17까지의 문법으로 작성된 코드를 이해하려면 두 표기의 차이를 알아둘 필요가 있다.

void 타입의 객체나 배열을 만들 수 없다. void는 객체에 저장할 값이나 객체의 크기를 정의하지 않기 때문이다.

```
// void value;       // 오류
// void values[10];  // 오류
```

다만 void*는 사용할 수 있다. void*는 가리키는 객체의 구체적인 타입 정보가 정해지지 않은 포인터이다.

```
int value = 100;
void* memory = &value;
```

void*에는 가리키는 객체의 타입 정보가 없으므로 값을 직접 읽거나 변경할 수 없다. 값을 사용하려면 원래 객체 포인터 타입으로 변환해야 한다.

C와 기존 C++ 코드에서는 널 포인터를 나타내기 위해 정수 0이나 NULL을 사용하기도 했다.

```
int* firstPointer = 0;
int* secondPointer = NULL;
```

그러나 0은 정수 리터럴이고, NULL도 구현에 따라 정수 형태의 매크로로 정의될 수 있다. C++11부터는 널 포인터를 명확하게 표현하기 위해 nullptr를 제공한다.

```
int* pointer = nullptr;
```

새로 작성하는 C++ 코드에서는 0이나 NULL 대신 nullptr를 사용한다.

```
int* intPointer = nullptr;
double* doublePointer = nullptr;
```

nullptr의 타입은 <cstddef>에 정의된 std::nullptr_t이다.

```
#include <cstddef>
std::nullptr_t nullValue = nullptr;
```

일반적인 프로그램에서 std::nullptr_t 타입의 변수를 직접 선언하는 경우는 많지 않다. 주로 함수 오버로딩이나 템플릿 코드에서 nullptr 자체를 구분할 때 사용한다. 이에 대한 특징과 내용은 함수 오버로딩을 다루는 단원에서 자세히 살펴본다.

<void와 nullptr 확인>

```
#include <iostream>
using namespace std;

void PrintMessage(/*매개 변수 없으면 void 안씀. 빈 괄호 유지*/)
{
    cout << "반환값이 없는 함수입니다.\n";
}

int main()
{
    PrintMessage();

    int value = 100;
    void* memory = &value;

    cout << "객체의 주소: " << memory << '\n';

    int* pointer = nullptr;
     if(pointer == nullptr)
        cout << "pointer는 널 포인터입니다.\n";
}
```

1.4 C++에서 추가된 편리한 문법과 기능

C++는 C의 기본 문법을 대부분 그대로 사용할 수 있으면서, 같은 작업을 더 간결하고 명확하게 표현할 수 있는 문법과 표준 라이브러리 기능을 추가하였다. 기존 방식이 사라진 것이 아니라 필요에 따라 선택할 수 있는 새로운 작성 방법이 더해진 것이다.

C++는 오랜 기간에 걸쳐 표준이 개정되었으며, C++11 이후에는 대체로 3년 주기로 새로운 표준이 발표되고 있다. 따라서 회사와 프로젝트마다 사용하는 C++ 버전이 다를 수 있다. 이 교재는 C++20을 기준으로 하며, C++98 이후에 추가된 기능은 제목이나 본문에 최초로 표준화된 버전을 표시한다.

이 절에서는 C에서 자주 사용하던 동적 메모리 할당, 제어문, 반복문과 난수 생성을 C++에서는 어떻게 작성할 수 있는지 살펴본다.

1.4.1 {new, delete}, {new[], delete[]} 를 이용한 동적 메모리 할당

C에서는 malloc() 함수로 필요한 크기의 메모리를 할당하고 free() 함수로 해제한다. 할당한 메모리를 0으로 초기화하려면 memset() 함수를 별도로 호출한다.

```c
// C 방식의 동적 메모리 할당과 해제
#include <stdlib.h>
#include <string.h>

void MemoryAllocFree(void)
{
    char* memory = malloc(100 * sizeof(char));
    if (memory != NULL)
    {
        memset(memory, 0, 100);
        free(memory);
    }
}
```

malloc()은 지정한 바이트 수만큼 메모리를 할당하고 void*를 반환한다. C에서는 void*를 다른 객체 포인터 타입에 암시적으로 대입할 수 있으므로 별도의 형 변환이 필요하지 않다.

C++에서도 C 표준 라이브러리의 malloc(), memset()과 free()를 사용할 수 있다. C++는 <stdlib.h>, <string.h>와 같은 C 헤더도 지원하지만, C++ 코드에서는 C 표준 라이브러리 헤더에 c를 붙인 <cstdlib>와 <cstring>을 사용한다.

```c
// C++에서 C 라이브러리를 사용한 동적 메모리 할당과 해제
#include <cstdlib>
#include <cstring>
using namespace std;

void MemoryAllocFree()
{
    char* memory = static_cast<char*>(malloc(100 * sizeof(char)));
    if(memory != nullptr)
    {
        memset(memory, 0, 100);
        free(memory);
    }
}
```

C++에서는 void*를 다른 객체 포인터 타입에 암시적으로 대입할 수 없으므로 static_cast를 이용한 명시적인 형 변환이 필요하다. 또한 널 포인터는 NULL 대신 nullptr(C++11)로 표현한다.

C++에서는 new, new[], delete, delete[] 연산자를 사용하여 객체와 배열을 동적으로 생성하고 해제할 수 있다. malloc(), memset()과 free()는 라이브러리 함수이므로 관련 헤더 파일을 포함해야 하지만, new와 delete는 C++ 언어가 제공하는 연산자이므로 기본적인 사용에는 별도의 헤더 파일이 필요하지 않다.

new[]를 사용하여 char 객체 100개를 동적으로 생성하고 모든 원소를 0으로 초기화해 보자.

```c
char* memory = new char[100]();
```

new char[100]()은 char 객체 100개를 동적으로 생성하고 모든 원소를 0으로 값 초기화한 뒤 첫 번째 원소의 주소를 반환한다. 따라서 C 코드처럼 memset()을 별도로 호출할 필요가 없다.

<new[]와 delete[]를 사용한 동적 메모리 할당과 해제>

```c
void MemoryNewDelete()
{
    // new[] 로 객체 배열 생성, ()를 사용하여 모든 배열 요소를 0으로 초기화.
    char* memory = new char[100]();

    // ... 메모리 사용 ...

    // 메모리 사용 후 delete[] 로 해제.
    delete[] memory;
}
```

기본 new는 메모리를 할당하지 못하면 std::bad_alloc 예외를 발생시킨다. 할당 실패를 예외 대신 nullptr로 확인하려면 <new>에 정의된 std::nothrow를 사용한다.

```
#include <new>
// std::nothrow를 적용, 할당 실패를 nullptr로 확인
void MemoryNewDeleteNothrow()
{
    char* memory = new (std::nothrow) char[100]();
    if(memory != nullptr)
    {
        delete[] memory;
    }
}
```

std::nothrow를 사용한 new는 메모리 할당에 실패하면 nullptr를 반환한다. new의 할당 실패 처리와 std::nothrow의 자세한 사용 방법은 new 연산자 단원에서 살펴본다.

new 연산자로 생성한 단일 객체는 delete로 해제하고, new[]로 생성한 객체 배열은 delete[]로 해제한다.

```
#include <new>
// 단일 객체
int* single = new (std::nothrow) int(200);
if (single != nullptr)
{
    delete single;
}

// 객체 배열
int* values = new (std::nothrow) int[1000]();
if (values != nullptr)
{
    delete[] values;
}
```

new int(200)은 int 객체 하나를 동적으로 생성하고 값 200으로 초기화한다. new int[1000]()은 int 객체 1000개를 동적으로 생성하고 모든 원소를 0으로 값 초기화한다.

동적 할당과 해제 짝 맞추기

동적 할당과 해제는 반드시 서로 대응하는 방식을 사용해야 한다. new로 생성한 단일 객체는 delete, new[]로 생성한 객체 배열은 delete[]로 해제하며, malloc(), calloc(), realloc()으로 할당한 메모리는 free()로 해제한다.

동적 할당		해제
단일 객체	**new**	**delete**
객체 배열	**new[]**	**delete[]**
C 방식 함수	malloc(), calloc(), realloc()	free()

new와 free(), malloc()과 delete처럼 서로 다른 방식을 조합해서는 안 된다. malloc()으로 할당한 메모리는 free()로 해제하고, new와 new[]로 생성한 객체는 각각 delete와 delete[]로 해제한다.

```
int* value = new int(10);
// free(value); // 잘못된 사용
delete value;

int* values = new int[10]();
// delete values; // 잘못된 사용
delete[] values;
```

new와 delete는 C++의 객체 생성과 수명 관리에 연결되는 기본 기능이다. 클래스 객체를 new로 생성하면 생성자가 호출되고, delete로 해제하면 소멸자가 호출된다. 생성자와 소멸자는 클래스 단원에서 자세히 살펴본다.
새로 작성하는 일반적인 C++ 코드에서는 원시 new와 delete를 직접 관리하기보다 std::vector와 같은 표준 컨테이너, 또는 스마트 포인터를 우선 사용한다. 자세한 사용 방법은 이후 단원에서 살펴본다.

1.4.2 제어문에서 변수 선언하기

C에서는 일반적으로 제어문에서 사용할 변수를 먼저 선언하고, 다음으로 조건을 검사한다.

```
int value = GetValue();
if (value > 0)
{
    std::cout << value << '\n';
}
```

C++에서는 if, switch, while의 조건에서 변수를 선언하여 변수의 사용 범위를 제어문 안으로 제한할 수 있다. 또한 C++17부터 if와 switch에 초기화 구문을 별도로 작성할 수 있다.

조건문에서의 선언**(Declaration in condition - C++98)**
if문의 조건 자리에 변수를 선언할 수 있다.

```
if (int value = GetValue())
{
    std::cout << value << '\n';
}
```

GetValue()가 반환한 값으로 value를 초기화한 뒤, value를 bool로 변환한 결과가 참이면 본문을 실행한다. 정수형에서는 0이 거짓이고 0이 아닌 값은 참이다.

포인터를 반환하는 함수에도 당연히 사용할 수 있다.

```
if (Character* character = FindCharacter())
{
    character->Update();
}
```

FindCharacter()가 유효한 포인터를 반환하면 본문을 실행하고, nullptr를 반환하면 실행하지 않는다.

조건에서 선언한 변수는 해당 제어문 안에서만 사용할 수 있다. if문에서 선언한 변수는 연결된 else문에서도 사용할 수 있다.

```
if (int value = GetValue())
{
    std::cout << "값: " << value << '\n';
}
else
{
    // else 문에서도 사용가능
    std::cout << "반환된 값: " << value << '\n';
}

// if-else 문의 범위를 value는 여기서 사용할 수 없음
// std::cout << "value: " << value << '\n';
```

초기자가 있는 **if** 문**(If statement with initializer - C++17)**
C++17부터는 if문의 조건 앞에 초기화 구문을 작성할 수 있다.

```
if (초기화 구문; 조건식)
```

먼저 초기화 구문을 실행하고, 이어서 조건식을 검사한다. 초기화 구문에서 선언한 변수는 조건식, if 본문과 연결된 else문에서 사용할 수 있다.

```
if (int value = GetValue(); value > 0)
{
    std::cout << "양수: " << value << '\n';
}
else
{
    std::cout << "0 또는 음수: " << value << '\n';
}
```

변수를 제어문 밖에 선언하지 않으므로 필요 이상으로 넓은 범위에서 이름이 유지되는 것을 막을 수 있다.

초기자가 있는 **switch**문**(Switch statement with initializer - C++17)**
switch문에도 초기화 구문을 작성할 수 있다.

```
switch (초기화 구문; 조건식)
```

초기화 구문에서 선언한 command는 switch문의 조건과 모든 case 영역에서 사용할 수 있다.

```
switch (int command = GetCommand(); command)
{
case 1:
    std::cout << "시작\n";
    break;
case 2:
    std::cout << "종료\n";
    break;
default:
    std::cout << "알 수 없는 명령\n";
    break;
}
```

while문에서 조건식 내 선언**(Declaration in condition - C++98)**
while문도 조건 자리에 변수를 선언할 수 있다.

```
while (int value = GetReadValue())
{
    std::cout << value << '\n';
}
```

반복할 때마다 ReadValue()의 반환값으로 value를 초기화하고 조건을 검사한다. 값이 0이 되면 반복을 끝낸다.

포인터를 차례로 얻는 코드에도 자주 사용된다.

```
while (Node* node = GetNextNode())
{
    node->Process();
}
```

GetNextNode()가 nullptr를 반환할 때 반복이 끝난다.

while 문 초기자를 아직까지 지원이 안된다. 따라서 다음 문법은 사용할 수 없다.

```
// 오류
// while (int value = GetReadValue(); value > 0)
// {
// }
```

while문에서 초기화와 별도의 조건식이 모두 필요하다면 C 방식으로 반복문 밖에서 변수를 선언하거나 반복문 안에서 값을 갱신하는 방식으로 작성한다.

```
int value = GetReadValue();
while (value > 0)
{
    std::cout << value << '\n';
    value = GetReadValue();
}
```

제어문 안의 변수 선언은 새로운 계산 기능을 만드는 문법이 아니라 변수의 사용 범위를 필요한 영역으로 제한하여 코드를 읽고 관리하기 쉽게 만드는 기능이다.

1.4.3 범위 기반 for문 (Range - for)

C에서 배열의 모든 원소를 순회하려면 인덱스를 이용하는 일반 for문을 사용한다.

```
int values[5] = { 10, 20, 30, 40, 50 };
for (int index = 0; index < 5; ++index)
{
    std::cout << values[index] << '\n';
}
```

C++에서도 이 형태를 그대로 사용할 수 있다. 인덱스가 필요하거나 일부 원소만 순회해야 한다면 일반 for문이 적합하다.

C++에는 배열이나 컨테이너의 모든 원소를 차례로 처리하기 위한 범위 기반 for문이 추가되었다.

범위 기반 **for**문**(Range-based for statement - C++11)**
범위 기반 for문의 기본 문법은 다음과 같다.

```
for (원소 선언 : 범위)
```

배열의 원소를 차례로 출력할 수 있다.

```
int values[] = { 10, 20, 30, 40, 50 };
for (int value : values)
{
    std::cout << value << '\n';
}
```

value에는 배열의 각 원소가 차례로 복사된다. 배열의 크기와 인덱스를 직접 작성하지 않으므로 모든 원소를 순회한다는 의도가 분명하게 드러난다.

std::vector와 같은 표준 컨테이너에도 같은 문법을 사용할 수 있다.

```
#include <vector>
std::vector<int> values = {10, 20, 30, 40, 50};
for (int value : values)
{
    std::cout << value << '\n';
}
```

범위 기반 for문에서 원소를 값으로 받으면 원본 원소가 복사된다. 참조를 이용해 원본을 변경하거나 복사 비용을 줄이는 방법은 참조 단원에서 자세히 살펴본다.

초기화 구문이 있는 범위 기반 **for**문**(C++20)**
C++20부터 범위 기반 for문 앞부분에 초기화 구문을 작성할 수 있다.

```
for (초기화 구문; 원소 선언 : 범위)
```

예를 들어 함수가 반환한 컨테이너를 지역 변수에 저장한 뒤 바로 순회할 수 있다.

```
for (std::vector<int> values = GetValues(); int value : values)
{
    std::cout << value << '\n';
}
```

values는 반복문을 시작하기 전에 한 번 초기화되며, 범위 기반 for문 전체에서 사용할 수 있다. 반복문이 끝나면 for 문 안에 선언한 values의 수명도 끝난다.

초기화 구문에는 반복에 필요한 추가 값을 선언할 수도 있다.

```
for (int index = 0; int value : values)
{
    std::cout << index << ": " << value << '\n';
    ++index;
}
```

여기서 index는 원소를 순회할 때마다 자동으로 증가하지 않으므로 반복문 본문에서 직접 증가시킨다.

C의 일반 for문을 기준으로 보면 C++에서는 세 가지 형태를 사용할 수 있다.

```
// C와 C++의 일반 for문
for (초기화; 조건; 증감)
{
}
// C++11 범위 기반 for문
for (원소 선언 : 범위)
```

```
{
}
// C++20 초기화 구문이 있는 범위 기반 for문
for (초기화 구문; 원소 선언 : 범위)
{
}
```

세 번째 형태는 완전히 별개의 반복문이라기보다 C++11의 범위 기반 for문에 초기화 구문이 추가된 형태이다. 그러나 코드를 작성할 때는 기존 일반 for문 외에 두 가지 범위 기반 작성 형태를 선택할 수 있다.

<범위 기반 for문 확인>

```
#include <iostream>
#include <vector>
using std::cout;

std::vector<int> GetValues()
{
    std::vector<int> values = {11, 22, 33, 44, 55};
    return values;
}

int main()
{
    cout << "C++11 범위 기반 for문\n";
    int values[] = {10, 20, 30, 40, 50};
    for(int value : values)
    {
        cout << value << ' ';
    }

    cout << "\n\n";
    cout << "C++20 초기화 구문이 있는 범위 기반 for문\n";
    for(std::vector<int> dynamicValues = GetValues();
         int value : dynamicValues)
    {
        cout << value << ' ';
    }
    cout << '\n';
}
```

1.4.4 C++ 난수 라이브러리(C++11)

C에서는 rand()로 난수를 얻고 srand()로 난수열의 시작값인 시드(seed)를 지정한다.

```
#include <cstdlib>
#include <ctime>

int range = 1000;
std::srand(static_cast<unsigned int>(std::time(nullptr)));
int rnd = std::rand() % (range + 1);
```

이 코드는 0부터 range 까지의 값을 얻기 위해 % 연산을 사용한다. 그러나 rand()의 품질과 표현 범위는 실행 환경에 따라 달라질 수 있고, % 연산으로 범위를 제한하면 각 값이 정확히 같은 확률로 나오지 않을 수 있다.

C++11부터 <random> 헤더에 난수 엔진과 분포가 추가되었다.

```
#include <random>
std::mt19937 engine(100);
std::uniform_int_distribution<int>  distribution(0, 99);
int value = distribution(engine);
```

C++ 난수 라이브러리는 난수 생성을 두 역할로 나눈다.

구성 요소	역할

난수 엔진	일정한 규칙에 따라 원시 난수열 생성
분포	엔진의 결과를 원하는 범위와 확률 분포로 변환

std::mt19937은 메르센 트위스터 알고리즘을 사용하는 난수 엔진이며, mt19937_64 64비트 난수 엔진이다. 생성자에 전달된 값은 시드 (Seed)이다.

```
std::mt19937 engine(100);
```

같은 엔진과 같은 시드를 사용하면 같은 난수열이 만들어진다. 게임의 오류를 재현하거나 테스트 결과를 반복해야 할 때 유용하다.

실행할 때마다 다른 시드값을 얻기 위해 std::random_device를 사용할 수 있다.

```
std::random_device seed;
std::mt19937 engine(seed());
```

```
// 또는 한줄로
std::mt19937 engine{std::random_device{}()};
```

std::uniform_int_distribution<int>는 지정한 범위의 정수가 균등한 확률로 나오도록 변환한다.

```
std::uniform_int_distribution<int>  dice(1, 6);
int result = dice(engine);
```

정수 분포의 양쪽 끝값은 모두 포함되므로 결과는 1부터 6까지이다.

실수 범위에는 std::uniform_real_distribution을 사용할 수 있다.

```
std::uniform_real_distribution<double> distribution(0.0, 1.0);
double value = distribution(engine);
```

주의 할 점은 std::uniform_real_distribution<double>(min, max) 은 정수형과 다르게 min <= value < max [min, max) 범위의 값을 생성한다.

난수 엔진은 값을 생성할 때마다 내부 상태가 바뀐다. 따라서 난수가 필요할 때마다 엔진을 새로 생성하거나 같은 시드로 다시 초기화하면 같은 값이 반복될 수 있다.

```
std::mt19937 engine{seed()};
for (int count = 0; count < 5; ++count)
{
    int value = distribution(engine);

    std::cout << value << '\n';
}
```

하나의 엔진을 유지하면서 필요한 분포 객체에 전달하는 방식으로 사용한다.
게임에서 같은 결과를 재현해야 한다면 고정된 시드를 사용하고, 실행할 때마다 다른 결과가 필요하다면 실행 환경에서 얻은 시드값을 사용할 수 있다.

<C++ 난수 생성>

```
#include <iostream>
#include <iomanip>
#include <random>
using namespace std;

int main()
{
    std::random_device seed;
    std::mt19937 engine(seed());

    cout << "std::mt19937: [주사위 랜덤]\n";
    std::uniform_int_distribution<int> dice(1, 6);
    for(int count = 0; count < 10; ++count)
    {
        cout << dice(engine) << ' ';
```

```
    }
    cout << "\n\n";

    std::mt19937_64 engine64(seed());

    cout << "std::mt19937_64: [0, 99]\n";
    std::uniform_int_distribution<int> percent(0, 99);
    for(int count = 0; count < 10; ++count)
    {
        cout << percent(engine64) << ' ';
    }
    cout << "\n\n";

    cout << "std::mt19937_64: [0.0 , 1.0]\n";
    cout << fixed << setprecision(7);
    std::uniform_real_distribution<double> distribution(0.0, 1.0);
    for(int count = 0; count < 10; ++count)
    {
        cout << distribution(engine64) << '\n';
    }
    cout << '\n';
}
```

<C, C++ 주요 차이 버전별 정리>

기능	표준 버전
new, delete, new[], delete[]	C++98
조건에서 변수 선언	C++98
if와 switch의 초기화 구문	C++17
범위 기반 for문	C++11
초기화 구문이 있는 범위 기반 for문	C++20
<random> 난수 라이브러리	C++11

C++ 프로젝트의 언어 표준이 낮게 설정되어 있으면 최신 문법을 지원하는 컴파일러에서도 해당 기능을 사용할 수 없다. 다른 회사나 기존 프로젝트의 코드를 작성할 때는 프로젝트가 사용하는 C++ 표준 버전을 먼저 확인해야 한다.

1.4.5 std::sort()를 이용한 정렬

C에서는 배열의 원소를 정렬하려면 정렬 알고리즘을 직접 작성하거나 qsort() 함수를 사용한다. C++에서는 <algorithm>에 정의된 std::sort()를 이용해 배열이나 컨테이너의 원소를 정렬할 수 있다.

std::sort()에는 정렬할 범위의 시작 위치와 마지막 원소 다음 위치를 전달한다.

```
std::sort(시작 위치, 마지막 원소 다음 위치);
```

std::sort 함수는 비교 함수가 없으면 오름차순으로 정렬한다.

```
int values[] = {1, 9, 2, 3, 5, 7, 4, 17};
std::sort(values, values + 8);
```

values는 배열의 첫 번째 원소를 가리키고, values + 8은 마지막 원소 다음 위치를 가리킨다. 원소 개수를 직접 작성하면 배열의 크기가 변경될 때 함께 수정해야 한다.

C++20의 std::span을 사용하면 배열의 원소를 복사하지 않고 배열 전체를 하나의 범위로 표현할 수 있다. 배열로부터 std::span을 생성하면 배열의 크기가 자동으로 결정되며, begin(), end(), size()를 이용할 수 있다.

```
span<int> range(values);
std::sort(range.begin(), range.end());
```

std::sort()는 별도의 정렬 기준을 지정하지 않으면 오름차순으로 정렬한다. 내림차순으로 정렬하려면 <functional>에 정의된 std::greater를 세 번째 인수로 전달한다.

```
std::sort(range.begin(), range.end(), std::greater<int>());
```

```
// 또는 타입을 생략해도 사용할 수 있다.
std::sort(range.begin(), range.end(), std::greater<>());
```

<std::sort 오름차순 내림차순 정렬>

```
#include <iostream>
#include <algorithm>
#include <span>
using std::span;
using std::cout;

int main()
{
    int values[] = {1, 9, 2, 3, 5, 7, 4, 17};

    cout <<"오름 차순 정렬" << '\n';
    span<int> range(values);
    std::sort(range.begin(), range.end());
    for(int v : values)
        cout << v << "  ";

    cout << "\n\n";

    cout <<"내림 차순 정렬" << '\n';
    std::sort(range.begin(), range.end(), std::greater<int>());
    for(int v : values)
        cout << v << "  ";
    cout << '\n';
}
```

```
// 실행 결과:

오름 차순 정렬
1  2  3  4  5  7  9  17

내림 차순 정렬
17  9  7  5  4  3  2  1
```

간단히 기본형으로 구성된 배열의 정렬을 살펴 보았다. 구조체 정렬과 사용자 정의 정렬 기준은 표준 알고리즘 단원에서 자세히 살펴본다.

1.5 학습 정리

C++는 C의 변수, 연산자, 제어문, 함수, 배열, 구조체와 포인터를 대부분 계승한다. 따라서 C를 학습한 독자는 기존 지식을 바탕으로 C++를 배울 수 있다. 그러나 C++는 C의 모든 프로그램을 그대로 받아들이는 완전한 상위 호환 언어는 아니며, 일부 타입 변환과 문법 규칙을 C보다 엄격하게 검사한다.

C++에서는 C에서 이어진 키워드와 함께 namespace, class, nullptr, auto처럼 새로운 언어 기능을 표현하는 키워드를 사용한다. 키워드는 언어에서 특별한 의미로 예약된 단어이므로 변수나 함수 등의 식별자로 사용할 수 없다.

C++에서는 cin, cout과 getline()을 이용해 값을 입력하고 출력할 수 있다. 변수의 타입에 따라 입출력 동작이 결정되므로 형식 지정자를 직접 작성하지 않으며, 일반 변수에 값을 입력할 때 주소 연산자 &를 사용하지 않는다.

C++는 C의 기본형 타입을 대부분 그대로 사용하면서 bool, wchar_t, char8_t, char16_t, char32_t, std::nullptr_t 등의 타입을 제공한다. bool은 논리 상태를 true와 false로 직접 표현하며, 널 포인터는 정수 0이나 NULL보다 nullptr로 표현하는 것이 의도를 명확하게 나타낸다.

정수 리터럴에는 타입을 지정하는 접미사를 사용할 수 있으며, C++14부터는 0b 접두사를 이용해 2진수 리터럴을 작성할 수 있다. 정확한 비트 폭이 필요한 경우에는 <cstdint>에 정의된 고정 폭 정수형을 사용할 수 있다.

부동소수점형은 float, double, long double로 구분하며 리터럴 접미사로 타입을 지정할 수 있다. 부동소수점값은 일부 실수를 정확하게 표현하지 못하고 근사값으로 저장하므로 계산과 비교에서 오차를 고려해야 한다.

C++에서는 new, new[], delete, delete[] 연산자를 이용해 객체와 배열을 동적으로 생성하고 해제할 수 있다. new로 생성한 단일 객체는 delete, new[]로 생성한 객체 배열은 delete[]로 해제해야 하며, malloc(), calloc(), realloc()으로 할당한 메모리는 free()로 해제해야 한다.

C++는 제어문 안에서 변수를 선언할 수 있으며, C++17부터는 if와 switch에 초기화 구문을 작성할 수 있다. 변수를 사용하는 제어문 안으로 유효 범위를 제한하면 불필요하게 오랫동안 변수가 유지되는 것을 막을 수 있다.

C++11부터 배열과 컨테이너의 모든 원소를 순회하는 범위 기반 for문을 사용할 수 있으며, C++20부터는 범위 기반 for문에도 초기화 구문을 작성할 수 있다.

C++11의 <random> 라이브러리는 난수 생성을 엔진과 분포로 나누어 처리한다. std::mt19937과 std::uniform_int_distribution을 이용하면 원하는 범위의 정수를 생성할 수 있으며, 고정된 시드를 사용하면 같은 난수열을 다시 재현할 수 있다.

C++에서는 <algorithm>에 정의된 std::sort()를 이용해 배열이나 컨테이너의 원소를 정렬할 수 있다. 정렬할 범위의 시작 위치와 마지막 원소 다음 위치를 전달하며, 별도의 정렬 기준을 지정하지 않으면 오름차순으로 정렬한다. C++20의 std::span을 사용하면 배열의 원소를 복사하지 않고 하나의 범위로 표현하여 begin(), end(), size()를 이용할 수 있다.

C++에서는 C 방식의 코드를 그대로 작성할 수도 있지만, 추가된 문법과 표준 라이브러리를 이용하면 변수의 범위, 객체의 수명과 자원의 소유 관계를 더 명확하게 표현할 수 있다.

1.6 학습 과제

공통 조건

- 입력에는 cin, 출력에는 cout을 사용한다.
- 난수 생성에는 C++ random을 사용한다. rand()와 srand()는 사용하지 않는다.
- 동적 생성과 해제는 new-delete 만 사용한다.
- 필요한 기능은 적절하게 함수로 분리한다.

과제 **1.** 랜덤 홀짝 맞히기
게임은 총 5판 3선승으로 진행한다.
컴퓨터가 1부터 100 사이의 정수를 무작위로 생성하고, 사용자가 홀수 또는 짝수를 선택하여 결과를 맞히는 프로그램을 작성한다.
사용자는 매 회 다음과 같이 입력한다.
1. 홀수
2. 짝수
매회 컴퓨터가 생성한 숫자와 사용자의 선택을 비교하여 정답 여부를 출력한다.
3번의 승이 5회의 게임이 끝나면 승패를 출력한다.

과제 **2.** 업다운 숫자 맞히기
컴퓨터가 10부터 99 사이의 정수 하나를 무작위로 생성한다. 사용자는 최대 7번 안에 정답을 맞혀야 한다.
사용자가 입력한 값이 정답보다 작으면 더 큰 값입니다., 정답보다 크면 더 작은 값입니다.를 출력한다.
정답을 맞히면 시도 횟수를 출력하고 게임을 종료한다.

과제 **3.** 랜덤 값 저장과 분석
학생 성적에 대한 int형 배열을 10~60 사이 동적으로 생성하되 반드시 new-delete 사용 사용한다.
동적으로 생성된 배열에 랜덤값을 10~99 값을 랜덤으로 채우되, 전부다 다른 값을 채워야 한다.
생성한 결과를 오름 차순으로 정렬하고 모두 출력한다.
또한 값을 계산한다.

- 전체 합계
- 평균값
- 최대값
- 최소값
- 60점 미만인 학생 수
- 60점 이상인 학생 수
- 70점 이상인 학생 수
- 80점 이상인 학생 수
- 90점 이상인 학생 수

가급적 범위 기반 for문을 사용한다.

02. C++ 프로그래밍의 기본

앞 장에서 C++가 C의 문법과 실행 방식을 계승하면서도 절차적 프로그래밍, 객체지향 프로그래밍, 제네릭 프로그래밍, 방대한 표준 라이브러리와 강화된 안전성을 함께 제공하는 언어라는 점을 살펴보았다. 또한 이후 예제를 작성하고 결과를 확인할 수 있도록 cin, cout, getline()의 기본 사용법을 먼저 소개하였다.

C++의 장점은 클래스와 객체지향 프로그래밍에만 있는 것이 아니다. C++로 작성한 절차적 프로그램도 네이티브 코드로 컴파일되며, 포인터와 메모리를 직접 다룰 수 있으므로 C에서 활용하던 실행 성능과 저수준 제어 능력을 그대로 유지할 수 있다. 여기에 C보다 엄격한 타입 검사와 명시적인 형 변환을 적용하여 잘못된 타입 사용을 컴파일 시점에 발견하고, 이름 공간을 이용해 이름 충돌을 방지하며, 참조와 함수 오버로딩을 이용해 함수의 의도를 더욱 명확하게 표현할 수 있다.

또한 다양한 초기화 방식과 상수 표현은 초기화되지 않은 값이나 의도하지 않은 데이터 변경을 줄이고, 표준 입출력 스트림은 값의 타입에 따라 적절한 입출력 방식을 선택한다. 기본 인수, 타입 추론과 여러 함수 확장 기능을 이용하면 반복되는 코드를 줄이고 같은 작업을 C보다 간결하게 작성할 수 있다. 따라서 클래스와 객체를 사용하지 않더라도 C++의 절차적 프로그래밍 기능만으로 C보다 편리하고 안전하면서도 높은 성능을 유지하는 프로그램을 개발할 수 있다.

본 장에서는 이름 공간으로 이름의 범위를 관리하고, 표준 입출력 스트림의 형식과 상태를 제어하며, 현재 실행 환경에서 타입의 크기와 표현 범위를 확인한다. 이어서 문자 인코딩과 부동소수점 오차, 초기화와 상수, 타입을 명확하게 표현하는 기능, 함수의 확장과 참조를 다룬다.

2.1 이름 공간

프로그램의 규모가 커지면 서로 다른 라이브러리나 모듈에서 같은 이름을 사용할 수 있다. 이름 공간(namespace)은 클래스, 함수, 타입과 변수 등, 같은 이름을 별도의 범위로 묶어 충돌을 방지한다.

2.1.1 이름 공간 정의와 범위 지정

Audio와 Graphics 이름 공간에는 이름이 같은 Initialize() 함수가 각각 정의되어 있다.

```
namespace Audio
{
    void Initialize()
    {
    }
}
namespace Graphics
{
    void Initialize()
    {
    }
}
```

두 함수는 서로 다른 이름 공간에 속하므로 함께 정의할 수 있다. 이름 공간에 속한 이름을 사용할 때는 범위 지정 연산자 :: 를 사용한다.

```
Audio::Initialize();
Graphics::Initialize();
```

:: 는 왼쪽의 범위에 속한 이름을 지정한다. 이름 공간뿐 아니라 클래스, 열거형과 전역 범위를 지정할 때도 사용한다.
이름 공간에는 함수뿐 아니라 변수와 타입도 정의할 수 있다.

```
namespace Game
{
    int score = 0;
    struct Player
    {
        int hp;
    };
}

Game::score = 100;
Game::Player player;
player.hp = 100;
```

2.1.2 이름 공간 다시 열기

같은 이름 공간을 여러 위치에서 나누어 정의할 수 있다.

```
namespace Game
{
    void Initialize()
    {
    }
}
namespace Game
{
    void Update()
    {
    }
}
```

두 블록은 모두 같은 Game 이름 공간에 속한다. 헤더 파일에는 선언을 두고 소스 파일에는 정의를 둘 때도 같은 이름 공간을 다시 연다.

```
// Game.h
// 함수 선언.
namespace Game
{
    void Initialize();
    void Update();
}

// Game.cpp
// 함수 구현.
namespace Game
{
    void Initialize(){}
}

// 범위 연산자로 분리해서 구현.
Game::Update()
{
}
```

이미 선언된 이름 공간이 있다면, 이후 같은 이름의 namespace 블록은 새로운 이름 공간을 만드는 것이 아니라 기존 이름 공간에 선언이나 정의를 추가하게 된다.

2.1.3 std 이름 공간과 using 선언

C++ 표준 라이브러리의 이름은 대부분 **std** 이름 공간에 정의되어 있다.

```
std::cout << "C++" << '\n';
std::string text = "Hello";
std::vector<int> values(10);
```

이름 앞에 **std::** 를 작성하면 해당 이름이 표준 라이브러리에 속한다는 사실이 분명하게 드러난다. 같은 이름을 반복해서 사용한다면 using 선언으로 특정 이름만 현재 범위로 가져올 수 있다.

```
using std::cin;
using std::cout;

int value = 0;
cin >> value;
cout << value << '\n';
```

using std::cout; 은 std 이름 공간 전체가 아니라 cout 이라는 이름만 현재 범위에서 사용할 수 있게 한다. 보통 이름의 출처를 분명하게 나타낼 때 **std::** 를 직접 작성하고, 같은 이름을 여러 번 사용하는 짧은 예제에서는 개별 using 선언을 사용한다.

2.1.4 using namespace

using namespace 지시문을 사용하면 지정한 이름 공간의 여러 이름을 범위 지정 없이 사용할 수 있다.

```
using namespace std;
```

```
cout << "Hello" << '\n';
string text = "C++";
```

작성해야 하는 코드는 줄어들지만 이름 공간에 포함된 많은 이름이 현재 범위의 이름 검색 대상에 들어온다. 여러 이름 공간에 같은 이름이 있으면 호출이 모호해질 수 있다.

```
namespace First
{
    void Print()
    {
    }
}
namespace Second
{
    void Print()
    {
    }
}

using namespace First;
using namespace Second;
// Print(); // 어느 Print()인지 결정할 수 없음
```

특히 헤더 파일에서 using namespace를 사용하면 그 헤더를 포함한 모든 소스 파일의 이름 검색에 영향을 준다. 따라서 가급적 헤더 파일에 using namespace std;를 작성하지 않는다.

2.1.5 중첩 이름 공간

이름 공간 안에 다른 이름 공간을 정의할 수 있다.

```
namespace A
{
    namespace B
    {
        namespace C
        {
            void Print()
            {
            }
        }
    }
}
```

각 범위를 ::로 연결하여 사용한다.

```
A::B::C::Print();
```

C++17부터 중첩된 이름 공간을 한 번에 정의할 수 있다.

```
namespace A::B::C
{
    void Update()
    {
    }
}
```

위 코드는 A 안에 B, B 안에 C를 차례로 정의한 것과 같은 의미이다. 중첩 단계가 두 단계로 제한되는 것은 아니며 필요한 만큼 연결할 수 있다.

```
namespace Game::System::Audio::Device
{
    void Initialize()
    {
    }
}
Game::System::Audio::Device::Initialize();
```

2.1.6 이름 공간 별칭과 전역 범위

긴 이름 공간에는 별칭을 지정할 수 있다.

```
namespace AudioDevice = Game::System::Audio::Device;
AudioDevice::Initialize();
```

이름 공간 별칭은 새로운 이름 공간을 만드는 것이 아니라 기존 이름 공간을 가리키는 다른 이름을 제공한다.
이름 앞에 ::만 작성하면 전역 이름 공간을 지정한다.

```
int value = 10;
namespace Game
{
    int value = 20;
    void Print()
    {
        std::cout << value << '\n';   // Game::value
        std::cout << ::value << '\n'; // 전역 value
    }
}
```

현재 범위에 같은 이름이 있더라도 ::value는 전역 범위의 이름을 선택한다.

2.1.7 이름 없는 이름 공간

이름이 없는 이름 공간에 정의한 이름은 현재 번역 단위 안에서만 사용할 수 있다.

```
namespace
{
    int internalValue = 0;
    void Helper()
    {
    }
}
```

다른 소스 파일에서 internalValue나 Helper()를 직접 사용할 수 없다. 소스 파일 내부에서만 사용할 함수와 변수를 제한할 때 사용한다.
번역 단위(translation unit)는 하나의 소스 파일이 전처리된 결과이다. 소스 파일에 포함된 헤더의 내용도 해당 번역 단위에 포함된다.
이름 없는 이름 공간은 소스 파일에서 사용하며 헤더 파일에 정의하면 그 헤더를 포함한 소스 파일마다 별도의 이름이 만들어질 수 있으므로 목적을 분명히 해야 한다.

2.1.8 인라인 이름 공간

인라인 이름 공간은 라이브러리의 버전을 구분하면서 바깥 이름 공간을 통해 내부 이름에 접근할 수 있게 한다.

```
namespace Library
{
    inline namespace V2
    {
        void Print()
        {
        }
    }
}
Library::Print();
Library::V2::Print();
```

V2가 인라인 이름 공간이므로 두 호출은 같은 함수를 가리킨다. 이 기능은 주로 라이브러리의 버전 관리에 사용한다. 일반적인 응용 프로그램에서는 자주 사용하지 않지만 이름 공간 문법을 읽기 위해 알아둘 필요가 있다.

<이름 공간 확인>

```
#include <iostream>
namespace Game::Audio
{
    void Initialize()
    {
        std::cout << "오디오 시스템 초기화\n";
```

```
    }
}
namespace Game::Graphics
{
    void Initialize()
    {
        std::cout << "그래픽스 시스템 초기화\n";
    }
}

namespace GA = Game::Audio;
int main()
{
    GA::Initialize();
    Game::Graphics::Initialize();
}
```

2.2 출력 스트림과 서식 지정

std::cout은 표준 출력 스트림이다. 출력 연산자 "<<" 를 이용하면 문자, 정수, 부동소수점값과 포인터 등 여러 타입의 값을 같은 문법으로 출력할 수 있다.

2.2.1 cout과 출력 연산자

```
#include <iostream>
int main()
{
    int level = 10;
    double speed = 3.5;

    std::cout << level << '\n';
    std::cout << speed << '\n';
}
```

"<<" 를 연속해서 사용하면 문자열과 여러 값을 하나의 출력문으로 연결할 수 있다.

```
std::cout << "Level: " << level << ", Speed: " << speed << '\n';
```

cout은 전달된 값의 타입에 맞는 출력 동작을 선택한다. 따라서 printf()처럼 형식 지정자와 값의 타입을 직접 맞추지 않는다.
나중에 사용자 정의 타입에 출력 연산자를 정의하면 클래스 객체도 같은 형태로 출력할 수 있다.

2.2.2 줄 바꿈과 출력 버퍼

'\n'을 출력하면 출력 위치가 한 줄 아래로 이동한다.

```
std::cout << "first\n";
std::cout << "second\n";
```

std::endl은 줄을 바꾸고 출력 버퍼를 함께 비운다.

```
std::cout << "작업 완료" << std::endl;
```

출력 버퍼는 출력할 데이터를 잠시 저장했다가 일정한 단위로 실제 출력 장치에 전달하는 공간이다. 단순히 줄만 바꿀 때는 '\n'을 사용하고, 화면에 출력된 내용이 즉시 보여야 할 때는 std::flush를 사용할 수 있다.

```
std::cout << "입력 대기 중..." << std::flush;
```

std::unitbuf를 지정하면 각 출력 연산 뒤에 버퍼를 비우며 std::nounitbuf로 해제한다.

```
std::cout << std::unitbuf;
std::cout << "즉시 출력";
std::cout << std::nounitbuf;
```

반복 출력에서 불필요하게 버퍼를 자주 비우면 성능이 떨어질 수 있으므로 목적에 맞게 사용한다.

std::cerr는 오류 메시지를 즉시 확인해야 할 때 사용하고, std::clog는 버퍼를 사용하는 진단 출력에 사용할 수 있다.

```
std::cerr << "파일을 열 수 없습니다.\n";
std::clog << "파일 열기 시도\n";
```

2.2.3 정수의 진법과 접두사

std::dec, std::hex, std::oct는 정수를 각각 10진수, 16진수와 8진수로 출력한다.

```
int value = 255;
std::cout << std::dec << value << '\n';
std::cout << std::hex << value << '\n';
std::cout << std::oct << value << '\n';
```

std::showbase를 사용하면 16진수와 8진수의 접두사를 함께 출력한다.

```
std::cout << std::showbase;
std::cout << std::hex << 255 << '\n'; // 0xff
std::cout << std::oct << 255 << '\n'; // 0377
```

std::uppercase는 16진수 문자와 지수 표기의 문자를 대문자로 출력한다.

```
std::cout << std::uppercase
          << std::hex
          << 255 << '\n'; // 0XFF
```

대응하는 std::nouppercase, std::noshowbase와 std::dec를 사용하면 원래 형식으로 되돌릴 수 있다.

2.2.4 2진수와 비트 단위 출력

표준 출력 스트림에는 std::bin 서식 조정자가 없다. 정수값을 고정된 수의 비트로 출력하려면 <bitset>의 std::bitset을 사용할 수 있다.

```
#include <bitset>
#include <iostream>
unsigned int value = 13;
std::bitset<8> bits(value);
std::cout << bits << '\n';

// 출력:
// 00001101
```

std::bitset<8>의 8은 저장하고 출력할 비트 수이다. 지정한 비트 수보다 큰 값은 하위 비트만 저장된다.

```
unsigned int value = 0b1010'1100;
std::bitset<8> byteBits(value);
std::bitset<16> wordBits(value);
std::cout << byteBits << '\n';
std::cout << wordBits << '\n';

// 출력:
// 10101100
// 0000000010101100
```

각 비트는 오른쪽의 최하위 비트부터 0번으로 센다.

```
std::bitset<8> flags(0b0000'0101);
bool firstBit = flags.test(0);
bool thirdBit = flags.test(2);
flags.set(1);
flags.reset(2);
flags.flip(0);
std::cout << flags << '\n';
```

test()는 비트값을 확인하고, set()은 1로, reset()은 0으로 변경한다. flip()은 해당 비트를 반전한다.

C++20의 std::format을 사용할 수 있는 환경에서는 형식 지정 문자열로 2진수 문자열을 만들 수도 있다.

```
#include <format>
std::string text = std::format("{:08b}", value);
```

std::bitset은 비트 자체를 저장하고 조작할 때 적합하고, std::format은 값을 지정한 출력 형식의 문자열로 만들 때 유용하다.

2.2.5 논리값과 부호 출력

bool 값은 기본적으로 0과 1로 출력된다.

```
bool result = true;
std::cout << result << '\n';
```

std::boolalpha를 사용하면 false와 true로 출력한다.

```
std::cout << std::boolalpha
          << result << '\n';
```

std::noboolalpha로 기본 형식으로 되돌릴 수 있다.
양수 앞에 +를 출력하려면 std::showpos를 사용한다.

```
std::cout << std::showpos
          << 10 << '\n'; // +10
```

std::noshowpos로 해제한다.

2.2.6 필드 폭, 채움 문자와 정렬

std::setw()는 바로 이어서 출력하는 하나의 값에 최소 필드 폭을 지정한다.

```
#include <iomanip>
std::cout << std::setw(6) << 10 << '\n';
std::cout << std::setw(6) << 200 << '\n';
```

std::setw()는 바로 이어지는 출력 하나에만 적용된다. 여러 값에 같은 폭을 적용하려면 각 값 앞에 다시 지정해야 한다.

```
std::cout << std::setw(6) << 10
          << std::setw(6) << 20 << '\n';
```

남는 공간의 문자는 std::setfill()로 지정한다.

```
std::cout << std::setfill('0')
          << std::setw(5)
          << 42 << '\n';
```

std::left와 std::right는 필드 안에서 값을 왼쪽과 오른쪽으로 정렬한다. std::internal은 부호나 진법 접두사는 왼쪽에 두고 값은 오른쪽에 정렬한다.

```
std::cout << std::setfill(' ');
std::cout << std::left
          << std::setw(10) << "Sword"
          << 1200 << '\n';
std::cout << std::right
          << std::setw(10) << "Shield"
          << 900 << '\n';
std::cout << std::internal
          << std::showpos
          << std::setw(8) << 1200 << '\n';
```

2.2.7 부동소수점 출력

std::fixed는 고정 소수점 형식, std::scientific은 지수 형식, std::hexfloat는 16진 부동소수점 형식을 사용한다.

```
double value = 1234.56789;
std::cout << std::fixed << value << '\n';
std::cout << std::scientific << value << '\n';
std::cout << std::hexfloat << value << '\n';
std::defaultfloat는 기본 부동소수점 형식으로 되돌린다.
```

std::setprecision()의 의미는 출력 형식에 따라 달라진다. 기본 형식에서는 전체 유효 자릿수, fixed와 scientific에서는 소수점 이하 자릿수를 지정한다.

```
std::cout << std::setprecision(4)
          << 1234.56789 << '\n';
std::cout << std::fixed
          << std::setprecision(2)
          << 1234.56789 << '\n';
```

std::showpoint는 소수 부분이 없어도 소수점과 뒤쪽 0을 표시한다.

```
std::cout << std::showpoint
          << std::fixed
          << std::setprecision(2)
          << 10.0 << '\n'; // 10.00
```

std::noshowpoint로 해제한다.

2.2.8 서식 조정자의 적용 범위

출력 서식은 한 번의 출력에만 적용되는 설정과 다시 변경할 때까지 유지되는 설정으로 구분된다.

서식	적용 범위
std::setw()	바로 이어지는 값 하나
std::setfill()	다시 변경할 때까지
std::setprecision()	다시 변경할 때까지
std::hex, std::oct, std::dec	다시 변경할 때까지
std::fixed, std::scientific, std::defaultfloat	다시 변경할 때까지
std::boolalpha, std::showbase, std::showpos	해제할 때까지
std::left, std::right, std::internal	다시 변경할 때까지

함수에서 출력 형식을 임시로 변경하면 호출한 곳의 스트림에도 설정이 남는다. 기존 설정을 보존해야 한다면 flags(), precision(), fill()의 값을 저장했다가 복원할 수 있다.

```
auto oldFlags = std::cout.flags();
auto oldPrecision = std::cout.precision();
char oldFill = std::cout.fill();
std::cout << std::hex
          << std::setprecision(4)
          << std::setfill('0');

// 출력 작업
// ...
std::cout.flags(oldFlags);
std::cout.precision(oldPrecision);
std::cout.fill(oldFill);
```

<출력 형식 지정>

```
#include <bitset>
#include <iomanip>
#include <iostream>
using std::cout;
```

```cpp
int main()
{
    int value = 255;
    bool isReady = true;
    double ratio = 1234.56789;

    cout << "진법 출력\n";
    cout << std::showbase;
    cout << std::dec << value << '\n';
    cout << std::hex << value << '\n';
    cout << std::oct << value << "\n\n";

    cout << std::dec << std::noshowbase;

    std::bitset<8> bits(value);
    cout << "2진수 출력\n";
    cout << bits << "\n\n";

    cout << "논리값 출력\n";
    cout << std::boolalpha
         << isReady << "\n\n";

    cout << "필드 폭과 정렬\n";
    cout << std::left
         << std::setw(12) << "Sword"
         << std::right
         << std::setw(6) << 1200 << '\n';

    cout << std::left
         << std::setw(12) << "Shield"
         << std::right
         << std::setw(6) << 900 << "\n\n";

    cout << "부동소수점 출력\n";
    cout << std::fixed
         << std::setprecision(2)
         << ratio << '\n';

    cout << std::scientific
         << std::setprecision(4)
         << ratio << '\n';

    cout << std::defaultfloat;
}
```

2.3 입력 스트림과 상태 관리

std::cin은 표준 입력 스트림이다. 입력 연산자 >>를 이용해 값을 읽으며, 입력한 데이터가 변수의 타입과 맞지 않으면 스트림에 실패 상태가 설정된다.

2.3.1 cin과 입력 연산자

```cpp
int level = 0;
double speed = 0.0;
std::cin >> level >> speed;
```

cin은 변수의 타입에 맞는 입력 동작을 선택하므로 scanf()의 형식 지정자를 작성하지 않는다. 일반 변수의 주소도 전달하지 않는다.

```cpp
scanf("%d", &level); // C
std::cin >> level;   // C++
```

입력 연산자 >>는 기본적으로 앞의 공백 문자를 건너뛰고 값을 읽는다. 공백 문자에는 스페이스, 탭과 줄 바꿈이 포함된다.
문자열을 >>로 읽으면 처음 만나는 공백 전까지만 저장된다.

```cpp
std::string name;
std::cin >> name;
```

사용자가 Kim Min Su를 입력하면 name에는 Kim만 저장된다.

2.3.2 std::string과 getline()

C에서는 문자열을 char 배열이나 char*로 표현한다. 문자열의 길이를 확인하거나 내용을 복사하고 결합하려면 별도의 함수를 사용해야 하며, 배열의 크기와 마지막 널 문자도 직접 관리해야 한다.
C++에서는 일반적인 문자열을 표현할 때 <string>에 정의된 std::string을 사용한다. std::string은 문자열의 길이와 저장 공간을 관리하며, 문자열의 대입, 복사, 비교와 결합을 일반 변수와 비슷한 방식으로 작성할 수 있다.

```
#include <iostream>
#include <string>
using std::cout;
using std::string;
int main()
{
    string name = "Knight";
    string job = "Warrior";
    cout << name << '\n';
    cout << job << '\n';
}
```

문자열의 대입과 결합
std::string은 = 연산자로 문자열을 대입하고 + 연산자로 여러 문자열을 결합할 수 있다. 기존 문자열의 뒤에 내용을 추가할 때는 += 연산자나 append()를 사용한다.

```
string firstName = "Dark";
string lastName = "Knight";
string name = firstName + " " + lastName;
string message = "Player";
message += ": ";
message.append(name);
message.append(3, '!');
cout << message << '\n';

// 출력:
// Player: Dark Knight!!!
```

append()는 문자열의 뒤에 다른 문자열을 추가한다. append(개수, 문자) 형식으로 같은 문자를 여러 번 추가할 수도 있다.

```
string line;
line.append(20, '-');
cout << line << '\n';
```

C 문자열처럼 strcpy()나 strcat()을 사용하거나 문자열을 저장할 배열의 크기를 직접 계산할 필요가 없다.

문자열 비교
std::string은 관계 연산자로 문자열의 내용을 비교할 수 있다.

```
string command = "attack";
if(command == "attack")
    cout << "공격합니다.\n";
else if(command == "defense")
    cout << "방어합니다.\n";
```

C 문자열에 == 연산자를 사용하면 문자열의 내용이 아니라 주소를 비교하지만, std::string에 == 연산자를 사용하면 문자열의 내용을 비교한다. <, >, <=, >= 연산자를 사용하면 사전식 순서로 문자열을 비교할 수 있다.

문자열의 길이와 상태
size() 또는 length()로 문자열의 길이를 확인할 수 있다. 두 함수는 같은 결과를 반환한다.

```
string name = "Knight";
cout << name.size() << '\n';
cout << name.length() << '\n';
```

문자열이 비어 있는지는 empty()로 확인하고, 문자열의 내용을 모두 제거할 때는 clear()를 사용한다.

```
string name;
if(name.empty())
    cout << "이름이 입력되지 않았습니다.\n";
name = "Knight";
name.clear();
```

문자열의 문자 접근
std::string은 배열처럼 인덱스를 사용하여 각 문자에 접근할 수 있다.

```
string name = "Knight";
cout << name[0] << '\n';
name[0] = 'N';
cout << name << '\n';
```

operator[]는 인덱스가 문자열의 범위를 벗어나는지 검사하지 않는다. at()은 범위를 검사하므로 인덱스의 안전성을 확인해야 할 때 사용할 수 있다.

```
cout << name.at(0) << '\n';
```

문자열의 인덱스는 0부터 시작하며, 문자열이 비어 있지 않을 때 유효한 마지막 인덱스는 size() - 1이다. 빈 문자열에서는 첫 번째 문자나 마지막 문자에 접근해서는 안 된다.

문자열 검색과 부분 문자열
find()는 문자열 안에서 문자나 문자열을 검색하고, 발견한 위치를 반환한다. 찾지 못하면 std::string::npos를 반환한다.

```
string itemName = "Iron Sword";
std::size_t position = itemName.find("Sword");
if(position != string::npos)
    cout << "Sword 위치: " << position << '\n';
```

substr()은 문자열의 일부를 새로운 std::string으로 반환한다.

```
string itemName = "Iron Sword";
string type = itemName.substr(5, 5);
cout << type << '\n';
```

substr(시작 위치, 문자 수)에서 두 번째 인수를 생략하면 시작 위치부터 문자열의 끝까지 반환한다.

cin >>를 이용한 문자열 입력
cin >>를 이용하면 공백이 나타나기 전까지 하나의 단어를 입력받는다.

```
string name;
cout << "이름: ";
std::cin >> name;
```

입력값이 "Dark Knight"라면 name에는 "Dark"만 저장된다.

```
Dark Knight
```

게임 명령어, 아이디와 같이 공백이 포함되지 않는 문자열은 cin >>로 입력할 수 있다.

getline()을 이용한 한 줄 입력
이름, 아이템 설명과 대화문처럼 공백을 포함할 수 있는 문자열은 getline()으로 입력한다.

```
string name;
cout << "캐릭터 이름: ";
std::getline(std::cin, name);
cout << "입력한 이름: " << name << '\n';
```

getline()은 입력 스트림에서 줄 바꿈 문자를 만날 때까지 문자를 읽어 std::string에 저장한다. 마지막 줄 바꿈 문자는 입력 스트림에서 제거되지만 문자열에는 저장되지 않는다.

```
캐릭터 이름: Dark Knight
입력한 이름: Dark Knight
```

getline()의 세 번째 인수에는 줄 바꿈 문자 대신 사용할 구분 문자를 지정할 수 있다.

```
string value;
std::getline(std::cin, value, ',');
```

위 코드는 쉼표를 만날 때까지 문자열을 입력받는다.

cin >>와 **getline()**을 함께 사용할 때
cin >>로 값을 입력받은 뒤에는 입력 스트림에 줄 바꿈 문자가 남는다. 이 상태에서 바로 getline()을 호출하면 남아 있는 줄 바꿈 문자를 읽고 빈 문자열을 반환한다.

```
int level;
string name;
cout << "레벨: ";
std::cin >> level;
cout << "이름: ";
std::getline(std::cin, name);
```

간단한 입력에서는 std::ws를 사용하여 앞에 남아 있는 공백과 줄 바꿈 문자를 제거할 수 있다.

```
std::getline(std::cin >> std::ws, name);
```

레벨과 공백이 포함된 캐릭터 이름을 차례로 입력받는 프로그램을 작성할 수 있다.

<std::string과 std::getline() 사용>

```
#include <iostream>
#include <string>
using std::cin;
using std::cout;
using std::getline;
using std::string;
int main()
{
    int level;
    string name;
    cout << "레벨: ";
    cin >> level;
    cout << "캐릭터 이름: ";
    getline(cin >> std::ws, name);
    cout << '\n';
    cout << "이름: " << name << '\n';
    cout << "레벨: " << level << '\n';
}
```

std::ws는 입력 스트림의 앞에 있는 모든 공백 문자를 제거하므로 사용자가 이름 앞에 입력한 공백도 제거된다. 앞쪽 공백을 문자열의 일부로 보존해야 한다면 ignore()를 이용해 이전 입력의 줄 바꿈 문자만 제거해야 한다.

<std::string의 주요 기능>

기능	함수 또는 연산자
문자열 대입	=
문자열 결합	+
문자열 추가	+=, append()
문자열 비교	==, !=, <, >
길이 확인	size(), length()
빈 문자열 확인	empty()
내용 제거	clear()
문자 접근	operator[], at()
문자열 검색	find()

부분 문자열	substr()
한 단어 입력	cin >>
한 줄 입력	getline()

2.3.3 문자 단위 입력

get()은 공백과 줄 바꿈을 포함한 문자 하나를 읽는다.

```
char character = '\0';
std::cin.get(character);
```

반환값을 이용해 입력 성공 여부를 확인할 수도 있다.

```
char character = '\0';
while (std::cin.get(character))
{
    std::cout << character;
}
```

peek()는 입력 스트림의 현재 위치에 있는 문자를 제거하지 않고 확인한다.

```
int next = std::cin.peek();
```

입력할 문자가 없거나 오류가 발생하면 문자값 대신 std::char_traits<char>::eof()와 비교할 수 있는 값을 반환하므로 결과 타입은 int이다.
unget()은 방금 읽은 문자를 다시 입력 스트림으로 돌리고, putback()은 지정한 문자를 되돌린다.

```
char character = '\0';
std::cin.get(character);
std::cin.unget();
```

문자 단위 파서를 만들 때 사용할 수 있지만 일반적인 숫자와 문자열 입력에는 >>와 getline()이 더 단순하다.

2.3.4 공백 처리

입력 연산자 >>는 기본적으로 앞쪽 공백을 건너뛴다. std::noskipws를 지정하면 공백도 일반 문자처럼 읽는다.

```
char character = '\0';
std::cin >> std::noskipws;
while (std::cin >> character)
{
    std::cout << character;
}
```

기본 동작으로 되돌리려면 std::skipws를 사용한다.
std::ws는 현재 위치부터 이어지는 공백 문자를 건너뛴다.

```
std::string name;
std::getline(std::cin >> std::ws, name);
```

std::ws는 스페이스와 탭뿐 아니라 줄 바꿈도 건너뛴다. 사용자가 입력한 앞쪽 공백을 보존해야 한다면 사용하지 않는다.

2.3.5 입력 상태

입력 데이터가 변수의 타입에 맞지 않으면 입력 스트림은 실패 상태가 된다.

```
int value = 0;
std::cin >> value;
```

정수를 입력해야 하는 위치에 abc를 입력하면 값을 읽지 못하고 실패 상태가 설정된다.
입력 연산 자체를 조건식에서 검사할 수 있다.

```cpp
int value = 0;
if (std::cin >> value)
{
    std::cout << "입력한 값: "
              << value << '\n';
}
else
{
    std::cout << "정수를 입력하지 않았습니다.\n";
}
```

<입력 스트림의 상태 확인 함수>

함수	의미
good()	오류 상태가 없음
eof()	입력의 끝에 도달함
fail()	형식 오류 또는 입력 실패
bad()	스트림 장치에서 복구하기 어려운 오류 발생

eof()는 마지막 값을 성공적으로 읽은 직후가 아니라 추가 읽기에서 더 이상 데이터가 없음을 확인했을 때 설정될 수 있다. 따라서 while (!std::cin.eof())보다 입력 연산 자체를 반복 조건으로 사용한다.

```cpp
int value = 0;
while (std::cin >> value)
{
    std::cout << value << '\n';
}
```

2.3.6 clear()와 ignore()

실패 상태가 설정되면 이후 입력도 정상적으로 처리되지 않는다. clear()는 스트림의 상태 플래그를 정상 상태로 되돌린다.

```cpp
std::cin.clear();
```

잘못 입력한 문자가 입력 버퍼에 남아 있으면 이후 입력에서도 같은 문제가 반복된다. ignore()를 사용해 남은 문자를 버릴 수 있다.

```cpp
#include <limits>
std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
```

이 코드는 현재 위치부터 줄 바꿈까지 문자를 버린다.

```cpp
#include <iostream>
#include <limits>
int main()
{
    int value = 0;
    while (true)
    {
        std::cout << "정수를 입력하세요: ";
        if (std::cin >> value)
        {
            break;
        }
        std::cout << "잘못된 입력입니다.\n";
        std::cin.clear();

std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
    std::cout << "입력한 값: " << value << '\n';
}
```

clear()는 상태만 해제하고 ignore()는 버퍼에 남은 문자를 제거한다. 두 함수의 역할을 구분해야 한다.

2.3.7 cin과 getline()의 혼합

cin >>로 값을 읽으면 값을 구분하는 줄 바꿈 문자가 입력 버퍼에 남을 수 있다.

```
int age = 0;
std::string name;

std::cin >> age;
std::getline(std::cin, name);
```

getline()이 남아 있던 줄 바꿈을 즉시 읽으면 name에 빈 문자열이 저장된다. 남은 줄을 제거한 뒤 한 줄을 입력받아야 한다.

```
#include <limits>
std::cin >> age;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
std::getline(std::cin, name);
```

앞쪽 공백을 보존할 필요가 없다면 std::ws를 함께 사용할 수도 있다.

```
std::getline(std::cin >> std::ws, name);
```

ignore()는 현재 줄의 나머지를 명시적으로 제거하고, std::ws는 이어지는 모든 공백을 건너뛴다는 차이가 있다.

<입력 오류 복구>

```
#include <iostream>
#include <limits>
#include <string>
using std::cin;
using std::cout;
using std::string;

int main()
{
    int age = 0;
    while (true)
    {
        cout << "나이를 입력하세요: ";
        if (cin >> age)
        {
            break;
        }
        cout << "정수를 입력해야 합니다.\n";
        cin.clear();
        cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
    cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');

    string name;
    cout << "이름을 입력하세요: ";
    getline(cin, name);
    cout << "이름: " << name << '\n';
    cout << "나이: " << age << '\n';
}
```

2.4 타입의 크기와 범위 확인

C와 C++ 표준은 대부분의 기본형 타입에 대해 정확한 바이트 크기를 정하지 않는다. 대신 각 타입이 표현해야 하는 최소 범위와 타입 사이의 크기 관계를 정하며, 실제 크기와 정렬 방식은 CPU 아키텍처, 운영체제, 컴파일러와 ABI에 따라 달라질 수 있다.

객체의 크기는 byte 단위로 표현하며, C와 C++에서 1 byte는 char 한 개가 차지하는 저장 단위이다. 따라서 sizeof(char)는 항상 1이다. 한 byte를 구성하는 비트 수는 구현에 따라 달라질 수 있으며 <climits> 또는 <limits.h>의 CHAR_BIT로 확인할 수 있다. 표준은 CHAR_BIT가 최소 8 이상임을 보장하며, 일반적인 PC 환경에서는 1 byte가 8 bit이다.

2.4.1 데이터 모델과 기본형의 크기

대표적인 64비트 Windows 환경은 LLP64 데이터 모델을 사용한다. int와 long은 32비트이고, long long과 포인터는 64비트이다. 일반적인 64비트 Linux 환경은 LP64 데이터 모델을 사용한다. int는 32비트이고, long, long long과 포인터는 64비트이다.

타입	표준이 보장하는 최소 폭 또는 조건	일반적인 64비트 환경의 크기
bool	구현에 따라 결정	1바이트
char	최소 8비트, 구현에 따라 결정	1바이트
wchar_t	구현에 따라 결정	Windows 2바이트, macOS·Linux 4바이트
char8_t	unsigned char와 동일	1바이트
char16_t	최소 16비트	2바이트
char32_t	최소 32비트	4바이트
short	최소 16비트	2바이트
int	최소 16비트	4바이트
long	최소 32비트	Windows 4바이트, macOS·Linux 8바이트
long long	최소 64비트	8바이트
float	구현에 따라 결정	4바이트
double	float 이상의 정밀도와 표현 범위	8바이트
long double	double 이상의 정밀도와 표현 범위	Windows 8바이트, macOS·Linux 8, 16바이트
void	객체를 만들 수 없으며 크기가 없음	해당 없음
std::nullptr_t	크기와 정렬이 void*와 동일	8바이트

부호 있는 정수형과 이에 대응하는 부호 없는 정수형은 같은 크기와 정렬을 사용한다.

```
sizeof(int) == sizeof(unsigned int);
sizeof(long) == sizeof(unsigned long);
```

같은 플랫폼과 ABI에서 C와 C++의 대응 기본형은 같은 데이터 모델을 사용한다. 따라서 C와 C++로 작성한 라이브러리가 기본형을 통해 데이터를 주고받을 수 있다. 다만 구조체의 배치나 호출 규약까지 포함한 바이너리 호환성은 컴파일러와 ABI의 규칙도 함께 고려해야 한다.

2.4.2 sizeof와 CHAR_BIT

sizeof를 사용하면 타입이나 객체가 차지하는 바이트 수를 확인할 수 있다.

```
std::cout << sizeof(int) << '\n';
std::cout << sizeof(double) << '\n';
int value = 100;
std::cout << sizeof(value) << '\n';
```

sizeof의 결과 타입은 std::size_t이다.

```
#include <cstddef>
std::size_t byteSize = sizeof(int);
```

sizeof의 피연산자는 일반적으로 실행되지 않는다.

```
int value = 10;
std::size_t size = sizeof(++value);
std::cout << value << '\n'; // 10
```

sizeof(++value)는 표현식의 타입 크기만 확인하므로 value는 증가하지 않는다.
배열에 sizeof를 적용하면 배열 전체의 바이트 수를 얻는다.

```
int values[10];
std::size_t byteSize = sizeof(values);
```

배열이 포인터 매개변수로 전달되면 배열 타입이 유지되지 않는다.

```
void PrintSize(int values[])
{
    std::cout << sizeof(values) << '\n';
}
```

함수의 매개변수에 작성한 int values[]는 int* values로 조정된다. 따라서 이 함수의 sizeof(values)는 배열 전체 크기가 아니라 포인터의 크기를 반환한다.
sizeof(char)는 항상 1이다. 여기서 1은 C++에서 정의한 1바이트를 의미한다. 1바이트를 구성하는 비트 수는 <climits>의 CHAR_BIT로 확인한다.

```
#include <climits>
std::cout << sizeof(char) << '\n';
std::cout << CHAR_BIT << '\n';
```

C++ 표준은 CHAR_BIT가 최소 8 이상임을 보장한다. 대부분의 현대적인 실행 환경에서는 8이다.

2.4.3 alignof

객체는 타입에 맞는 주소에 배치되어야 한다. alignof를 사용하면 타입이 요구하는 정렬 단위를 바이트 수로 확인할 수 있다.

```
std::cout << alignof(char) << '\n';
std::cout << alignof(int) << '\n';
std::cout << alignof(double) << '\n';
```

alignof의 결과 타입도 std::size_t이다.

```
std::size_t alignment = alignof(double);
```

정렬 단위가 4라면 해당 타입의 객체는 일반적으로 4의 배수인 주소에 배치된다. 타입의 크기와 정렬 단위는 같을 수도 있지만 항상 같은 것은 아니다.
alignas를 사용하면 객체나 타입에 더 큰 정렬 조건을 지정할 수 있다.

```
alignas(16) int values[4];
```

이 배열은 최소 16바이트 정렬을 요구한다. 실행 환경이 지원하지 않는 정렬이나 타입의 자연 정렬보다 약한 정렬을 잘못 지정하면 컴파일 오류가 발생할 수 있다.
구조체에서는 멤버의 정렬 조건을 맞추기 위해 멤버 사이 또는 구조체 끝에 패딩이 추가될 수 있다.

```
struct Data
{
    char code;
    int value;
};
```

char가 1바이트이고 int가 4바이트인 환경에서도 sizeof(Data)는 단순히 5가 아닐 수 있다. 구조체의 메모리 배치와 패딩은 클래스의 메모리 구조를 다루는 단원에서 자세히 살펴본다.

2.4.4 std::numeric_limits

<limits>의 std::numeric_limits를 사용하면 타입의 표현 범위와 정밀도를 확인할 수 있다.

```
#include <limits>
std::cout << std::numeric_limits<int>::min() << '\n';
std::cout << std::numeric_limits<int>::max() << '\n';
std::cout << std::numeric_limits<unsigned int>::max() << '\n';
```

부동소수점형에서는 min()의 의미에 주의해야 한다.

```
std::cout << std::numeric_limits<double>::min() << '\n';
std::cout << std::numeric_limits<double>::lowest() << '\n';
std::cout << std::numeric_limits<double>::max() << '\n';
```

std::numeric_limits<double>::min()은 가장 작은 음수가 아니라 0보다 큰 가장 작은 정규화 값을 나타낸다. 가장 작은 유한값은 lowest()로 확인한다.

정밀도와 관련된 정보도 확인할 수 있다.

```
std::cout << std::numeric_limits<float>::digits10 << '\n';
std::cout << std::numeric_limits<double>::digits10 << '\n';
std::cout << std::numeric_limits<double>::max_digits10 << '\n';
```

digits10은 값의 변동 없이 표현할 수 있는 십진 유효 자릿수의 기준이고, max_digits10은 부동소수점값을 문자열로 출력한 뒤 다시 읽었을 때 원래 값을 복원하는 데 필요한 십진 자릿수의 기준이다.

2.4.5 std::size_t와 std::ptrdiff_t

배열의 원소 수와 메모리 크기처럼 음수가 될 수 없는 크기는 std::size_t로 표현한다. sizeof, alignof, std::size()의 결과도 std::size_t이다.

```
#include <cstddef>
#include <iterator>
int values[10];
std::size_t count = std::size(values);
std::size_t byteSize = sizeof(values);
```

count에는 배열의 원소 수인 10이 저장되고, byteSize에는 배열 전체가 차지하는 바이트 수가 저장된다.

std::size_t는 부호 없는 정수형이다. 따라서 감소 반복문에서는 조건을 주의해야 한다.

```
std::size_t count = 10;
// count >= 0은 항상 참이므로 적합하지 않음
// for (std::size_t i = count - 1; i >= 0; --i)
// {
// }
```

두 포인터가 같은 배열의 원소를 가리킬 때 포인터를 빼면 위치 차이를 구할 수 있다. 결과 타입은 std::ptrdiff_t이다.

```
int values[10];
int* first = &values[2];
int* second = &values[7];
std::ptrdiff_t distance = second - first;
```

second - first의 결과는 바이트 수가 아니라 두 포인터의 위치 차이에 해당하는 원소의 개수이다. 위 코드에서 distance는 5가 된다. 포인터 뺄셈은 같은 배열을 가리키는 포인터 사이에서 의미가 있다.

std::ptrdiff_t는 두 위치의 차이를 표현하므로 음수도 저장할 수 있는 부호 있는 정수형이다.

```
std::ptrdiff_t reverseDistance = first - second; // -5
```

C++20의 std::ssize()는 배열이나 컨테이너의 원소 수를 부호 있는 정수형으로 반환한다.

```
#include <iterator>
int values[10];
auto count = std::ssize(values);
```

감소 반복이나 다른 부호 있는 값과 계산해야 할 때 std::size()의 부호 없는 결과보다 사용하기 편리할 수 있다.

<타입의 크기와 범위 확인>

```
#include <climits>
#include <cstddef>
#include <iostream>
#include <iterator>
```

```cpp
#include <limits>
using std::cout;

int main()
{
    cout << "1바이트 비트 수: " << CHAR_BIT << "\n\n";

    cout << "타입의 크기\n";
    cout << "sizeof(char)  : " << sizeof(char) << '\n';
    cout << "sizeof(int)   : " << sizeof(int) << '\n';
    cout << "sizeof(double): " << sizeof(double) << "\n\n";

    cout << "타입의 정렬 단위\n";
    cout << "alignof(char)  : " << alignof(char) << '\n';
    cout << "alignof(int)   : " << alignof(int) << '\n';
    cout << "alignof(double): " << alignof(double) << "\n\n";

    cout << "타입의 표현 범위\n";
    cout << "int 최소값: "
         << std::numeric_limits<int>::min() << '\n';
    cout << "int 최대값: "
         << std::numeric_limits<int>::max() << '\n';
    cout << "double 십진 유효 자릿수: "
         << std::numeric_limits<double>::digits10 << "\n\n";

    int values[10];

    std::size_t count = std::size(values);
    std::size_t byteSize = sizeof(values);

    int* first = &values[2];
    int* second = &values[7];

    std::ptrdiff_t distance = second - first;

    cout << "배열 원소 수: " << count << '\n';
    cout << "배열 전체 크기: " << byteSize << "바이트\n";
    cout << "두 포인터 사이 원소 수: " << distance << '\n';
}
```

타입의 크기를 특정 플랫폼의 값으로 고정해서 가정하기보다 필요한 표현 범위와 사용 목적에 맞는 타입을 선택해야 한다. 파일 형식이나 네트워크 패킷처럼 정확한 비트 폭이 필요한 경우에는 <cstdint>의 고정 폭 정수형을 사용한다.

2.5 문자형과 문자 인코딩

문자형의 크기와 문자 인코딩은 같은 개념이 아니다. 문자형은 인코딩을 구성하는 코드 단위를 저장하고, 문자 인코딩은 문자를 하나 이상의 코드 단위로 변환하는 규칙을 정의한다.

2.5.1 char 계열

char, signed char, unsigned char는 서로 다른 타입이다.

```cpp
char character = 'A';
signed char signedValue = -10;
unsigned char unsignedValue = 200;
```

char는 일반 문자와 문자열을 구성하는 코드 단위를 표현할 때 사용한다. 그러나 char가 반드시 ASCII 문자만 표현하는 것은 아니다. 소스 파일과 실행 환경의 문자 집합 설정에 따라 다른 문자 인코딩의 코드 단위가 저장될 수 있다.

일반 char가 부호 있는 타입처럼 동작하는지 부호 없는 타입처럼 동작하는지는 실행 환경에 따라 달라진다. 작은 정수값의 부호가 중요하다면 signed char 또는 unsigned char를 명시한다.

바이너리 데이터의 한 바이트를 다룰 때는 unsigned char 또는 std::byte를 사용할 수 있다. std::byte는 정수 계산보다 바이트 단위 데이터라는 의도를 분명하게 나타낸다.

```cpp
#include <cstddef>
std::byte data = static_cast<std::byte>(0x7F);
```

std::byte는 중괄호 초기화를 사용하므로 초기화 문법은 해당 단원에서 자세히 살펴본다.

2.5.2 와이드 문자와 Unicode 코드 단위

<C++ 문자형과 문자 리터럴>

표기	타입	용도
'A'	char	일반 문자 코드 단위
L'A'	wchar_t	와이드 문자 코드 단위
u8'A'	char8_t	UTF-8 코드 단위
u'A'	char16_t	UTF-16 코드 단위
U'A'	char32_t	UTF-32 코드 단위

```
char character = 'A';
wchar_t wideCharacter = L'A';
char8_t utf8Character = u8'A';
char16_t utf16Character = u'A';
char32_t utf32Character = U'A';
```

wchar_t의 크기와 인코딩 방식은 실행 환경에 따라 다르다. Windows에서는 일반적으로 16비트이고 UTF-16 코드 단위로 사용된다. 다른 환경에서는 32비트일 수 있다. 따라서 wchar_t가 항상 특정 Unicode 인코딩을 나타낸다고 가정해서는 안 된다.

char8_t는 C++20에서 추가되었으며 UTF-8 코드 단위를 표현한다. char16_t는 UTF-16 코드 단위, char32_t는 UTF-32 코드 단위를 표현한다. Unicode는 각 문자에 코드 포인트(code point)를 부여한다. 문자 인코딩은 하나의 코드 포인트를 하나 이상의 코드 단위(code unit)로 저장한다.

한글 문자 '가'의 코드 포인트는 U+AC00이다.

```
const char8_t utf8Text[] = u8"\uAC00";
const char16_t utf16Text[] = u"\uAC00";
const char32_t utf32Text[] = U"\uAC00";
```

'가'는 UTF-8에서 3개의 코드 단위, UTF-16과 UTF-32에서는 각각 1개의 코드 단위로 표현된다.

```
// char8_t character = u8'가'; // 여러 UTF-8 코드 단위가 필요하므로 오류
char16_t utf16Character = u'\uAC00';
char32_t utf32Character = U'\uAC00';
```

UTF-16에서도 모든 Unicode 코드 포인트가 하나의 char16_t에 저장되는 것은 아니다. 기본 다국어 평면 밖의 코드 포인트는 두 개의 UTF-16 코드 단위인 서로게이트 쌍으로 표현된다.

UTF-32는 하나의 코드 포인트를 하나의 char32_t 코드 단위로 표현할 수 있지만, 사용자가 화면에서 하나의 문자로 인식하는 단위와 코드 포인트 수가 항상 같은 것은 아니다. 결합 문자나 이모지 조합은 여러 코드 포인트로 하나의 표시 문자를 구성할 수 있다.

2.5.3 문자열 배열과 널 문자

문자열 리터럴로 초기화한 배열의 마지막에는 문자열의 끝을 나타내는 널 문자가 포함된다.

```
const char text[] = "ABC";
```

이 배열은 'A', 'B', 'C', '\0'의 네 원소로 구성된다.

```
'A', 'B', 'C', '\0'
```

std::size()를 사용하면 배열의 전체 원소 수를 얻을 수 있다.

```
#include <iterator>

std::size_t count = std::size(text); // 4
```

문자열 인코딩에 사용된 코드 단위 수만 구하려면 널 문자에 해당하는 마지막 원소를 제외한다.

```
std::size_t codeUnitCount = std::size(text) - 1;
```

같은 방법을 UTF 문자열 배열에도 사용할 수 있다.

```
const char8_t utf8Text[] = u8"\uAC00";
const char16_t utf16Text[] = u"\uAC00";
const char32_t utf32Text[] = U"\uAC00";
std::size_t utf8Count = std::size(utf8Text) - 1;
std::size_t utf16Count = std::size(utf16Text) - 1;
std::size_t utf32Count = std::size(utf32Text) - 1;
```

이 값은 코드 단위의 개수이지 사용자에게 보이는 문자 수를 항상 의미하는 것은 아니다.

2.5.4 문자형과 출력 스트림

char 문자열은 cout으로 출력할 수 있다.

```
const char text[] = "Hello";
std::cout << text << '\n';
```

그러나 char8_t, char16_t, char32_t 문자열을 cout에 전달한다고 해당 문자가 자동으로 현재 출력 환경에 맞게 변환되는 것은 아니다.

```
const char8_t text[] = u8"가";
// std::cout << text; // C++20에서는 직접 출력할 수 없음
```

문자열 출력에는 저장된 인코딩, 콘솔이나 파일이 사용하는 인코딩, 필요한 변환 방식이 모두 맞아야 한다. 실제 문자열 변환과 플랫폼별 문자 출력은 문자열 단원에서 자세히 살펴본다.

<문자 인코딩의 코드 단위 수>

```
#include <iostream>
#include <iterator>
using std::cout;

int main()
{
    const char8_t utf8Text[] = u8"\uAC00";
    const char16_t utf16Text[] = u"\uAC00";
    const char32_t utf32Text[] = U"\uAC00";
    cout << "UTF-8  '가' 코드 단위 수: " << std::size(utf8Text) - 1 << '\n';
    cout << "UTF-16 '가' 코드 단위 수: " << std::size(utf16Text) - 1 << '\n';
    cout << "UTF-32 '가' 코드 단위 수: " << std::size(utf32Text) - 1 << '\n';
}

// 실행 결과:
// UTF-8  '가' 코드 단위 수: 3
// UTF-16 '가' 코드 단위 수: 1
// UTF-32 '가' 코드 단위 수: 1
```

2.6 부동소수점형과 오차

부동소수점형은 한정된 비트 안에서 넓은 범위의 실수를 표현한다. 그 대가로 대부분의 값을 근사하여 저장한다. 따라서 부동소수점값을 사용할 때는 표현 오차와 비교 방법을 이해해야 한다.

2.6.1 실수값의 근사 표현

대부분의 실행 환경은 IEEE 754 방식의 2진 부동소수점 표현을 사용한다. 값은 부호, 지수와 유효 숫자에 해당하는 부분으로 나누어 저장된다.
2진수로 유한하게 표현할 수 없는 십진 소수는 가장 가까운 표현 가능 값으로 저장된다. 0.1과 0.2가 대표적인 예이다.

```
double first = 0.1;
double second = 0.2;
double result = first + second;
```

result에는 수학적으로 정확한 0.3이 아니라 그에 매우 가까운 표현 가능 값이 저장될 수 있다.

```
#include <iomanip>
#include <iostream>
std::cout << std::setprecision(17) << result << '\n';
```

일반적인 환경에서는 0.30000000000000004와 비슷한 값이 출력된다.

```
0.30000000000000004
```

부동소수점형의 오차는 연산이 잘못되었다는 뜻이 아니라 한정된 비트로 실수를 근사해서 표현한 결과이다.

2.6.2 == 비교의 의미

부동소수점값에 **==**를 사용하는 것이 항상 잘못된 것은 아니다.

```
double first = 0.5;
double second = first;
bool result = first == second;
```

같은 값을 그대로 복사했거나 2진수로 정확하게 표현되는 값을 비교할 때는 정확한 비교가 목적에 맞을 수 있다.
그러나 여러 연산을 거친 결과가 특정 십진수와 같은지 검사할 때는 오차를 고려해야 한다.

```
double value = 0.1 + 0.2;
bool result = value == 0.3;
```

수학적으로는 같은 값이지만 result가 false가 될 수 있다.

2.6.3 절대 오차와 상대 오차

두 값의 차이에 대한 절대값이 허용 범위 이하인지 검사할 수 있다.

```
#include <cmath>

double first = 0.1 + 0.2;
double second = 0.3;
double tolerance = 1e-9;
bool result =  std::abs(first - second) <= tolerance;
```

이 방법을 절대 오차 비교라고 한다. 비교하는 값의 범위가 일정하거나 0에 가까운 값을 비교할 때 사용하기 쉽다.
값의 크기가 매우 커지면 같은 절대 오차를 모든 값에 적용하기 어려울 수 있다. 이때는 비교하는 값의 크기에 비례한 상대 오차를 사용할 수 있다.

```
#include <algorithm>
#include <cmath>

bool NearlyEqual(double first, double second
                , double absoluteTolerance
                , double relativeTolerance)
{
  if (first == second)
  {
      return true;
  }
  if (!std::isfinite(first) || !std::isfinite(second))
  {
    return false;
  }

  double difference = std::abs(first - second);
  if (difference <= absoluteTolerance)
  {
    return true;
  }

  double scale = std::max(std::abs(first), std::abs(second));
  return difference <= scale * relativeTolerance;
}
```

절대 오차는 0에 가까운 값의 비교를 보완하고, 상대 오차는 값의 크기에 따라 허용 범위를 조정한다.
허용 오차는 계산의 목적과 데이터의 범위에 맞게 정해야 한다. 모든 계산에 하나의 고정된 값을 기계적으로 적용해서는 안 된다.

2.6.4 머신 입실론(Machine Epsilon)

std::numeric_limits<T>::epsilon()은 1과 1보다 큰 값 중 가장 가까운 표현 가능 값의 차이를 반환한다.

```
#include <limits>
double machineEpsilon = std::numeric_limits<double>::epsilon();
```

이 값을 머신 입실론(machine epsilon)이라고 한다. 머신 입실론은 부동소수점형의 정밀도를 나타내는 기준 중 하나이다.

```
std::cout << std::numeric_limits<float>::epsilon()  << '\n';
std::cout << std::numeric_limits<double>::epsilon() << '\n';
```

double의 머신 입실론은 모든 값의 간격을 뜻하지 않는다. 부동소수점값 사이의 간격은 값의 크기에 따라 달라진다. 따라서 머신 입실론을 모든 실수 비교의 허용 오차로 그대로 사용해서는 안 된다.

2.6.5 무한대와 NaN

부동소수점형은 실행 환경에 따라 양의 무한대, 음의 무한대와 NaN(Not a Number)을 표현할 수 있다.

```
#include <limits>

double infinity =  std::numeric_limits<double>::infinity();
double notNumber = std::numeric_limits<double>::quiet_NaN();
```

<cmath>의 함수를 이용해 상태를 확인할 수 있다.

```
#include <cmath>
std::isfinite(infinity);
std::isinf(infinity);
std::isnan(notNumber);
```

NaN은 자신을 포함한 어떤 값과도 같다고 비교되지 않는다.

```
bool result = notNumber == notNumber; // false
```

NaN인지 확인할 때는 ==가 아니라 std::isnan()을 사용한다.

2.6.6 정확한 값이 필요한 데이터

금액이나 고정된 소수 자릿수처럼 값이 정확해야 하는 데이터에는 부동소수점형 사용에 주의해야 한다.

금액을 원 또는 센트 단위의 정수로 저장할 수 있다.

```
int priceInWon = 1250;
long long priceInCent = 125075;
```

이 방식은 소수점 위치를 프로그램의 규칙으로 고정하고 정수 연산을 사용한다. 매우 큰 금액이나 다양한 소수 자릿수가 필요하다면 별도의 고정 소수점 또는 십진 연산 타입을 사용할 수 있다.

<부동소수점 연산과 오차 비교>

```
#include <algorithm>
#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
using std::cout;

bool NearlyEqual(double first, double second
                , double absoluteTolerance
                , double relativeTolerance)
{
    if (first == second)
    {
        return true;
    }
    if (!std::isfinite(first) || !std::isfinite(second))
```

```
    {
        return false;
    }

    double difference = std::abs(first - second);
    if (difference <= absoluteTolerance)
    {
        return true;
    }

    double scale = std::max(std::abs(first), std::abs(second));
    return difference <= scale * relativeTolerance;
}

int main()
{
    double first = 0.1;
    double second = 0.2;
    double expected = 0.3;
    double value = first + second;
    bool exactResult = value == expected;
    bool nearResult =  NearlyEqual(value, expected, 1e-12, 1e-12);

    cout << std::boolalpha;
    cout << std::setprecision(17);
    cout << "0.1 + 0.2: " << value << '\n';
    cout << "0.3      : " << expected << '\n';
    cout << "정확한 비교: " << exactResult << '\n';
    cout << "오차 비교  : " << nearResult << "\n\n";
    cout << "double의 머신 입실론: "
         << std::numeric_limits<double>::epsilon()
         << '\n';
}
```

2.7 초기화와 const

C에서는 주로 대입 연산자와 비슷한 형태로 변수를 초기화한다. C++는 기본형, 배열과 클래스 객체에 사용할 수 있는 여러 초기화 문법을 제공한다. 초기화 방법에 따라 허용되는 변환과 호출되는 생성자가 달라질 수 있으므로 초기화와 대입을 구분해야 한다.

2.7.1 초기화와 대입

초기화는 객체가 만들어질 때 처음 값을 정하는 과정이다.

```
int value = 10;
```

대입은 이미 존재하는 객체의 값을 변경하는 연산이다.

```
value = 20;
```

코드 모양은 비슷하지만 초기화와 대입은 서로 다른 과정이다. 클래스 객체에서는 초기화할 때 생성자가 호출되고, 대입할 때 대입 연산자가 사용될 수 있다.

<C++의 초기화 문법>

```
int first = 10; // 대입 형식 초기화
int second(10); // 직접 초기화
int third{ 10 }; // 중괄호 초기화
```

기본형에서는 세 변수 모두 10으로 초기화된다. 클래스 타입에서는 선택되는 생성자가 달라질 수 있으며, 자세한 내용은 생성자 단원에서 살펴본다.

2.7.2 기본 초기화와 값 초기화

지역 기본형 변수를 초기값 없이 선언하면 값이 정해지지 않는다.

```
int value;
// std::cout << value; // 초기화되지 않은 값 사용
```

초기화되지 않은 값을 읽으면 올바른 결과를 기대할 수 없다.
빈 중괄호를 사용하면 값 초기화가 이루어진다.

```
int count{};
double speed{};
bool alive{};
int* pointer{};
```

기본형은 각각 0, 0.0, false, 널 포인터값에 해당하는 값으로 초기화된다.
배열도 빈 중괄호로 초기화하면 각 원소가 0에 해당하는 값으로 초기화된다.

```
int values[100]{};
```

일부 원소만 지정하면 나머지 원소는 0에 해당하는 값으로 초기화된다.

```
int values[5]{ 10, 20 };
```

<배열에 저장된 값>

```
10, 20, 0, 0, 0
```

2.7.3 중괄호 초기화 (List-initialization) 와 축소 변환 (Narrowing Conversion)

C++11에서 도입된 리스트 초기화(List-initialization)는 중괄호 {}를 사용하는 초기화 문법으로, 균일 초기화(Uniform Initialization)라고도 한다.

중괄호 초기화는 기본형, 배열, 구조체와 클래스 객체에 일관된 형태로 사용할 수 있다.

```
int count{ 10 };
double speed{ 3.5 };
bool enabled{ true };
```

중괄호 초기화는 값이 손실될 수 있는 (Narrowing Conversion)을 허용하지 않는다.

```
// int value{ 3.5 }; // 오류
```

대입 형식 초기화에서는 변환이 허용되지만 소수점 이하 값이 손실된다.

```
int value = 3.5;
```

큰 정수값을 작은 정수형으로 변환하는 경우도 중괄호 초기화가 문제를 발견하는 데 도움이 된다.

```
int value = 1000;
// char code{ value }; // 오류: 실행 시점 값의 축소 변환
```

상수 표현식의 값이 대상 타입으로 정확히 표현될 수 있는 경우에는 일부 정수 변환이 허용될 수 있다.

```
constexpr int value = 100;
char code{ value };
```

중괄호 초기화가 모든 값 손실을 자동으로 방지하는 것은 아니다. 허용된 연산 이후에 발생하는 오버플로나 부동소수점 계산 오차는 별도로 고려해야 한다.
이 절 이후의 예제에서는 중괄호 초기화가 의도를 더 분명하게 나타내는 경우 사용할 수 있다.

2.7.4 const와 변경 제한

const는 객체를 초기화한 이후 그 객체를 통해 값을 변경할 수 없도록 제한한다.

```
const int maxCount{ 100 };
// maxCount = 200; // 오류
```

기본형 const 객체는 선언할 때 값을 초기화해야 한다.

```
// const int count; // 오류
const int count{ 10 };
```

const 자체는 값이 컴파일 타임에 결정되는지 런타임에 결정되는지를 의미하지 않는다. 값이 결정되는 시점은 초기화에 사용한 표현식에 따라 달라진다.

```
const int maxCount{ 100 };          // 컴파일 타임에 값을 결정할 수 있음
const int userCount{ GetCount() };  // 런타임에 값이 결정됨
```

두 객체 모두 초기화 후 변경할 수 없다. 그러나 maxCount는 컴파일 타임에 값을 결정할 수 있는 반면, userCount는 함수 호출 결과에 따라 실행 시점에 값이 결정된다.

C와 C++ 모두 const 객체를 실행 시점에 결정되는 값으로 초기화할 수 있다. 그러나 컴파일 타임에 값을 결정할 수 있는 const 객체를 상수 표현식으로 취급하는 규칙에는 차이가 있다.
C에서 const로 선언한 정수형 객체는 값을 변경할 수 없지만 정수 상수 표현식(integer constant expression)으로 인정되지 않는다.

```
const int SIZE = 128;
```

컴파일러가 SIZE의 값이 128임을 알 수 있더라도 C 언어의 정수 상수 표현식은 아니므로, 파일 범위 배열의 크기나 case 레이블처럼 정수 상수 표현식이 필요한 위치에는 사용할 수 없다.
반면 C++에서는 정수형 또는 열거형 const 객체가 상수 표현식으로 초기화된 경우 정수 상수 표현식으로 사용할 수 있다.

```
const int maxCount{ 100 };
int values[maxCount];
```

그러나 C++에서도 모든 const 객체가 컴파일 타임 상수인 것은 아니다.

```
const int userCount{ GetCount() };
// int values[userCount]; // 오류
```

userCount는 초기화 이후 변경할 수 없지만 컴파일 타임에 값을 결정할 수 없으므로, 표준 C++의 고정 크기 배열처럼 상수 표현식이 필요한 위치에는 사용할 수 없다.

실행 시점에 결정되는 크기의 배열이 필요하다면 const 와 상관 없이, std::vector를 사용할 수 있다.

```
#include <vector>
const int userCount{ GetCount() };
std::vector<int> values(userCount);
```

C++11에서는 컴파일 타임에 사용할 값을 명확하게 표현할 수 있도록 constexpr가 도입되었다.

2.7.5 constexpr

컴파일 시점에 사용할 상수임을 명확하게 나타내려면 constexpr를 사용한다.

```
constexpr int maxCount{ 100 };
int values[maxCount];
```

constexpr 변수는 암시적으로 const이며 초기값을 컴파일 시점에 계산할 수 있어야 한다.

```
int GetCount();
// constexpr int count{ GetCount() }; // 오류
```

함수에도 constexpr를 지정할 수 있다.

```
constexpr int Square(int value)
{
    return value * value;
}
```

constexpr 함수가 상수 표현식으로 사용되면 컴파일 시점에 계산된다.

```
constexpr int size{ Square(10) };
static_assert(size == 100);
```

실행 시점의 값을 전달하면 constexpr 함수도 일반 함수처럼 실행 시점에 계산될 수 있다.

```
int value{};
std::cin >> value;
int result{ Square(value) };
```

const는 초기화 이후 값의 변경을 제한하고, constexpr는 컴파일 시점에 계산할 수 있는 값이나 함수를 표현한다.

2.7.6 consteval과 static_assert

C++20에서는 컴파일 타임 평가와 초기화를 더 명확하게 제어하기 위해 consteval 지정자(consteval specifier)와 constinit 지정자(constinit specifier)가 추가되었다.

consteval을 지정한 함수는 즉시 함수(immediate function)가 되며, 호출 결과가 반드시 컴파일 시점에 계산되어야 한다.

```
consteval int Cube(int value)
{
    return value * value * value;
}
constexpr int result = Cube(3);
```

실행 시점에 결정되는 값을 전달하면 오류이다.

```
int value = 0;
std::cin >> value;
// int result = Cube(value); // 오류
```

static_assert는 컴파일 시점 조건을 검사한다.

```
static_assert(sizeof(int) >= 4);
static_assert(Cube(3) == 27);
```

조건이 거짓이면 컴파일을 중단하고, 설명 문자열을 함께 작성할 수 있다.

```
static_assert(sizeof(void*) == 8, "64비트 환경이 필요합니다.");
```

constexpr 함수는 상수 표현식과 실행 시점 계산에 모두 사용할 수 있지만, consteval 함수는 반드시 컴파일 시점에 계산되어야 한다.

2.7.7 constinit

constinit은 정적 저장 기간(static storage duration) 또는 스레드 저장 기간(thread storage duration)을 가진 변수의 정적 초기화(static initialization)를 보장한다.

```
constinit int globalCount{10};
```

constinit은 변수를 상수로 만들지 않는다.

```
constinit int globalCount{10};
void Update()
{
    globalCount = 20; // 변경 가능
}
```

constinit은 전역 변수나 정적 변수 등이 동적 초기화되는 것을 방지하는 데 사용할 수 있다. 지역 자동 변수에는 사용할 수 없다.

```
void Function()
{
    // constinit int value = 10; // 오류
}
```

const는 변경 제한, constexpr는 상수 표현식 사용 가능성, consteval은 반드시 컴파일 시점에 평가되는 함수, constinit은 정적 초기화를 각각 나타낸다.

2.7.8 포인터와 const

포인터에서는 포인터가 가리키는 값을 변경할 수 없는 경우와 포인터 자체를 변경할 수 없는 경우를 구분한다.

```
int first{ 10 };
int second{ 20 };
const int* pointerToConst{ &first };
int* const constPointer{ &first };
const int* const constPointerToConst{ &first };
```

선언	가리키는 값 변경	다른 주소 대입
const int* pointerToConst	불가능	가능
int* const constPointer	가능	불가능
const int* const constPointerToConst	불가능	불가능

pointerToConst는 다른 int 객체를 가리킬 수 있지만 포인터를 통해 값을 변경할 수 없다.

```
pointerToConst = &second;
// *pointerToConst = 30; // 오류
```

constPointer는 가리키는 값을 변경할 수 있지만 다른 주소를 저장할 수 없다.

```
*constPointer = 30;
// constPointer = &second; // 오류
```

constPointerToConst는 포인터가 가리키는 값과 포인터 자체를 모두 변경할 수 없다.
포인터 선언은 이름에서 가까운 부분부터 읽으면 구분하기 쉽다.

```
- const int*       : const int를 가리키는 포인터
- int* const       : int를 가리키는 const 포인터
- const int* const : const int를 가리키는 const 포인터
```

상수 참조를 함수 매개변수에 사용하는 방법은 2.10.3절에서 살펴보았다. 클래스의 상수 객체와 상수 멤버 함수는 클래스 단원에서 자세히 다룬다.

<초기화와 상수 확인>

```
#include <iostream>
using std::cout;

int GetCount()
{
    return 5;
}
constexpr int Square(int value)
{
    return value * value;
}

int main()
{
    int first{};
    int values[5]{ 10, 20 };
    const int runtimeCount{ GetCount() };
    constexpr int compileTimeCount{ Square(4) };
    int number{ 10 };
    const int* pointerToConst{ &number };

    cout << "값 초기화: " << first << '\n';
    cout << "배열: ";
    for (int value : values)
    {
```

```
        cout << value << ' ';
    }
    cout << '\n';
    cout << "실행 시점 const       : "  << runtimeCount << '\n';
    cout << "컴파일 타임 constexpr : "  << compileTimeCount << '\n';
    cout << "포인터가 가리키는 값   : "  << *pointerToConst << '\n';
}
```

2.8 타입 표현과 안전성

C++는 C의 저수준 제어 기능을 유지하면서도 변환의 의도와 타입의 의미를 코드에 명확하게 표현하는 기능을 제공한다. 이 절에서는 C++ 형 변환 연산자, enum class, auto, decltype과 타입 별칭을 살펴본다.

2.8.1 C++ 형 변환 연산자

C 스타일 형 변환은 서로 다른 목적의 변환을 같은 문법으로 표현한다.

```
double speed{ 10.5 };
int value = (int)speed;
```

C++는 변환의 목적에 따라 네 가지 형 변환 연산자를 제공한다.

```
static_cast
```

static_cast는 관련된 타입 사이의 일반적인 변환에 사용한다.

```
double speed{ 10.5 };
int value{ static_cast<int>(speed) };
```

실수에서 정수로 변환하면 소수점 이하 값이 버려진다. static_cast는 변환을 명시적으로 표현하지만 값 손실을 자동으로 막아 주지는 않는다. 열거형을 정수형으로 변환하거나 void*를 원래 객체 포인터 타입으로 변환할 때도 사용할 수 있다.

```
void* memory{};
int* pointer{static_cast<int*>(memory)};
```

기본 클래스와 파생 클래스 사이의 변환에도 사용할 수 있지만, 안전한 다운캐스팅이 필요한 경우에는 클래스 관계와 실제 객체 타입을 고려해야 한다.

```
const_cast
```

const_cast는 포인터나 참조 타입의 상수성을 추가하거나 제거할 때 사용한다.

```
int value{ 10 };
const int* constPointer{ &value };
int* pointer{const_cast<int*>(constPointer)};
*pointer = 20;
```

원래 객체 value가 상수가 아니므로 값을 변경할 수 있다.
처음부터 const로 정의한 객체의 상수성을 제거한 뒤 값을 변경하면 정의되지 않은 동작이 된다.

```
const int value{ 10 };
const int* constPointer{ &value };
int* pointer{const_cast<int*>(constPointer)};
// *pointer = 20; // 사용해서는 안 됨
```

const_cast가 필요하다는 것은 함수나 라이브러리의 인터페이스가 상수성을 올바르게 표현하지 못하고 있을 가능성도 있으므로 사용 전에 설계를 먼저 확인한다.

```
reinterpret_cast
```

reinterpret_cast는 포인터나 정수의 저수준 표현을 다른 타입으로 해석할 때 사용한다.

```cpp
#include <cstdint>
int value{ 10 };
int* pointer{ &value };
std::uintptr_t
address{reinterpret_cast<std::uintptr_t>(pointer)};
```

이 변환은 값의 의미를 유지하는 일반적인 형 변환이 아니다. 운영체제 API, 하드웨어 제어, 메모리 형식 분석과 같은 제한된 저수준 코드에서 사용한다.
reinterpret_cast로 변환할 수 있다고 해서 변환한 포인터를 통해 해당 타입의 객체처럼 안전하게 접근할 수 있다는 뜻은 아니다. 객체의 실제 타입, 정렬, 수명과 별칭 규칙을 함께 고려해야 한다.

```cpp
dynamic_cast
```

dynamic_cast는 다형성을 사용하는 클래스 계층에서 실행 시점의 실제 객체 타입을 확인하며 변환할 때 사용한다.

```cpp
// Derived* derived{
//     dynamic_cast<Derived*>(basePointer)
// };
```

포인터 변환이 실패하면 nullptr를 반환하고, 참조 변환이 실패하면 예외가 발생한다. 상속과 가상 함수를 먼저 이해해야 하므로 자세한 사용법은 상속과 다형성 단원에서 살펴본다.
C++ 형 변환 연산자는 변환을 자동으로 안전하게 만드는 기능이 아니다. 어떤 목적의 변환인지 코드에 드러내어 검토하기 쉽게 만드는 기능이다.

2.8.2 enum class

기존 열거형의 열거자 이름은 열거형이 선언된 범위로 들어오며 정수형으로 암시적으로 변환될 수 있다.

```cpp
enum State
{
    Idle,
    Running
};
State state{ Idle };
int value{ Running };
```

서로 다른 열거형에서 같은 열거자 이름을 사용하면 충돌할 수 있다.

```cpp
// enum CharacterState
// {
//     Idle,
//     Running
// };
//
// enum NetworkState
// {
//     Idle,
//     Connected
// };
```

enum class는 열거자 이름을 열거형의 범위 안에 두고 정수형으로의 암시적 변환을 허용하지 않는다.

```cpp
enum class State
{
    Idle,
    Running,
    Finished
};
State state{ State::Idle };
// int value{ State::Running }; // 오류
```

정수값이 필요하면 변환을 명시한다.

```cpp
int value{static_cast<int>(State::Running)};
```

서로 다른 열거형에서 같은 열거자 이름도 사용할 수 있다.

```
enum class CharacterState
{
    Idle,
    Running
};
enum class NetworkState
{
    Idle,
    Connected
};
```

열거형의 기저 정수형을 지정할 수도 있다.

```
enum class State : unsigned char
{
    Idle,
    Running,
    Finished
};
```

C++20부터 using enum으로 특정 열거형의 열거자 이름을 현재 범위로 가져올 수 있다.

```
using enum State;
State first = Idle;
State second = Running;
```

범위 지정이 반복되는 코드를 줄일 수 있지만, 여러 열거형의 이름이 섞이면 출처가 불분명해질 수 있으므로 좁은 범위에서 사용한다.
enum class는 상태나 종류를 일반 정수와 구분된 타입으로 다룰 수 있게 한다. 함수의 매개변수에 사용하면 허용되는 값의 종류도 분명해진다.

```
void ChangeState(State state);
```

2.8.3 auto

auto를 사용하면 초기값으로부터 변수의 타입을 추론할 수 있다.

```
auto count{ 10 };      // int
auto speed{ 3.5f };    // float
auto enabled{ true };  // bool
```

auto는 타입을 없애는 기능이 아니다. 컴파일러가 초기값을 바탕으로 타입을 결정하며, 결정된 타입은 실행 중에 변경되지 않는다.

```
auto value{ 10 };
// value = "Hello"; // 오류
```

auto만 사용하면 참조와 최상위 const가 제거된 값 타입이 추론된다.

```
const int original{ 10 };
auto copiedValue = original; // int
```

참조가 필요하면 auto&를 사용한다.

```
int value{ 10 };
auto copiedValue = value;
auto& reference = value;
const auto& constReference = value;
```

copiedValue 를 변경해도 value는 바뀌지 않지만, reference를 변경하면 value가 변경된다. constReference를 통해서는 값을 변경할 수 없다.
중괄호를 사용할 때는 형태에 따라 추론 결과가 달라질 수 있다.

```
auto first{ 10 };       // int
auto second = { 10 };   // std::initializer_list<int>
```

```
auto third = { 10, 20 };// std::initializer_list<int>
```

직접 목록 초기화에서 원소가 하나이면 해당 원소의 타입을 추론하고, =와 중괄호를 함께 사용하면 std::initializer_list가 추론될 수 있다. 원소 타입이 서로 다르면 추론할 수 없다.

```
// auto values = { 10, 3.5 }; // 오류
```

표준 라이브러리의 반복자처럼 타입 이름이 길고 복잡할 때 auto가 유용하다.

```
#include <vector>
std::vector<int> values(10);
auto iter = values.begin();
```

타입 자체가 코드의 의미를 전달하는 경우에는 직접 작성하는 편이 읽기 쉬울 수 있다.

```
int playerCount{ 10 };
```

모든 지역 변수를 무조건 auto로 선언하기보다 초기값만으로 타입과 의미를 분명하게 알 수 있는지 고려한다.

2.8.4 decltype

decltype은 변수나 표현식의 타입을 얻는다.

```
int value{ 10 };
decltype(value) otherValue{ 20 };
```

decltype(value)는 변수 value의 선언된 타입인 int가 된다.
변수 이름을 괄호로 묶으면 표현식의 값 범주가 반영되어 결과가 달라질 수 있다.

```
int value{ 10 };

decltype(value) first{ 20 };
decltype((value)) second{ value };
```

decltype(value)는 int이고, decltype((value))는 좌측값 표현식을 반영하여 int&가 된다.

decltype에 일반 표현식을 전달하면 해당 표현식의 결과 타입을 얻을 수 있다.

```
double ratio{ 3.5 };
decltype(value + ratio) result{ value + ratio };
```

value + ratio의 결과 타입이 double이므로 result도 double이다.

auto는 초기값으로부터 새 변수의 타입을 추론할 때 사용하고, decltype은 특정 표현식의 타입을 코드에서 그대로 사용해야 할 때 활용한다.
함수의 반환 타입에서 표현식의 참조와 const까지 그대로 유지해야 할 때는 decltype(auto)를 사용할 수 있다.

```
decltype(auto) GetValue(int& value)
{
    return (value);
}
```

괄호에 따라 반환 타입이 달라질 수 있으므로 decltype(auto)는 의도를 정확하게 이해하고 사용해야 한다.

2.8.5 using을 이용한 타입 별칭

C에서는 typedef를 이용해 기존 타입에 다른 이름을 부여한다.

```
typedef unsigned int UInt;
typedef void (*Callback)(int);
```

C++에서도 typedef를 사용할 수 있지만 using은 별칭과 원래 타입의 관계를 왼쪽에서 오른쪽으로 읽을 수 있다.

```
using UInt = unsigned int;
```

```
using Callback = void(*)(int);
```

복잡한 표준 라이브러리 타입에도 별칭을 지정할 수 있다.

```
#include <vector>
using ScoreList = std::vector<int>;
ScoreList scores(3);
```

타입 별칭은 새로운 타입을 만드는 것이 아니다.

```
UInt first{ 10U };
unsigned int second{ first };
```

UInt와 unsigned int는 같은 타입이다. 서로 구분되는 새로운 타입이 필요하다면 구조체나 클래스를 정의해야 한다.
함수 포인터 별칭은 typedef보다 using 문법이 읽기 쉽다.

```
using Callback = void(*)(int);
void OnEvent(int value)
{
    std::cout << value << '\n';
}
Callback callback{ OnEvent };
```

템플릿 별칭을 사용하면 템플릿 타입의 일부를 고정한 새로운 별칭을 만들 수 있다. 자세한 내용은 템플릿 단원에서 살펴본다.

<타입 표현과 안전성 확인>

```
#include <iostream>
#include <vector>
using std::cout;

enum class State : unsigned char
{
    Idle,
    Running,
    Finished
};
using ScoreList = std::vector<int>;

int main()
{
    double speed{ 10.5 };
    int convertedSpeed{static_cast<int>(speed)};
    State state{ State::Running };
    int stateValue{static_cast<int>(state)};
    ScoreList scores{ 100, 200, 300 };
    auto count{ scores.size() };
    const auto& firstScore{ scores[0] };

    cout << "변환한 속도   : "  << convertedSpeed << '\n';
    cout << "상태의 정수값 : "  << stateValue << '\n';
    cout << "점수 개수     : "  << count << '\n';
    cout << "첫 번째 점수 : "  << firstScore << '\n';
}
```

2.9 함수 기능의 확장

C++는 C의 함수 문법을 계승하면서 함수 오버로딩, 기본 인수, 인라인 함수와 반환 타입 추론 같은 기능을 추가하였다. 이러한 기능을 이용하면 같은 역할의 함수를 일관된 이름으로 구성하고, 호출 코드를 간결하게 작성하며, 타입에 따라 함수의 결과를 표현할 수 있다.

2.9.1 함수 오버로딩

매개변수의 타입이나 개수가 다르면 같은 이름의 함수를 여러 개 정의할 수 있다. 이를 함수 오버로딩(function overloading)이라고 한다.

```
int Add(int left, int right)
{
```

```
    return left + right;
}
double Add(double left, double right)
{
    return left + right;
}
int Add(int first, int second, int third)
{
    return first + second + third;
}
```

호출할 때 전달한 인수의 개수와 타입에 따라 적절한 함수가 선택된다.

```
int firstResult = Add(10, 20);
double secondResult = Add(1.5, 2.5);
int thirdResult = Add(10, 20, 30);
```

C에서는 타입마다 서로 다른 함수 이름을 사용해야 하는 경우가 많았다.

```
int AddInt(int left, int right);
double AddDouble(double left, double right);
```

C++에서는 같은 목적의 기능을 하나의 이름으로 구성할 수 있다.
함수를 선택할 때는 정확히 일치하는 타입, 정수 승격, 일반적인 표준 변환 등의 순서가 고려된다.

```
void Print(int value);
void Print(double value);

Print(10);   // Print(int)
Print(3.5);  // Print(double)
```

여러 함수가 같은 정도로 적합하면 호출이 모호해질 수 있다.

```
void Process(long value);
void Process(double value);
// Process(10); // 두 변환의 우선순위가 같아 모호함
```

호출 의도를 분명하게 하려면 인수의 타입을 명시적으로 맞출 수 있다.

```
Process(static_cast<long>(10));
```

반환 타입만 다른 함수는 오버로딩할 수 없다.

```
// int GetValue();
// double GetValue();
```

함수 호출만으로는 어떤 반환 타입을 원하는지 결정할 수 없기 때문이다.

<함수 오버로딩 확인>

```
#include <iostream>
using std::cout;

int Add(int left, int right)
{
    return left + right;
}
double Add(double left, double right)
{
    return left + right;
}
int Add(int first, int second, int third)
{
    return first + second + third;
}

int main()
{
```

```
    cout << "Add(int, int)      : "  << Add(10, 20) << '\n';
    cout << "Add(double, double): "  << Add(1.5, 2.5) << '\n';
    cout << "Add(int, int, int) : "  << Add(10, 20, 30) << '\n';
}
```

2.9.2 기본 인수

함수의 매개변수에 기본값을 지정할 수 있다. 호출할 때 해당 인수를 생략하면 기본값이 사용된다.

```
void PrintMessage(const char* message, int count = 1)
{
    for (int i = 0; i < count; ++i)
    {
        std::cout << message << '\n';
    }
}
PrintMessage("Hello");
PrintMessage("Hello", 3);
```

첫 번째 호출에서는 count에 기본값 1이 사용되고, 두 번째 호출에서는 전달한 값 3이 사용된다.

기본 인수는 오른쪽 매개변수부터 연속해서 지정해야 한다.

```
void Move(int x, int y = 0, int z = 0);
```

기본값이 있는 매개변수 뒤에 기본값이 없는 매개변수를 둘 수 없다.

```
// void Move(int x = 0, int y); // 오류
```

본 교재에서는 함수 선언과 정의를 분리할 때 기본 인수를 함수 선언에만 작성하고 정의에서는 반복하지 않는다. 호출하는 코드가 함수 선언을 통해 기본값을 확인할 수 있기 때문이다.

```
// 함수 선언에서 기본인수 값 설정.
void PrintMessage(const char* message, int count = 1);

// 함수 구현에서는 기본 인수 값을 설정하지 않음.
void PrintMessage(const char* message, int count)
{
    for (int i = 0; i < count; ++i)
    {
        std::cout << message << '\n';
    }
}
```

기본 인수에 사용된 표현식은 함수를 호출할 때 평가된다.

```
int GetDefaultCount();
void PrintMessage(const char* message,  int count = GetDefaultCount());
```

기본 인수와 오버로딩을 함께 사용할 때는 호출이 모호해지지 않도록 해야 한다.

```
void Print(int value);
void Print(int value, int count = 1);
// Print(10); // 두 함수가 모두 호출 후보가 됨
```

2.9.3 인라인 함수

C의 함수형 매크로는 전처리 과정에서 전달한 식을 문자 단위로 치환한다.

```
#define SQUARE(value) value * value
int result = SQUARE(1 + 2);
```

전처리 후에는 다음과 같은 식이 만들어진다.

```
int result = 1 + 2 * 1 + 2;
```

연산자 우선순위 때문에 결과는 9가 아니라 5가 된다.
매크로 정의에 괄호를 추가할 수 있지만 전달한 식이 여러 번 평가되는 문제는 남을 수 있다.

```
#define MAXIMUM(a, b) ((a) > (b) ? (a) : (b))
int first = 10;
int second = 20;

// 인수의 식이 여러 번 평가될 수 있음
int result = MAXIMUM(++first, ++second);
```

C++에서는 같은 목적의 계산을 함수로 작성할 수 있다.

```
inline int Square(int value)
{
    return value * value;
}
```

인라인 함수는 일반 함수이므로 매개변수와 반환값의 타입을 검사하고, 인수는 함수 호출 규칙에 따라 한 번 평가된다.

inline을 지정했다고 컴파일러가 반드시 함수 본문을 호출 위치에 펼치는 것은 아니다. 실제 인라인 최적화 여부는 함수의 크기, 복잡도, 호출 위치와 최적화 설정 등을 고려하여 컴파일러가 결정한다. 반대로 inline을 지정하지 않은 함수도 컴파일러가 인라인 처리할 수 있다.

C++에서 inline이 언어적으로 보장하는 중요한 의미는 동일한 함수 정의를 여러 번역 단위에 둘 수 있게 하는 것이다. 따라서 헤더 파일에 함수 정의를 작성해야 할 때 사용한다.

```
inline int Add(int left, int right)
{
    return left + right;
}
```

클래스 정의 안에서 본문까지 작성한 멤버 함수는 암시적으로 인라인 함수가 된다.

```
class Counter
{
public:
    int GetValue() const
    {
        return value;
    }

private:
    int value = 0;
};
```

짧은 함수라고 모두 inline을 직접 붙일 필요는 없다. 실행 성능을 위한 인라인 최적화는 컴파일러가 결정하게 하고, inline은 헤더 파일에 정의를 둘 때의 언어 규칙으로 이해하는 것이 좋다.

2.9.4 함수의 반환 타입 추론

C++14부터는 auto를 사용하여 함수의 return 문으로부터 반환 타입을 추론할 수 있다.

```
auto Add(int left, int right)
{
    return left + right;
}
```

위 함수의 반환 타입은 int로 추론된다.
반환문이 여러 개라면 모든 반환값의 타입이 같아야 한다.

```
auto GetValue(bool condition)
{
    if (condition)
    {
```

```
        return 10;
    }
    return 20;
}
```

반환문의 타입이 int와 double로 달라지면 반환 타입을 하나로 추론할 수 없어 오류가 발생한다.

```
// auto GetValue(bool condition)
// {
//     if (condition)
//     {
//         return 10;   // int
//     }
//     return 20.0;     // double
// }
```

반환값이 없는 함수에서도 auto는 void로 추론될 수 있다.

```
auto Print()
{
    std::cout << "Hello" << '\n';
}
```

다만 반환값이 없다는 의도를 명확하게 나타내기 위해 일반 함수에서는 void를 직접 작성하는 편이 읽기 쉽다.
함수 이름과 매개변수 목록 뒤에 반환 타입을 작성하는 후행 반환 타입(trailing return type)도 사용할 수 있다.

```
auto Multiply(double left, double right) -> double
{
    return left * right;
}
```

후행 반환 타입은 반환 타입이 매개변수 표현식에 의존하는 템플릿에서 유용하다.

```
template <typename T, typename U>
auto Add(T left, U right) -> decltype(left + right)
{
    return left + right;
}
```

C++14 이후에는 위 템플릿도 단순히 auto로 작성할 수 있지만, 후행 반환 타입은 반환 타입을 명시적으로 표현하거나 복잡한 타입 관계를 나타낼 때 여전히 사용된다.

일반 함수에서는 반환 타입을 직접 작성하는 편이 더 명확한 경우가 많다. 반환 타입 추론은 타입이 길거나 템플릿 표현식에 따라 결정될 때 선택적으로 사용한다.

2.10 참조

참조(reference)는 기존 객체에 붙인 다른 이름이다. 포인터와 마찬가지로 기존 객체에 접근할 수 있지만 선언과 사용 방법, 표현하는 의미가 다르다.

2.10.1 참조의 개념

참조는 선언할 때 참조할 대상을 지정해야 한다.

```
int value = 10;
int& reference = value;
```

reference는 별도의 정수 객체가 아니라 value를 나타내는 다른 이름이다.

```
reference = 20;
std::cout << value << '\n'; // 20
```

일반적인 좌측값 참조는 초기화한 뒤 다른 객체를 참조하도록 변경할 수 없다.

```
int first = 10;
int second = 20;
int& reference = first;
reference = second;
```

마지막 대입은 참조 대상을 second로 바꾸는 것이 아니다. second의 값 20을 참조 대상인 first에 대입한다.

참조 자체를 사용하기 위해 * 연산자를 작성할 필요가 없으며, 호출할 때 주소 연산자 &도 작성하지 않는다.

<포인터와 참조의 기본 차이>

구분	포인터	참조
선언 후 대상 변경	가능	불가능
널 상태 표현	nullptr 사용 가능	일반적으로 유효한 객체에 연결
대상 접근	*pointer	일반 변수와 같은 문법
호출 시 주소 전달	&value	value

참조도 내부 구현에서는 주소를 이용할 수 있지만, C++ 문법에서 참조는 객체의 별칭으로 다룬다.

2.10.2 값, 주소와 참조에 의한 전달

값을 매개변수로 전달하면 함수의 매개변수에 복사본이 만들어진다.

```
void IncreaseValue(int value)
{
    ++value;
}
```

함수 내부에서 복사본만 변경하므로 호출한 변수는 바뀌지 않는다.

포인터를 전달하면 객체의 주소를 통해 원래 값을 변경할 수 있다.

```
void IncreasePointer(int* value)
{
    if (value != nullptr)
    {
        ++(*value);
    }
}
```

참조를 전달하면 포인터 연산 없이 호출한 객체를 직접 사용하는 형태로 작성할 수 있다.

```
void IncreaseReference(int& value)
{
    ++value;
}
```

<값, 포인터와 참조 전달 호출>

```
int first = 10;
int second = 10;
int third = 10;

IncreaseValue(first);
IncreasePointer(&second);
IncreaseReference(third);
```

first는 10을 유지하고 second와 third는 11이 된다.

참조 매개변수는 함수가 전달받은 객체를 직접 변경할 수 있다는 뜻을 가진다. 널 값을 허용하거나 호출할 대상을 선택적으로 전달해야 한다면 포인터가 더 적합하다.

<값, 포인터, 참조 전달>

```cpp
#include <iostream>
using std::cout;

void IncreaseValue(int value)
{
    ++value;
}
void IncreasePointer(int* value)
{
    if (value != nullptr)
    {
        ++(*value);
    }
}
void IncreaseReference(int& value)
{
    ++value;
}

int main()
{
    int first = 10;
    int second = 10;
    int third = 10;
    IncreaseValue(first);
    IncreasePointer(&second);
    IncreaseReference(third);
    cout << "값 전달    : " << first << '\n';
    cout << "포인터 전달: " << second << '\n';
    cout << "참조 전달  : " << third << '\n';
}
```

앞에서는 하나의 값을 증가시키는 함수를 이용해 값, 포인터와 참조 전달의 기본 동작을 살펴보았다. 이번에는 두 정수의 값을 교환하는 함수를 이용해 세 전달 방식의 함수 코드, 호출 방법과 원본 객체의 변경 여부를 정리한다.

<값, 주소, 참조 전달 비교>

구분	값 전달 (Call by Value)	주소 전달 (Call by Address)	참조 전달 (Call by Reference)

함수 코드	`void Swap(int a, int b)`
	`{`
	`  int t = a;`
	`  a = b;`
	`  b = t;`
	`}`
		`void Swap(int* a, int* b)`
		`{`
		`  int t = *a;`
		`  *a = *b;`
		`  *b = t;`
		`}`
			`void Swap(int& a, int& b)`
			`{`
			`  int t = a;`
			`  a = b;`
			`  b = t;`
			`}`

함수 호출	Swap(x, y);	Swap(&x, &y);	Swap(x, y);
전달되는 대상	인수의 값	객체의 주소	원본 객체에 대한 참조
원본 변경	변경되지 않음	변경됨	변경됨
매개변수 사용	일반 변수와 같음	*로 역참조	일반 변수와 같음

2.10.3 상수 참조(lvalue reference to const)

const를 지정한 참조는 참조를 통해 원본 객체의 값을 변경할 수 없다. 비상수 객체를 상수 참조로 연결해도 원본 객체 자체가 상수가 되는 것은 아니다.

```cpp
int value = 10;
const int& reference = value;

value = 20;          // 가능
// reference = 30;   // 오류
const int& refInt2 = 100; // 임시 객체 참조
```

읽기만 필요한 객체를 함수에 전달할 때는 상수 참조를 사용할 수 있다.

```cpp
#include <string>

void Print(const std::string& text)
{
    std::cout << text << '\n';
}
```

상수 참조는 객체를 복사하지 않으면서 참조를 통해 값을 변경하지 못하게 제한한다.

```cpp
void Print(const std::string& text)
{
    // text.clear(); // 오류
    std::cout << text << '\n';
}
```

int, double처럼 복사 비용이 작은 기본형은 값으로 전달하는 편이 단순하다.

```cpp
void PrintLevel(int level);
```

문자열이나 컨테이너처럼 복사 비용이 큰 객체를 읽기만 한다면 상수 참조를 사용할 수 있다.

```cpp
void PrintItems(const std::vector<int>& items);
```

상수 참조에는 상수 객체와 비상수 객체를 모두 전달할 수 있다.

```cpp
std::string first = "Hello";
const std::string second = "C++";

Print(first);
Print(second);
```

상수 참조는 임시 객체에도 연결할 수 있다.

```cpp
const std::string& text = std::string("Hello");
```

이처럼 지역 상수 참조가 임시 객체에 직접 연결되면 임시 객체의 수명은 참조의 수명까지 연장된다.

2.10.4 참조 반환

함수는 참조를 반환할 수 있다.

```cpp
int& GetElement(int* values, int index)
{
    return values[index];
}
```

반환된 참조는 원래 배열의 원소를 나타내므로 값을 읽거나 변경할 수 있다.

```cpp
int values[3] = { 10, 20, 30 };
GetElement(values, 1) = 100;
```

이후 values[1]은 100이 된다.
참조를 반환할 때는 참조 대상이 함수 호출이 끝난 뒤에도 살아 있어야 한다.

```cpp
int& CreateValue()
{
    int value = 10;
    return value; // 함수 종료 후 value의 수명이 끝나므로 댕글링
참조 발생
}
```

지역 변수 value는 함수가 끝날 때 소멸한다. 이 함수가 반환한 참조는 수명이 끝난 객체를 가리키는 댕글링 참조가 된다.

배열이나 컨테이너의 원소, 호출한 객체의 멤버처럼 함수 호출 이후에도 존재하는 객체는 참조로 반환할 수 있다. 그러나 참조를 통해 외부에서 원본을 변경할 수 있으므로 함수가 변경 권한까지 제공해야 하는지 고려해야 한다.

읽기만 허용하려면 상수 참조를 반환할 수 있다.

```
const int& GetElement(const int* values, int index)
{
    return values[index];
}
```

<값, 주소, 참조 반환 비교>

구분	값 반환	주소 반환	참조 반환

함수 코드	`int GetElm(int* arr, int n)`
	`{`
	`  return arr[n];`
	`}`
		`int* GetElm(int* arr, int n)`
		`{`
		`  return &arr[n];`
		`}`
			`int& GetElm(int* arr, int n)`
			`{`
			`  return arr[n];`
			`}`

반환 대상	값의 복사본	객체의 주소	원본 객체에 대한 참조
사용	`int value=GetElm(values,1);`	`*GetElm(values,1)=100;`	`GetElm(values,1)=100;`
원본 변경	변경 불가	변경 가능	변경 가능
수명 조건	독립된 값, 원본 수명과 무관	사용 시점까지 대상 객체 생존 필수	사용 시점까지 대상 객체 생존 필수
널 표현	해당 없음	nullptr 반환 가능	널 참조 불가
주의 사항	큰 객체는 복사 가능성 고려	유효한 객체 주소 반환	사용 시점까지 대상 객체 생존 필수

C++에서 값 반환은 복사만을 의미하지 않는다. 모던 C++에서는 복사 생략과 이동을 활용해 불필요한 객체 복사를 줄일 수 있다. C에는 이동 의미론이 없으므로 객체를 값으로 반환할 때 복사 비용을 고려해야 하지만, C++은 이동을 통해 이러한 비용을 줄일 수 있다.

우측값 참조(T&&)는 이동 의미론을 지원하는 데 사용된다. 자세한 내용은 복사와 이동을 다루는 단원에서 살펴본다.

2.11 학습 정리

이름 공간은 함수, 타입과 변수의 이름을 범위별로 구분하여 충돌을 줄인다. 이름 공간은 여러 위치에서 다시 열 수 있고, namespace A::B::C 형식으로 여러 단계의 중첩 이름 공간을 정의할 수 있다. 개별 using 선언, 이름 공간 별칭, 이름 없는 이름 공간과 전역 범위 지정도 목적에 맞게 사용한다. using namespace는 이름 검색 범위를 넓히므로 특히 헤더 파일에서는 사용하지 않는다.

출력 스트림은 값의 타입에 맞는 출력 동작을 선택한다. 서식 조정자를 이용해 진법, 접두사, 대소문자, 부호, 필드 폭, 정렬과 부동소수점 형식을 지정할 수 있다. 표준 스트림에는 2진수 서식 조정자가 없으므로 std::bitset이나 C++20의 std::format을 사용할 수 있다. std::setw()는 다음 값 하나에만 적용되지만 대부분의 서식 설정은 다시 변경할 때까지 유지된다.

입력 스트림은 입력한 데이터가 변수의 타입에 맞지 않으면 실패 상태가 된다. 입력 연산 자체를 조건식으로 검사하고, 복구할 때는 clear()로 상태를 해제한 뒤 ignore()로 잘못된 입력을 제거한다. cin >>와 getline()을 함께 사용할 때는 남아 있는 줄 바꿈을 처리해야 한다. 문자 단위 입력에는 get(), peek(), unget() 등을 사용할 수 있다.

기본형의 크기와 정렬은 실행 환경에 따라 달라질 수 있다. sizeof, alignof, alignas, CHAR_BIT, std::numeric_limits로 현재 환경의 정보를 확인한다. 배열의 원소 수와 메모리 크기는 std::size_t, 같은 배열 안의 포인터 위치 차이는 std::ptrdiff_t로 표현하며 C++20에서는 std::ssize()로 부호 있는 원소 수를 얻을 수 있다.

문자형은 문자 인코딩의 코드 단위를 저장한다. 코드 포인트, 코드 단위와 화면에 표시되는 문자 수는 서로 다를 수 있다. UTF-8, UTF-16과 UTF-32 문자열 배열의 마지막에는 널 문자가 포함되며, std::size()로 전체 코드 단위 수를 확인할 수 있다.

부동소수점형은 대부분의 실수값을 근사하여 저장한다. 연산 결과를 비교할 때는 계산의 목적과 값의 범위에 맞는 절대 오차와 상대 오차를 사용할 수 있다. 머신 입실론은 타입의 정밀도를 나타내는 기준이지만 모든 비교에 그대로 사용할 수 있는 범용 허용 오차는 아니다. 무한대와 NaN은 std::isfinite(), std::isinf(), std::isnan()으로 확인한다.

초기화는 객체가 만들어질 때 값을 정하는 과정이고 대입은 이미 존재하는 객체의 값을 변경하는 과정이다. 중괄호 초기화는 일부 축소 변환을 방지한다. const는 변경 제한, constexpr는 상수 표현식, consteval은 반드시 컴파일 시점에 계산되는 함수, constinit은 정적 저장 기간 객체의 정적 초기화를 나타낸다. static_assert는 컴파일 시점 조건을 검사한다.

C++ 형 변환 연산자는 변환의 목적을 코드에 드러낸다. enum class는 열거자 이름의 범위와 정수형 변환을 제한한다. auto, decltype, decltype(auto)는 정적 타입을 유지하면서 타입을 추론하거나 표현식의 타입을 재사용한다. using 타입 별칭은 기존 타입에 다른 이름을 제공한다.

함수 오버로딩과 기본 인수는 같은 역할의 함수를 일관된 이름으로 구성하고 호출을 간결하게 만든다. inline은 최적화를 강제하지 않으며 동일한 정의를 여러 번역 단위에 둘 수 있게 한다. 함수 반환 타입은 auto와 후행 반환 타입으로 표현할 수 있다.

참조는 기존 객체의 다른 이름이다. 함수의 인수 전달 방식은 값에 의한 전달(call by value), 주소에 의한 전달(call by address), 참조에 의한 전달(call by reference)로 구분할 수 있다. 값 전달은 인수의 복사본을 만들므로 원본이 변경되지 않는다. 주소 전달은 객체의 주소를 포인터로 전달하고 역참조하여 원본에 접근하며, 참조 전달은 참조 매개변수를 통해 원본에 직접 접근한다.

널 상태가 필요하면 포인터가 적합하고, 복사하지 않고 읽기만 할 객체는 상수 참조로 전달할 수 있다. C++의 값 반환은 복사 생략과 이동을 통해 불필요한 객체 복사를 줄일 수 있다. 참조 반환은 원본 객체를 직접 사용할 수 있게 하지만, 대상 객체의 수명을 보장해야 한다.

2.12 학습 과제

과제 **1.** 자료형별 데이터 분석기
함수 오버로딩을 이용해 정수, 실수와 문자열을 분석하는 프로그램을 작성한다.
사용자는 분석할 자료형을 선택하고 값을 입력한다. 같은 이름의 Analyze() 함수를 다음과 같이 오버로딩한다.

```
void Analyze(int value);
void Analyze(double value);
void Analyze(const std::string& value);
```

정수형은 10진수, 16진수와 2진수로 출력하고 짝수인지 홀수인지 판단한다. 실수형은 소수점 이하 자릿수를 지정하여 출력하고 정수 부분과 소수 부분을 구분한다. 문자열은 길이, 빈 문자열 여부, 영문자 수, 숫자 수와 공백 수를 출력한다.

과제 **2. URL** 문자열 분석기
URL 한 줄을 입력받아 구성 요소를 분리하여 출력하는 프로그램을 작성한다.
입력 URL의 형식은 다음과 같다.

```
https://www.example.com:8080/api/user?id=10
```

다음 항목을 분리한다.

```
프로토콜 : https
호스트   : www.example.com
포트     : 8080
경로     : /api/user
질의문   : id=10
```

std::string의 find(), substr(), size(), empty()를 이용한다. 포트와 질의문이 없는 URL도 처리한다.

과제 **3.** 비밀번호 안전성 검사기
사용자가 입력한 비밀번호의 안전성을 검사하는 프로그램을 작성한다.
비밀번호는 std::string으로 입력받고 다음 조건을 검사한다.

- 길이가 8자 이상인지 확인한다.
- 영문 대문자가 포함되어 있는지 확인한다.
- 영문 소문자가 포함되어 있는지 확인한다.
- 숫자가 포함되어 있는지 확인한다.
- 특수 문자가 포함되어 있는지 확인한다.
- 공백이 포함되어 있으면 사용할 수 없는 비밀번호로 처리한다.

만족한 조건의 수에 따라 약함, 보통과 강함으로 구분하고 만족하지 못한 조건을 함께 출력한다.

```
int CalculatePasswordScore(const std::string& password);
void PrintPasswordResult(const std::string& password, int score);
```

과제 **4.** 프로세스 메모리 사용량 보고서
실행 중인 프로그램의 정보를 입력받아 메모리 사용량 순으로 출력하는 모의 작업 관리자 프로그램을 작성한다.
프로세스는 이름, 프로세스 번호와 메모리 사용량을 가진다.

```
struct Process
{
    std::string name;
    int pid;
    int memory;
};
```

프로세스 수는 5개에서 10개 사이로 입력받고 구조체 배열을 동적 할당한다. 프로그램 이름은 getline()으로 입력받으며, 프로세스 번호와 메모리 사용량은 정수로 입력받는다.
메모리 사용량을 기준으로 내림차순 정렬하고 다음 내용을 출력한다.

- 프로세스 이름
- 프로세스 번호
- 메모리 사용량
- 전체 메모리 사용량
- 평균 메모리 사용량
- 메모리를 가장 많이 사용하는 프로세스

과제 **5.** 소프트웨어 버전 비교기
두 소프트웨어의 버전 문자열을 입력받아 어느 버전이 더 최신인지 판단하는 프로그램을 작성한다.
버전 문자열은 다음 형식으로 입력한다.

```
1.12.3
1.9.15
```

점 .을 기준으로 주 버전, 부 버전과 패치 버전을 분리한다.

```
struct Version
{
    int major;
    int minor;
    int patch;
};
```

주 버전부터 차례대로 값을 비교하여 최신 버전을 판단한다.

```
1.12.3이 1.9.15보다 최신 버전입니다.
```

잘못된 형식도 검사한다.

```
1.2
1.2.3.4
1.a.3
```